

Sistemas Embarcados I - UFU

Daniel P. Carvalho

Contents

Apresentação	7
Arquitetura e plataformas para sistemas embarcados	9
A arquitetura ARM Cortex-M	10
O que você precisa para começar	11
Placas de desenvolvimento	11
Adaptadores de depuração	13
Bibliografia e Literatura de Referência	16
Livro-Texto Oficial	16
Documentação Técnica de Fabricante (STMicroelectronics)	16
Linguagem C e Ferramentas de Desenvolvimento	16

I

Semana 1 – Arquitetura de Computadores

1	Fundamentos de Arquitetura de Computadores	21
1.1	O que é Arquitetura de Computadores?	21
1.1.1	O que define uma ISA?	21
1.2	O Modelo de Von Neumann	22
1.2.1	O Ciclo Fetch-Decode-Execute	25
1.2.2	O Gargalo de Von Neumann	26
1.3	Arquitetura Harvard	26
1.4	CISC vs. RISC	27
1.4.1	A Regra Load-Store	27
1.4.2	Por que isso importa para processadores modernos?	28
1.5	Memória e Registradores	28
1.5.1	Hierarquia de Memória	28
1.5.2	Memória Volátil vs. Não-Volátil	28
1.5.3	Tecnologias de memória não-volátil	29
1.5.4	Registradores	30
1.5.5	A Pilha (Stack)	30
1.5.6	Modos de Endereçamento	31
1.6	Exemplos de Arquiteturas de Conjunto de Instruções	31
1.7	Questionário de revisão	33
1.8	Referências	34
2	Prática: Fundamentos de Linux	35
2.1	Por que terminal antes de interface gráfica?	35
2.2	Um pouco de história	35
2.3	Sistemas de arquivos: Windows vs. Linux	35
2.3.1	Caminhos absolutos e relativos	36
2.4	Abrindo o terminal e o primeiro comando	36
2.5	Anatomia de um comando	37

2.6	Navegação básica	37
2.6.1	Repetindo comandos com <code>history</code>	38
2.7	Criando, movendo e removendo arquivos	38
2.7.1	Wildcards (curingas)	39
2.8	Editores de texto no terminal	40
2.8.1	<code>nano</code> : simples e direto	40
2.8.2	<code>vim</code> : onipresente, porém exigente	40
2.9	Buscando arquivos e conteúdo	41
2.9.1	<code>find</code> : buscando arquivos pela árvore	41
2.9.2	<code>grep</code> : buscando texto dentro dos arquivos	41
2.10	Superusuário e gerenciamento de pacotes	41
2.11	Tipos de arquivo e permissões	42
2.11.1	Arquivos ocultos	42
2.12	Compondo comandos com pipes	42
2.13	Redirecionamento e encadeamento de comandos	43
2.13.1	Redirecionando a saída para um arquivo	43
2.13.2	Saída padrão, saída de erro e <code>2>&1</code>	43
2.13.3	Encadeando comandos com <code>;</code> e o código de saída <code>\$?</code>	44
2.14	Filtros de texto	44
2.14.1	<code>cat</code> — exibir o conteúdo inteiro	44
2.14.2	<code>head</code> e <code>tail</code> — ver o início ou o final	44
2.14.3	<code>wc</code> — contar linhas, palavras e bytes	45
2.14.4	<code>tr</code> — substituir ou eliminar caracteres	45
2.14.5	<code>diff</code> — comparar dois arquivos	45
2.15	Identificando e inspecionando arquivos	45
2.15.1	<code>file</code> — descobrir o tipo real de um arquivo	45
2.15.2	<code>od</code> — inspecionar o conteúdo binário em hexadecimal	46
2.16	Variáveis de ambiente e o <code>PATH</code>	46
2.16.1	Consultando variáveis de ambiente	47
2.16.2	O <code>PATH</code> : onde o <i>shell</i> encontra os programas	47
2.16.3	Criando e exportando variáveis	47
2.17	Onde isso se aplica no curso	48
2.18	Leitura complementar	48
2.19	Questionário de revisão	49

II

Semana 2 — Arquitetura ARM Cortex-M e Fluxo de Desenvolvimento

3	Introdução aos Processadores ARM Cortex-M	53
3.1	A ARM e a evolução da sua arquitetura	54
3.1.1	ARMv4 e o conjunto de instruções Thumb	54
3.1.2	ARMv5 e ARMv6: interoperabilidade e Thumb-2	55
3.1.3	ARMv7 e os perfis A, R e M	55
3.2	Processador, núcleo e microcontrolador	56
3.3	Harvard ou Von Neumann?	56
3.4	RISC ou CISC?	57
3.5	Os conjuntos de instruções A32, T16 e T32	57
3.5.1	Modos de endereçamento concretos no ARMv7-M	58

3.6	O modelo de programação	60
3.6.1	Modos de operação, estados de execução e níveis de acesso	60
3.6.2	O banco de registradores	61
3.6.3	Registradores especiais	62
3.6.4	Registradores da unidade de ponto flutuante	65
3.7	O sistema de memória	65
3.7.1	Um espaço de endereçamento de 4 GB	65
3.7.2	Atributos arquiteturais de memória: Normal vs. Device Memory	66
3.7.3	A pilha	67
3.8	Exceções e interrupções	67
3.8.1	O NVIC	69
3.8.2	A tabela de vetores	69
3.9	Depuração: JTAG e SWD	70
3.10	A sequência de reset	70
3.11	Questionário de revisão	70
3.12	Referências	71
4	Prática: Do C ao Binário	73
4.1	Do código-fonte ao executável	74
4.1.1	As ferramentas do <i>toolchain</i>	76
4.2	Parte 1 — Quanto vale <code>g_a + g_b</code>?	76
4.3	Parte 2 — E no Cortex-M4?	77
4.3.1	Lendo a desmontagem	79
4.3.2	A regra Load-Store: x86 × ARM	80
4.3.3	A convenção de chamada	81
4.3.4	Instruções de 16 e de 32 bits	81
4.3.5	O que o <i>linker</i> fez, e o que ainda falta	82
4.3.6	Caminhando para o hardware: Ponteiros e Extensão de Sinal	82
4.4	Referências	84

III

Semana 3 — O Microcontrolador STM32F411

5	Os Microcontroladores STM32 e a Série F4	87
5.1	Do núcleo ao microcontrolador	87
5.2	A linha STM32	88
5.2.1	Decifrando o código de uma peça	88
5.3	A documentação de um STM32	89
5.4	A série STM32F4	89
5.4.1	O STM32F411	90
5.5	Os barramentos do STM32F411	90
5.6	O mapa de memória do STM32F411	91
5.7	As memórias do STM32F411	92
5.7.1	Gravar a Flash não é como escrever na SRAM	92
5.8	A organização da Flash e a sua proteção	93
5.8.1	Setores	93
5.8.2	<i>Option bytes</i> e proteção	93
5.9	De onde o processador parte: boot e espelhamento	94
5.10	Reset	95

5.11	Alimentação	95
5.11.1	Domínios de alimentação	95
5.11.2	Escalas de tensão e a frequência máxima	96
5.11.3	Supervisão da alimentação	96
5.11.4	Modos de baixo consumo	96
5.12	O sistema de <i>clock</i>	97
5.12.1	As fontes	97
5.12.2	O PLL	97
5.12.3	O <i>clock</i> do sistema e os <i>prescalers</i>	98
5.12.4	A Flash não acompanha o processador	98
5.12.5	Executar código da SRAM	99
5.13	Os registradores que encontraremos primeiro	100
5.14	Do mapa de memória à organização do programa	100
5.14.1	Como um programa ocupa a memória	100
5.14.2	As duas memórias do STM32F411	101
5.14.3	Onde cada seção deveria morar	101
5.14.4	Segmentação da SRAM: pilha, <i>heap</i> e a falta de proteção	102
5.14.5	A distribuição em um executável real	103
5.15	Questionário de revisão	103
5.16	Referências	104
6	Prática: Código de Inicialização e <i>Linker Script</i>	105
6.1	Preparando a pasta	105
6.2	O que precisa acontecer antes do <code>main()</code>	106
6.3	Parte 1 — O código de inicialização	107
6.3.1	Passo a passo do <code>startup.c</code>	107
6.4	Parte 2 — O <i>linker script</i>	117
6.4.1	Antes de digitar: o que é um <i>linker script</i>	117
6.4.2	Passo a passo do <code>stm32f411-rom.ld</code>	117
6.4.3	Ligando o programa	120
6.4.4	Conferindo a imagem	121
6.5	Juntando as peças	122
6.6	Referências	123

Apresentação

Bem-vindo à disciplina de Sistemas Embarcados I. Diferente de um computador de propósito geral — como um desktop ou laptop, projetado para executar uma infinidade de tarefas, desde edição de vídeo até navegação na web —, um **Sistema Embarcado** (*Embedded System*) é um sistema computacional micro-processado projetado para realizar uma função **dedicada ou específica**, dentro de um sistema maior. O termo “embarcado” refere-se justamente a isso: o sistema de processamento está **embutido** dentro de um dispositivo maior, muitas vezes como parte de um sistema mecânico ou elétrico — o “cérebro” invisível de produtos que usamos todos os dias, sem que percebamos que existe um computador ali dentro.



Figure 0.1: Sistema Embarcado: A união de Hardware e Firmware

Quatro características tendem a marcar esse tipo de sistema, e nenhuma delas é uma preocupação central no design de um notebook ou smartphone — lá, o que importa é desempenho bruto e experiência de uso. O **custo** costuma ser o primeiro fator: produzido em larga escala, um sistema embarcado precisa manter uma margem por unidade muito mais apertada que a de um computador pessoal, então cada componente extra pesa no orçamento. A **eficiência energética** vem em seguida — muitos desses sistemas operam por bateria, ou sob restrições rígidas de dissipação térmica, sem cooler nem ventilação ativa. Já o **determinismo** é o que mais diferencia os dois mundos: um sistema embarcado precisa responder a eventos externos dentro de prazos estritos e previsíveis — um airbag que dispara alguns milissegundos atrasado não serve pra nada, mas um clique de mouse que demora um pouco mais passa despercebido. Por fim, a **robustez**: esses sistemas operam em ambientes variados (vibração, temperatura, umidade, ruído elétrico), muitas vezes de forma ininterrupta por anos, sem reinicialização — algo que nenhum notebook precisa aguentar. Em um sistema embarcado, a otimização é outra: fazer exatamente o necessário, com o menor custo, consumo e risco de falha possíveis.

Sistemas embarcados estão presentes em praticamente toda a tecnologia moderna — a estimativa comum na indústria é que, para cada microprocessador de propósito geral vendido (o tipo que vai dentro de um PC), existem **dezenas** de microcontroladores embarcados, a grande maioria deles totalmente invisível para quem usa o produto final:

Imagem: Saúde

Saúde

Marcapassos, monitores cardíacos, bombas de infusão.

Imagem: Automotivo

Automotivo

Injeção eletrônica, freios ABS, airbags.

Imagem: Doméstico

Doméstico

Micro-ondas, máquinas de lavar, smart TVs.

Imagem: Industrial

Industrial

CLPs, sensores inteligentes.

Essa onipresença é o pano de fundo desta disciplina: diferente de um PC, um sistema embarcado frequentemente não tem um sistema operacional completo, executa uma função fixa definida em projeto, opera com recursos de memória e processamento muito mais limitados, e tem um tempo de resposta que costuma ser crítico — “rápido o suficiente” raramente basta. Esse contraste é o ponto de partida de todo o curso: você já sabe programar para o primeiro tipo de sistema (com Programação Procedimental em C); agora vamos aprender a projetar para o segundo, onde cada byte de memória e cada ciclo de clock precisam ser levados em conta, e onde não existe um sistema operacional escondendo o hardware por baixo dos panos.

O grande diferencial desta disciplina está na abordagem sistêmica: você aprenderá a projetar o **hardware** — incluindo esquemas eletrônicos e o design de PCBs — em conjunto com o desenvolvimento do **firmware**, elaborando o código de baixo nível que dá vida ao sistema. Dessa forma, você compreenderá a essência dessas tecnologias, desde seus fundamentos físicos e lógicos até a implementação de soluções robustas em dispositivos de alta complexidade.

Para alcançar esse nível de proficiência, utilizaremos a metodologia **Bottom-Up** (de baixo para cima). Nesta metodologia, o aprendizado é construído em etapas evolutivas, onde cada fase resolve um desafio técnico real e introduz um novo nível de sofisticação. Nesta abordagem, **não utilizamos “caixas pretas”**. Em vez disso, cada componente do sistema é compreendido em sua totalidade, desde os circuitos eletrônicos e o design de hardware até a programação de baixo nível e a integração de sistemas. A cada etapa, você enfrentará desafios técnicos reais, como a implementação de um driver para um periférico específico ou a otimização do consumo de energia, garantindo que o aprendizado seja prático, relevante e alinhado com as demandas do mercado.

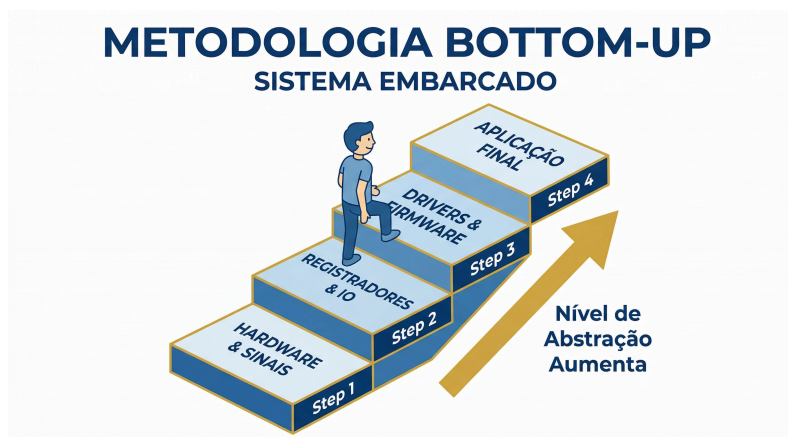


Figure 0.2: Metodologia Bottom-Up: Subindo os degraus da abstração

Frameworks de abstração como o **Arduino**, a **HAL (Hardware Abstraction Layer)** da ST ou o **ESP-IDF** da Espressif são ferramentas legítimas e amplamente utilizadas na indústria e na prototipagem — inclusive por engenheiros experientes, quando produtividade importa mais do que controle fino. Todos eles aceleram o desenvolvimento ao encapsular a complexidade do hardware em funções de alto nível, como `delay()`, `HAL_GPIO_WritePin()` ou os drivers do IDF, que mascaram o funcionamento real de timers e registradores. Nesta disciplina, no entanto, a ordem é invertida: nosso foco é a transparência total desde a base. Aqui, você não utilizará funções de temporização ou bibliotecas prontas sem antes dominar o hardware e o firmware que as sustentam, compreendendo exatamente como os periféricos interagem com o núcleo do processador — para que, se um dia optar por usar essas abstrações, seja por escolha consciente, e não por depender de uma caixa preta.

Essa metodologia permite vivenciar os desafios reais da engenharia, como a gestão otimizada de recursos limitados, a depuração em nível de hardware e a criação de interfaces que unem eficiência e manutenibilidade. Ao escalar cada camada da arquitetura, você compreenderá a fundamentação técnica por trás de cada decisão de design, transformando a complexidade em um processo de desenvolvimento lógico, estruturado e profissional.

Arquitetura e plataformas para sistemas embarcados

É bem provável que você já tenha alguma experiência com **Arduino** ou **ESP32** — são, hoje, a porta de entrada mais comum para eletrônica e programação de microcontroladores, e por bons motivos: são baratos, acessíveis e têm um ecossistema gigante de bibliotecas prontas. Mas eles são só dois pontos em um universo bem mais amplo de arquiteturas disputando esse espaço:

Imagem: Microcontroladores de 8 bits (AVR/PIC)

Microcontroladores de 8 bits

A família **AVR** (o chip por trás do Arduino Uno clássico) e o **PIC** da Microchip. Simples, baratos, com décadas de mercado consolidado, mas limitados em desempenho, memória e periféricos para aplicações mais exigentes.

Imagem: RISC-V

RISC-V

Uma arquitetura de conjunto de instruções *aberta* (sem custo de licenciamento), crescendo rápido na indústria, mas ainda com ecossistema de ferramentas e depuração menos maduro que os concorrentes estabelecidos.

Imagem: Xtensa (ESP32)

Xtensa

O núcleo proprietário da Espressif, usado no **ESP32**. Seu grande diferencial é a conectividade Wi-Fi/BLE já integrada ao chip, o que o tornou o queridinho de projetos de IoT.

Imagem: ARM Cortex-M

ARM Cortex-M

A linha de núcleos ARM desenhada especificamente para microcontroladores (diferente do Cortex-A, que equipa smartphones e computadores rodando sistemas operacionais completos).

Colocando essas quatro arquiteturas lado a lado, dá pra enxergar um espectro entre dois polos: de um lado, **controle de baixo nível** — acesso direto a registradores, sem nenhuma camada de abstração imposta entre você e o hardware; do outro, **ecossistema pronto e facilidade de uso** — bibliotecas, frameworks e ferramentas que reduzem o esforço inicial, ao custo de esconder parte do hardware por baixo. Os microcontroladores de 8 bits (AVR/PIC) e o RISC-V ficam mais próximos do polo de controle: pouco ou nenhum framework pronto, você lida direto com o silício. O Xtensa do ESP32, puxado pelo framework Arduino-ESP32/ESP-IDF, fica mais perto do polo oposto — conectividade e bibliotecas prontas, mas boa parte do hardware abstraída. O **ARM Cortex-M** ocupa uma posição intermediária particularmente favorável: mantém o acesso total a registradores, sem camada obrigatória entre você e o hardware, mas com um ecossistema de ferramentas de desenvolvimento, depuração e fabricantes maduro o bastante para sustentar produtos comerciais sérios.

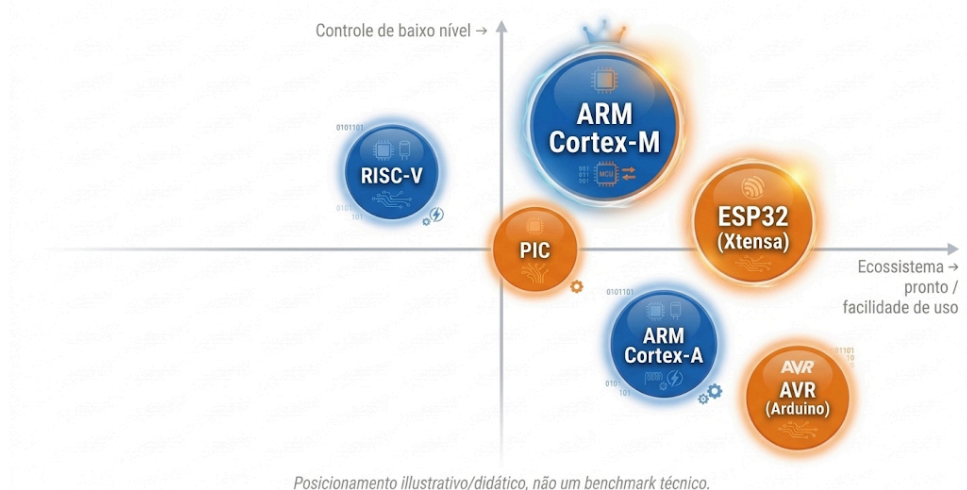


Figure 0.3: Panorama de plataformas para sistemas embarcados: posicionamento ilustrativo entre controle de baixo nível e ecossistema pronto/facilidade de uso.

A arquitetura ARM Cortex-M

É justamente essa posição intermediária que justifica a escolha do curso. A **ARM** não é um chip — é uma *arquitetura de processador*: um conjunto de regras de design que a empresa britânica ARM Holdings licencia para fabricantes de semicondutores, que então produzem seus próprios chips seguindo essas regras. É por isso que STMicroelectronics, NXP, Texas Instruments e Renesas competem entre si vendendo chips completamente diferentes, mas todos com núcleo ARM por dentro. Dentro da família ARM, o **Cortex-M** é a linha voltada a microcontroladores: chips pequenos, de baixo consumo, que rodam sem sistema operacional e controlam diretamente o hardware. Um **STM32**, por exemplo — a família de chips que utilizaremos no curso — é um microcontrolador da ST construído sobre um núcleo Cortex-M.

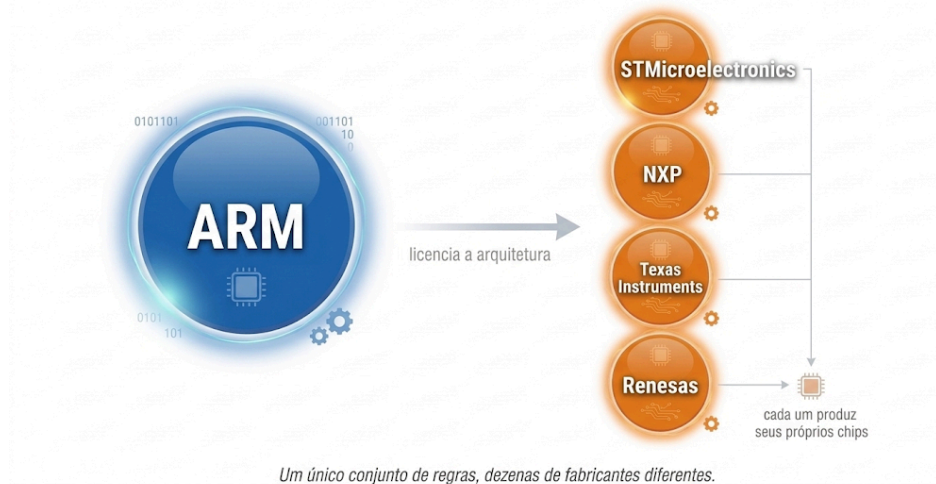


Figure 0.4: Um único conjunto de regras, dezenas de fabricantes diferentes: a ARM licencia sua arquitetura para diversos fabricantes de semicondutores.

Este curso é fundamentado nessa arquitetura, o padrão *de facto* na indústria global de tecnologia. A escolha não é apenas uma decisão pedagógica, mas um alinhamento estratégico com as demandas reais do mercado de engenharia:

- **Liderança Absoluta de Mercado:** Com um ecossistema que movimenta mais de **30 bilhões de chips anualmente**, a arquitetura ARM sustenta cerca de **90% dos dispositivos móveis do mundo** e é a espinha dorsal da eletrônica automotiva e industrial moderna. Estudar ARM é aprender a tecnologia que move o planeta.
- **Controle Profissional vs. Prototipagem (ARM vs. Arduino/AVR):** o Arduino é uma excelente porta de entrada para o hobby, mas o ARM Cortex-M oferece os recursos exigidos em produtos comerciais de alta performance. Aqui, exploramos o controle total sobre o hardware através do acesso direto a **registradores, DMA e gerenciamento avançado de energia**, competências essenciais para o desenvolvimento de produtos otimizados e escaláveis — e que o Arduino, por design, esconde do programador.
- **Determinismo e Confiabilidade (ARM vs. ESP32):** o ESP32 é uma ferramenta poderosa para IoT, mas a linha ARM (como os STM32) oferece um ecossistema de **depuração (Debug) profissional via SWD/JTAG** incomparável. Isso permite ao engenheiro analisar o ciclo de instrução e garantir o determinismo temporal em sistemas críticos, onde falhas de milissegundos não são uma opção — algo que a pilha de conectividade do ESP32 não foi projetada para priorizar.
- **Versatilidade e Independência de Fabricante:** dominar o conjunto de instruções e a arquitetura ARM permite que você transite livremente entre os maiores fabricantes de semicondutores do mundo (**STMicroelectronics, NXP, Texas Instruments, Renesas**). Você não aprende a programar um chip específico, mas sim a arquitetura que define o padrão global da engenharia de sistemas embarcados — logo, tudo o que aprender aqui permanece válido mesmo se, amanhã, seu empregador utilizar outro fabricante.

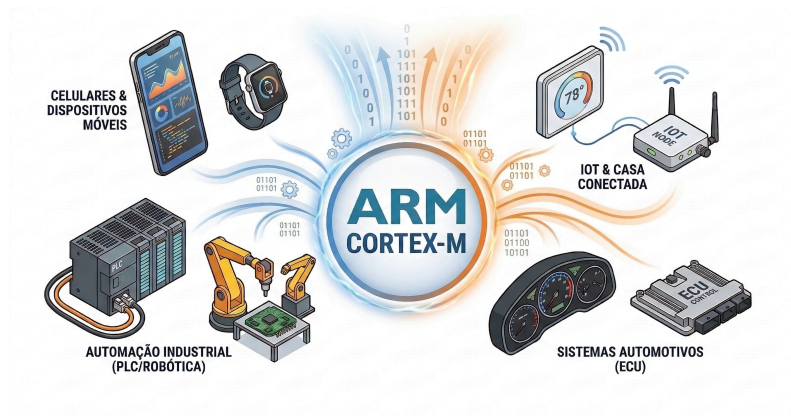


Figure 0.5: O Ecosistema ARM Cortex-M: Uma arquitetura unificada conectando dispositivos móveis, automação industrial (CLP/Robótica), sistemas automotivos (ECU) e Internet das Coisas (IoT).

O que você precisa para começar

Para desenvolver software para um microcontrolador, três peças de hardware/software são indispensáveis: uma **placa de desenvolvimento**, um **adaptador de depuração**, e o **driver de software** correspondente. Providencie o quanto antes — quanto mais cedo o kit chegar nas suas mãos, mais tempo você tem antes de precisar dele de verdade: o roteiro de Ambiente e Ferramentas já termina com a validação na placa.

Placas de desenvolvimento

Uma placa de desenvolvimento (*development board*, ou simplesmente *kit*) traz o microcontrolador já montado, junto com regulador de tensão, cristal oscilador, conector USB, e geralmente um LED e um botão para os primeiros testes — tudo que você precisa para energizar o chip e começar a gravar código, sem ter que soldar nada.

Fabricantes de microcontroladores costumam oferecer várias famílias de placas, com propósitos diferentes:

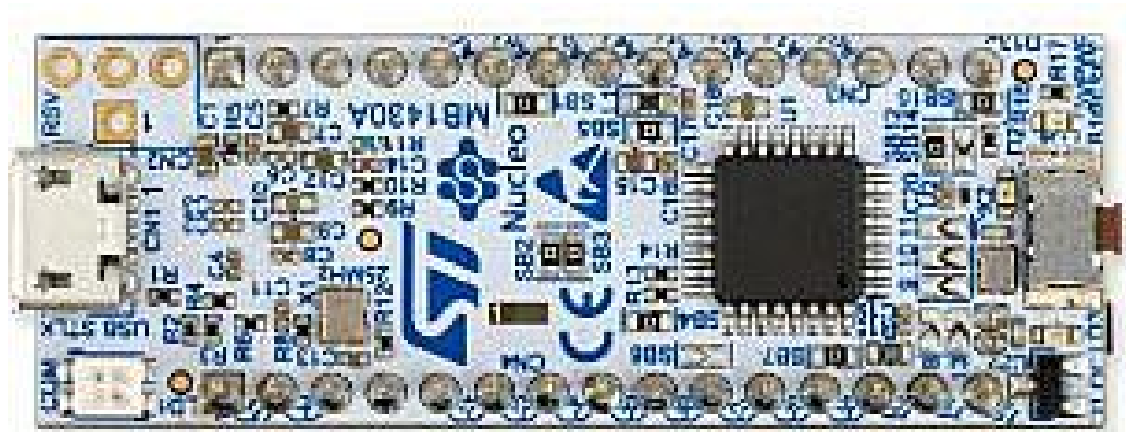


Figure 0.6: Placa STM32 Nucleo. Foto: Wikimedia Commons, CC BY-SA 4.0.

Nucleo (ST)

Linha de placas de baixo custo da ST, voltada a prototipagem rápida. Já vem com o adaptador de depuração **ST-LINK integrado** na própria placa (não precisa de um separado), e pinagem compatível com conectores Arduino (Uno) — o que facilita usar *shields* prontos. Existe uma Nucleo para praticamente cada linha da ST (F0, F1, F4, L4, G0...).

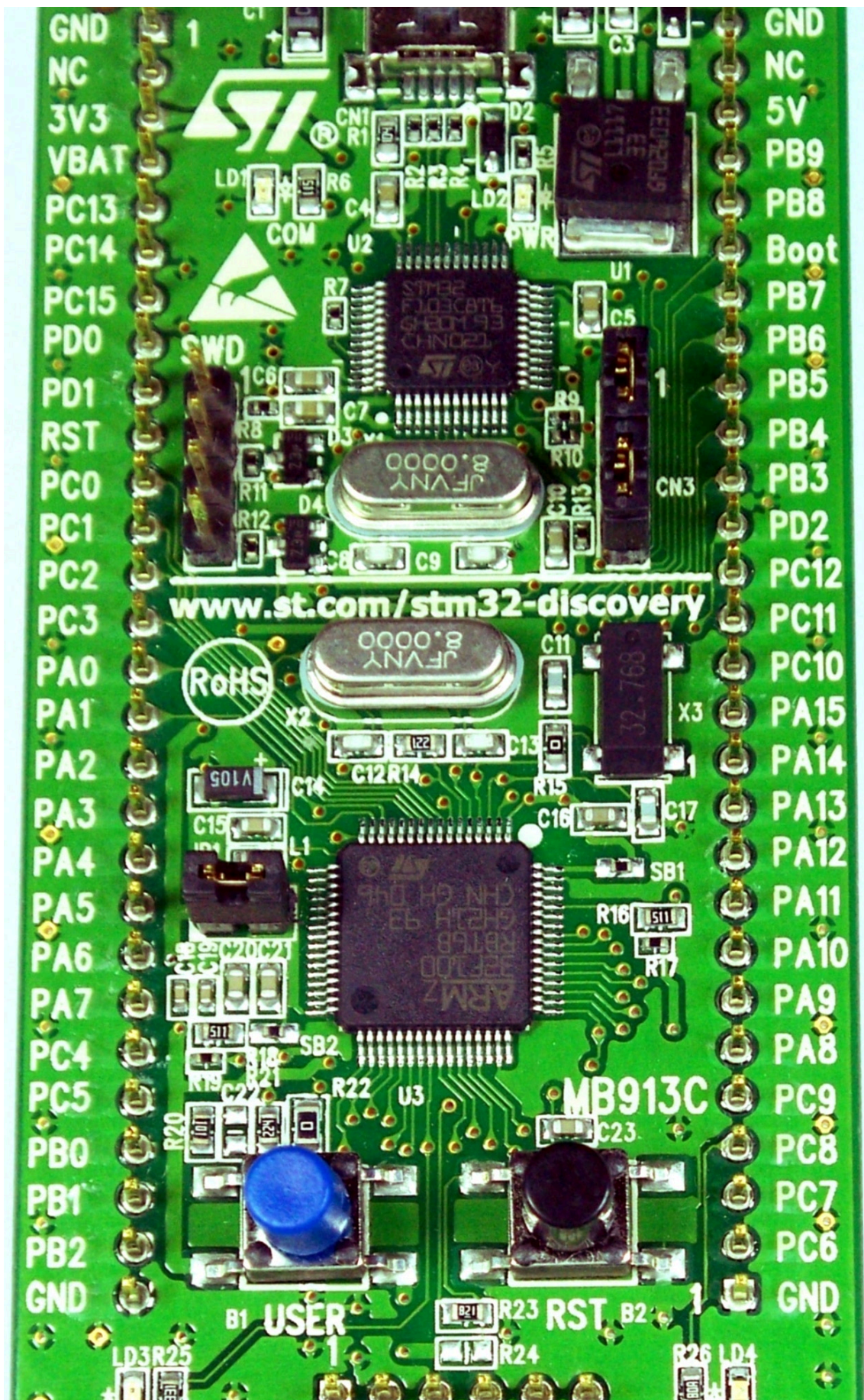


Figure 0.7: Placa STM32 Discovery. Foto: Wikimedia Commons, CC BY-SA 3.0.

Discovery (ST)

Linha voltada à demonstração de recursos específicos do chip — costuma trazer periféricos extras na própria placa (acelerômetro, display, codec de áudio), também com adaptador de depuração integrado. Mais cara que a Nucleo, mas útil quando o periférico embutido é relevante para o seu projeto.

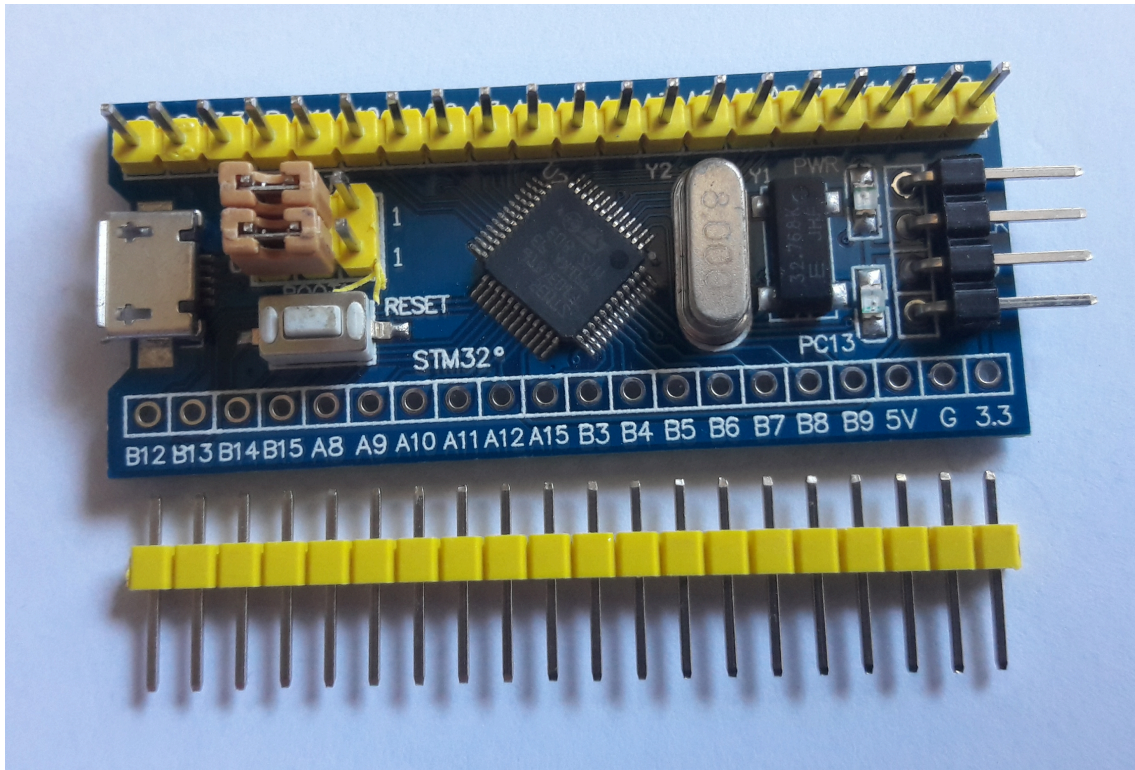


Figure 0.8: Blue Pill (STM32F103, Cortex-M3). Foto: Wikimedia Commons, CC BY-SA 4.0.

Blue Pill / Black Pill (placas “genéricas”)

Placas minimalistas, fabricadas por terceiros (não pela ST) a partir de projetos abertos — **não trazem** adaptador de depuração integrado, o que exige comprar um separado. Em compensação, custam uma fração do preço de uma Nucleo/Discovery. A **Blue Pill** usa o STM32F103 (Cortex-M3); a **Black Pill**, que usaremos neste curso, usa o STM32F411 (Cortex-M4).

Adaptadores de depuração

O adaptador de depuração (*debug adapter* ou *debug probe*) é a interface física — via protocolo **SWD** (*Serial Wire Debug*, 2 fios) ou **JTAG** (mais fios, mais antigo) — que permite ao seu computador gravar o programa na Flash do microcontrolador e inspecionar registradores/memória durante a execução.

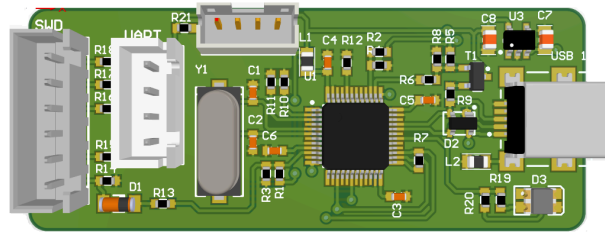


Figure 0.9: Adaptador ST-LINK/V2, versão clone (a mesma vendida em lojas como AliExpress/Mercado Livre).

ST-LINK/V2

O adaptador padrão da ST, usado nas Nucleo/Discovery (integrado) e vendido separadamente como um pequeno dongle USB — inclusive em versões **clone** (não-oficiais, fabricadas por terceiros), vendidas por poucos dólares em lojas como AliExpress e Mercado Livre. Para uma placa “genérica” como a Black Pill, que não traz debug adapter embutido, esse é o acessório mais comum e barato do mercado — e o que usaremos neste curso.



Figure 0.10: Segger J-Link EDU. Foto: Wikimedia Commons, CC BY-SA 3.0.

J-Link (Segger)

Linha profissional de adaptadores de depuração, compatível com praticamente qualquer chip ARM (não só STM32). Muito mais robusta e rápida que o ST-LINK, com suporte a recursos avançados de depuração — mas também muito mais cara: a versão profissional custa centenas de dólares. A Segger oferece uma versão **EDU**, com licença para uso educacional/não-comercial a um preço bem mais acessível. Não é necessária para este curso, mas é a ferramenta que você provavelmente vai encontrar em um ambiente profissional.

Kit oficial do curso e alternativas

Utilizaremos a **STM32F411 Black Pill** + um **adaptador ST-LINK/V2** (clone) como kit oficial do curso — a combinação mais barata e amplamente disponível para um núcleo Cortex-M4 completo.

Se você já possui outra placa, ou preferir adquirir uma diferente, **qualquer kit baseado em Cortex-M3 ou Cortex-M4** é aceito neste curso — Nucleo, Discovery, Blue Pill, ou qualquer outra. Os conceitos e o processo de desenvolvimento que vamos aprender são os mesmos, independente do kit específico — só alguns detalhes de mapa de memória e periféricos vão variar, e isso fica por sua conta ajustar consultando o *datasheet* do seu chip. Se optar por uma Nucleo/Discovery, você já tem o debug adapter embutido — não precisa comprar um ST-LINK separado.

Por fim, o **driver de software**: para o sistema operacional reconhecer o adaptador de depuração como um dispositivo USB válido. Isso é resolvido no roteiro de Ambiente e Ferramentas (seção de acesso a dispositivos USB no WSL2).

Bibliografia e Literatura de Referência

O material didático deste site foi estruturado para servir como **guia pedagógico prático e fio condutor** das aulas e projetos de bancada. Contudo, a formação sólida de um engenheiro de sistemas embarcados exige o hábito de consultar a literatura profissional e os manuais técnicos dos fabricantes de silício. A disciplina adota as seguintes referências:

Livro-Texto Oficial

[LIVRO-TEXTO]

Referência Principal

The Definitive Guide to ARM® Cortex®-M3 and Cortex®-M4 Processors

Autor: Joseph Yiu

Edição: 3ª Edição (2013) — Editora Newnes / Elsevier. ISBN: 978-0-12-408082-9 (1.055 páginas).

Considerada a “bíblia” da arquitetura Cortex-M no mundo inteiro, esta obra foi escrita por um dos engenheiros especialistas seniores da própria ARM. O livro cobre desde a microarquitetura interna, modos de operação, pipeline e sistema de memória até o conjunto de instruções Thumb-2, tratamento de exceções/interrupções (NVIC) e técnicas de desenvolvimento com ferramentas GNU (GCC, GDB).

Como utilizar: Ao longo dos capítulos desta apostila, você encontrará boxes indicando capítulos e seções específicas do livro do Joseph Yiu para leitura obrigatória de aprofundamento teórico.

Documentação Técnica de Fabricante (STMicroelectronics)

Para além do livro-texto de arquitetura, o desenvolvimento *bare-metal* exige a leitura direta da documentação técnica fornecida pelo fabricante do microcontrolador:

1. **RM0383 Reference Manual — STM32F411xC/E**: Manual de 844 páginas descrevendo detalhadamente todos os registradores, mapas de memória, controle de clock (RCC), pinos de E/S (GPIO) e periféricos do chip. É a nossa fonte primária de endereços e máscaras de bits.
2. **PM0214 Programming Manual — STM32 Cortex®-M4 MCUs and MPUs**: Manual focado no núcleo de processamento, no conjunto de instruções e nos periféricos do próprio core ARM (NVIC, SysTick, MPU, FPU).
3. **DS10314 Datasheet — STM32F411xC STM32F411xE**: Especificações elétricas, pinagens, encapsulamento e características temporais do chip.

Linguagem C e Ferramentas de Desenvolvimento

- **Kernighan, B. W.; Ritchie, D. M.** *The C Programming Language*, 2ª edição. Prentice Hall, 1988 — A referência canônica para o padrão da linguagem C, essencial para entender regras de escopo, tempo de vida, ligação e manipulação de ponteiros.
- **Documentação Oficial GNU**: [GCC Manual](#), [GNU Linker \(ld\)](#) e [GNU Make Manual](#).

[Iniciar Conteúdo da Disciplina](#)



Semana 1 — Arquitetura de Computadores

1	Fundamentos de Arquitetura de Computadores	21
1.1	O que é Arquitetura de Computadores?	21
1.2	O Modelo de Von Neumann	22
1.3	Arquitetura Harvard	26
1.4	CISC vs. RISC	27
1.5	Memória e Registradores	28
1.6	Exemplos de Arquiteturas de Conjunto de Instruções	31
1.7	Questionário de revisão	33
1.8	Referências	34
2	Prática: Fundamentos de Linux	35
2.1	Por que terminal antes de interface gráfica?	35
2.2	Um pouco de história	35
2.3	Sistemas de arquivos: Windows vs. Linux	35
2.4	Abrindo o terminal e o primeiro comando	36
2.5	Anatomia de um comando	37
2.6	Navegação básica	37
2.7	Criando, movendo e removendo arquivos	38
2.8	Editores de texto no terminal	40
2.9	Buscando arquivos e conteúdo	41
2.10	Superusuário e gerenciamento de pacotes	41
2.11	Tipos de arquivo e permissões	42
2.12	Compondo comandos com pipes	42
2.13	Redirecionamento e encadeamento de comandos	43
2.14	Filtros de texto	44
2.15	Identificando e inspecionando arquivos	45
2.16	Variáveis de ambiente e o PATH	46
2.17	Onde isso se aplica no curso	48
2.18	Leitura complementar	48
2.19	Questionário de revisão	49

1. Fundamentos de Arquitetura de Computadores

i Objetivos de aprendizagem

Ao final deste capítulo, você deverá ser capaz de:

- distinguir arquitetura (ISA), organização e implementação de um computador;
- descrever os modelos de Von Neumann e Harvard, o ciclo *fetch-decode-execute* e o gargalo de Von Neumann;
- comparar as filosofias CISC e RISC e explicar a regra *Load-Store*;
- descrever a hierarquia de memória, os registradores, a pilha e os modos de endereçamento.

Esses conceitos são a base do capítulo seguinte, em que os aplicaremos ao ARM Cortex-M.

Antes de tratar especificamente de ARM ou Cortex-M, é necessário revisar alguns conceitos fundamentais de arquitetura de computadores: trata-se de noções aplicáveis a qualquer processador, que fornecerão o vocabulário adequado para a compreensão das decisões de projeto adotadas pela ARM — decisões que serão exploradas nos capítulos seguintes.

1.1 O que é Arquitetura de Computadores?

Arquitetura de Computadores corresponde ao projeto conceitual e à estrutura operacional fundamental de um sistema de computador, definindo de que forma os componentes de hardware se conectam e operam conjuntamente para a execução de tarefas. É prática comum dividir essa área em três subáreas distintas:

- **Conjunto de instruções (ISA — *Instruction Set Architecture*)**: constitui a interface entre o software e o hardware, correspondendo, em certo sentido, ao “idioma” do computador — a forma pela qual o software percebe o hardware subjacente.
- **Organização do Computador**: ocupa-se das unidades operacionais e de suas interconexões, isto é, da forma como as partes são implementadas e conectadas entre si — trata-se do projeto físico e lógico do funcionamento interno do processador.
- **Implementação do Hardware**: compreende os detalhes físicos da construção das unidades de organização, tais como a tecnologia do circuito, o projeto VLSI e a velocidade de *clock*.

Os termos “Arquitetura de Computadores” e “Organização de Computadores” são, com frequência, empregados de forma intercambiável, embora representem níveis de abstração distintos: a **Organização** concentra-se na estrutura e na implementação física do sistema, ao passo que a **Arquitetura** concentra-se na funcionalidade — isto é, na forma como o sistema se apresenta a quem o programa.

1.1.1 O que define uma ISA?

A **ISA** constitui, na prática, um contrato: tudo aquilo que o software pode assumir como garantido por parte do hardware, independentemente da forma como o fabricante implementou internamente esse hardware. Um mesmo ISA pode ser implementado por chips internamente distintos entre si, e o software continua a funcionar corretamente em qualquer um deles. Uma ISA precisa especificar, entre outros aspectos:

- **Conjunto de instruções**: quais operações o processador é capaz de executar (somar, comparar, desviar, acessar memória, entre outras).
- **Formato das instruções**: a forma como os bits de cada instrução se organizam — tamanho fixo ou variável, quantidade de bits destinada ao código da operação (*opcode*) e quantidade destinada aos operandos.

- **Modelo de registradores:** a quantidade de registradores existentes e a largura, em bits, de cada um deles.
- **Tipos de dados nativamente suportados:** inteiros de 8, 16, 32 ou 64 bits, ponto flutuante, entre outros.
- **Convenções de memória:** a *endianness* (ordem dos bytes na memória) e as regras de alinhamento.
- **Modos de endereçamento:** as diferentes formas de especificar a localização do dado a ser processado por uma instrução — assunto que será examinado detalhadamente mais adiante neste capítulo.

Ao longo da história da computação, fabricantes distintos responderam a essas questões de maneiras bastante diversas — exemplos concretos serão apresentados mais adiante neste capítulo.

1.2 O Modelo de Von Neumann

Em 1945, von Neumann descreveu, no célebre “*First Draft of a Report on the EDVAC*”, uma ideia que parece óbvia nos dias atuais, mas que era, à época, absolutamente revolucionária: **instruções e dados podem residir na mesma memória, sendo tratados exatamente da mesma forma.**



Figure 1.1: John von Neumann (1903–1957). Foto: Los Alamos National Laboratory (domínio público).

Quem foi John von Neumann?

Matemático húngaro-americano, von Neumann é, por vezes, referido como “**o Einstein esquecido**” — alusão à extensão pouco comum de suas contribuições, muitas vezes ofuscada pela fama pública de Einstein. Foi peça fundamental no Projeto Manhattan, fundou a teoria dos jogos em sua forma moderna (com aplicações que se estendem da economia à biologia evolutiva), contribuiu para os fundamentos matemáticos da mecânica quântica e ainda projetou os primeiros algoritmos de simulação numérica de tempo e clima. A arquitetura que leva o seu nome — assunto desta seção — constituiu apenas mais um item, entre tantos outros, de uma carreira que colegas descreviam como a mente mais rápida que já haviam encontrado.

Até então, computadores como o **ENIAC** (1945) — no qual von Neumann também atuou como consultor — eram “programados” de maneira física: por meio da reconexão de cabos e do ajuste manual de chaves para cada novo problema, procedimento que podia demandar dias inteiros de trabalho de uma equipe completa.

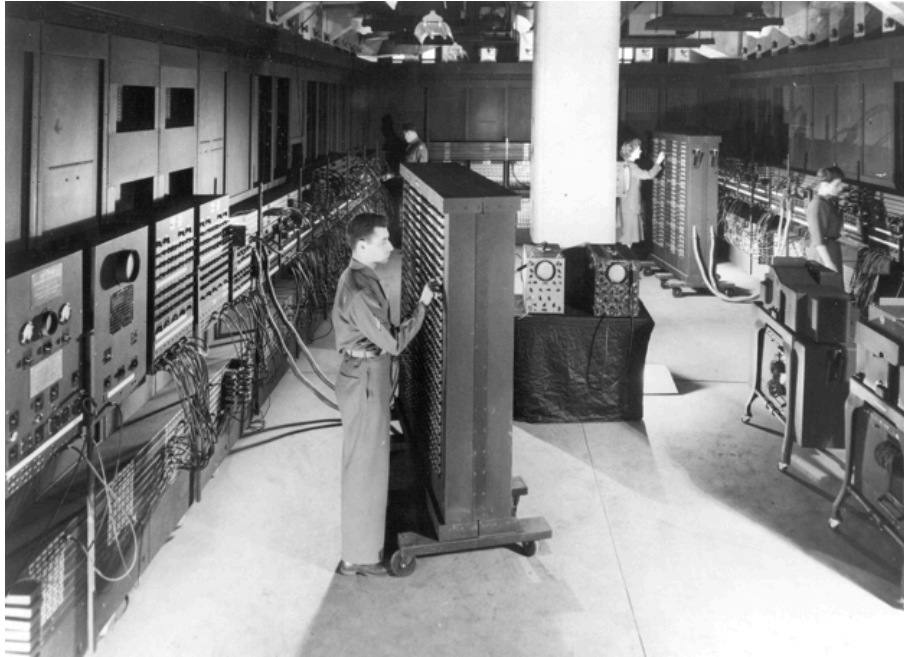


Figure 1.2: O ENIAC: reprogramar significava fisicamente reconectar cabos e ajustar painéis. Foto: Exército dos EUA (domínio público).

Para executar um programa diferente no ENIAC, era necessário desmontar e remontar boa parte dessas conexões físicas. A proposta de von Neumann eliminou essa barreira: uma vez que instruções nada mais são do que números armazenados na memória, “reprogramar” o computador passa a significar, tão somente, **trocar o conteúdo da memória** — sem qualquer necessidade de reconectar cabos. É o que se denomina **conceito de programa armazenado** (*stored-program concept*), fundamento de, literalmente, todo computador já utilizado.

Um computador de arquitetura Von Neumann possui três componentes centrais, interligados por um **barramento** único:

- uma **memória** única, na qual são armazenadas tanto instruções quanto dados;
- uma **CPU**, composta por uma **Unidade Lógica Aritmética (ULA)**, responsável pela execução dos cálculos, e por uma **Unidade de Controle**, responsável por coordenar a busca e a execução das instruções;
- **dispositivos de entrada/saída (I/O)**, destinados à comunicação com o mundo externo.

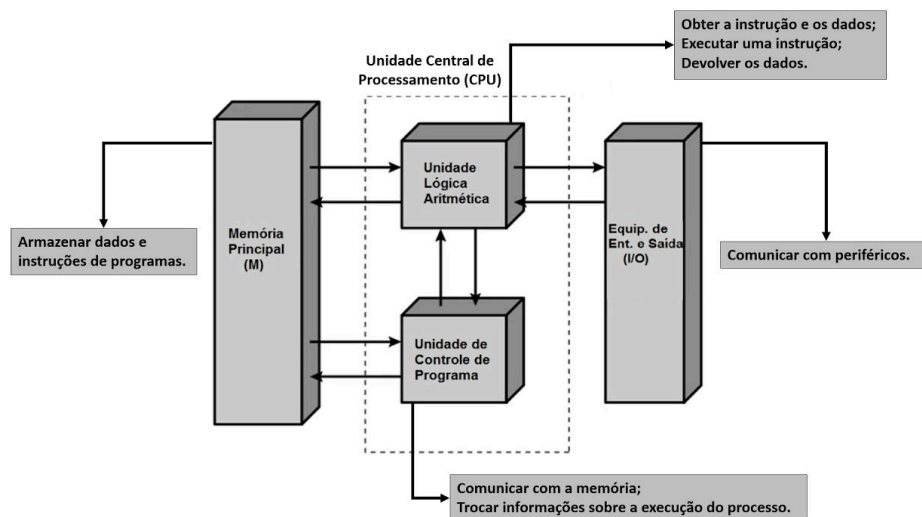


Figure 1.3: Modelo de Von Neumann: memória, CPU (ULA + Unidade de Controle) e dispositivos de entrada/saída.

1.2.1 O Ciclo Fetch-Decode-Execute

De que maneira, na prática, a Unidade de Controle processa um programa armazenado na memória? Por meio de um ciclo que se repete de forma contínua, desde o instante em que o processador é ligado até o momento em que é desligado — o **ciclo Fetch-Decode-Execute** (busca-decodificação-execução):

1. **Fetch (busca):** a Unidade de Controle busca, na memória, a próxima instrução a ser executada, valendo-se do endereço armazenado em um **registrador** especial (pequeno espaço de memória, de altíssima velocidade, embutido diretamente na CPU — assunto detalhado mais adiante neste capítulo), o **Program Counter (PC)**, que, imediatamente após a busca, já é incrementado, passando a apontar para a instrução subsequente.
2. **Decode (decodificação):** a instrução buscada — que corresponde, em essência, a um número binário — é interpretada, determinando-se qual operação deverá ser realizada e com quais operandos.
3. **Execute (execução):** a operação decodificada é, de fato, executada — tipicamente pela ULA, que efetua a leitura dos registradores envolvidos, calcula o resultado correspondente e o grava novamente em um registrador (princípio detalhado na seção referente à regra Load-Store).

Concluída a terceira etapa, o ciclo recomeça a partir da primeira, buscando a instrução seguinte — e é essa repetição incessante, ocorrendo bilhões de vezes por segundo, que confere a impressão de que um computador realiza diversas tarefas simultaneamente.

💡 Leitura complementar

Para o leitor interessado em observar esse ciclo tomando forma literalmente em hardware, com registradores e barramentos visíveis, indicam-se duas referências clássicas:

- **Malvino, A. P.; Brown, J. A.** *Digital Computer Electronics* — obra que introduz o **SAP-1** (*Simple As Possible*), computador didático minimalista, construído passo a passo com o propósito específico de tornar tangível o ciclo *fetch-decode-execute*.
- **Ben Eater** — *Building an 8-bit breadboard computer* — versão moderna e gratuita dessa mesma proposta: uma série de vídeos que constrói, na prática, um computador simples em *proto-board*, registrador por registrador.

Constitui outra alternativa moderna e gratuita, com proposta ligeiramente distinta — parte de portas lógicas e avança até a execução de um programa de alto nível —, o **Nand2Tetris** (Nisan & Schocken).

1.2.2 O Gargalo de Von Neumann

Uma consequência direta do fato de instruções e dados compartilharem a mesma memória e o mesmo barramento consiste na impossibilidade de a CPU buscar uma instrução e, simultaneamente, ler ou escrever um dado: as duas operações disputam o mesmo caminho de comunicação. Essa limitação de *throughput* é conhecida como **Gargalo de Von Neumann** (*Von Neumann Bottleneck*).

Para tornar essa limitação concreta, considere-se o seguinte trecho de código, que efetua a leitura de um valor da memória, soma 1 a esse valor e grava o resultado novamente:

```
LDR R1, [R0]    ; lê um dado da memória
ADD R1, R1, #1   ; soma 1 (só entre registradores)
STR R1, [R0]    ; escreve o dado de volta na memória
```

Cada uma dessas três instruções necessita, primeiramente, ser buscada na memória (*fetch*) antes de sua execução — e duas delas (**LDR** e **STR**) precisam, adicionalmente, acessar a memória para efetuar a leitura ou a escrita do dado propriamente dito. Uma vez que existe apenas **um único barramento**, interligando a CPU a uma única memória compartilhada, tais acessos não podem ocorrer simultaneamente: o dado somente pode trafegar após a busca da instrução liberar o barramento:

Ciclo	Barramento único (Von Neumann)
1	Busca LDR (instrução)
2	Lê [R0] (dado) — espera o ciclo 1 liberar o barramento
3	Busca ADD (instrução)
4	Executa ADD (não acessa memória)
5	Busca STR (instrução)
6	Escreve [R0] (dado) — espera de novo

Seis ciclos, portanto, cada um utilizando o barramento uma única vez, em sequência. A seguir, será examinado de que forma a arquitetura Harvard ataca precisamente esse problema.

1.3 Arquitetura Harvard

A arquitetura **Harvard** (denominação em homenagem ao computador Harvard Mark I, de 1944) resolve o Gargalo de Von Neumann de maneira direta: em vez de uma única memória e um único barramento compartilhados entre instruções e dados, emprega **dois barramentos fisicamente separados** — um dedicado exclusivamente a instruções, outro dedicado exclusivamente a dados.

O ganho reside no fato de que, sendo os dois barramentos independentes, a CPU passa a ser capaz de buscar a próxima instrução e, simultaneamente, acessar um dado referente à instrução atual — as duas operações deixam de disputar o mesmo caminho de comunicação. Retomando o mesmo exemplo de código:

Ciclo	Barramento de instrução	Barramento de dados
1	Busca LDR	—
2	Busca ADD (em paralelo)	Lê [R0] (em paralelo)
3	Busca STR	—
4	— (nenhuma instrução nova ainda)	Escreve [R0] (em paralelo com o ciclo anterior, dependendo do pipeline)

Realiza-se, assim, o mesmo trabalho em um número menor de ciclos totais de barramento, uma vez que a busca de instrução e o acesso a dado deixam de ser mutuamente exclusivos.

Na prática, a arquitetura Harvard em sua forma “pura” (com memórias de instrução e de dados fisicamente separadas, sem qualquer possibilidade de acessar uma delas como se fosse a outra) constitui-se em algo raro, restrito a alguns DSPs especializados. O que se observa com maior frequência — inclusive no próprio Cortex-M que será utilizado neste curso — é a arquitetura **Harvard Modificada**: barramentos separados, de modo a obter o desempenho anteriormente demonstrado, mantendo-se, contudo, os dois

espaços de memória visíveis em um único mapa de endereços unificado, do ponto de vista de quem programa. Esse assunto será examinado detalhadamente ao tratarmos da arquitetura do Cortex-M.

1.4 CISC vs. RISC

Até a década de 1980, prevalecia, na indústria, a tendência de se criarem conjuntos de instruções cada vez mais **ricos e complexos** — instruções em maior número, cada uma realizando mais tarefas, com modos de endereçamento progressivamente mais flexíveis. Essa filosofia tornou-se conhecida como **CISC** (*Complex Instruction Set Computer*), permanecendo viva, até os dias atuais, na arquitetura x86.

O raciocínio subjacente ao CISC era coerente para a época: sendo a memória cara e escassa, instruções mais “poderosas” — uma única instrução realizando o trabalho de várias — davam origem a programas de menor tamanho. Ademais, transferir complexidade para o hardware ou para o microcódigo do processador facilitava consideravelmente a tarefa de quem desenvolvia compiladores; cabe observar que, à época, os compiladores frequentemente não eram tão sofisticados, e boa parte do código ainda era escrita diretamente em assembly.

No início da década de 1980, pesquisadores de Berkeley (projeto *Berkeley RISC*, liderado por David Patterson) e de Stanford (projeto *MIPS*, liderado por John Hennessy — ambos, décadas mais tarde, laureados com o Prêmio Turing em razão desse trabalho) questionaram essa premissa. Constataram que, na prática, os compiladores utilizavam apenas uma fração reduzida das instruções complexas disponíveis no CISC, sendo a maior parte do tempo de execução dedicada a instruções simples. A proposta apresentada consistia em substituir poucas instruções complexas por um conjunto amplo de instruções simples, todas de tamanho fixo, executadas em tempos previsíveis. Nascia, assim, o **RISC** (*Reduced Instruction Set Computer*) — filosofia subjacente às arquiteturas ARM, MIPS, RISC-V e SPARC.

Característica	CISC (ex. x86)	RISC (ex. ARM)
Tamanho das instruções	Variável, complexas	Fixo, simples
Onde a complexidade mora	No hardware/microcódigo do processador	No compilador
Quantidade de instruções	Centenas, muitas especializadas	Dezenas, poucas e ortogonais
Filosofia de origem	Memória era cara → instruções densas economizam espaço	Instruções simples e uniformes → mais fáceis de otimizar e paralelizar (pipeline)

1.4.1 A Regra Load-Store

A **arquitetura Load-Store** (ou “carrega-armazena”) constitui a regra concreta relativa ao acesso à memória, seguida por praticamente todo processador RISC: operações aritméticas e lógicas somente podem ser realizadas sobre dados já presentes nos registradores da CPU, nunca diretamente sobre a memória.

O princípio Load-Store divide as instruções em duas categorias rigorosamente distintas:

1. Instruções de acesso à memória (Load/Store) — as únicas responsáveis pela transferência de dados entre a memória e os registradores:

- **LOAD:** copia um dado da memória para um registrador — por exemplo, `LDR R1, [endereço]`.
- **STORE:** copia um dado de um registrador para a memória — por exemplo, `STR R1, [endereço]`.

2. Instruções de operação (ALU) — abrangem todas as instruções aritméticas e lógicas (soma, subtração, AND, OR, entre outras), as quais só podem operar sobre registradores. Por exemplo, `ADD R3, R1, R2` soma R1 a R2 e grava o resultado em R3 — não sendo permitido somar diretamente um valor proveniente da memória.

Compare-se essa regra com uma arquitetura **Register-Memory**, como o x86 (CISC), na qual a ALU pode operar diretamente sobre a memória:

Exemplo de soma	Load-Store (RISC — ex. ARM)	Register-Memory (CISC — ex. x86)
$C = A + B$	3 instruções: <code>LDR R1, [A] → LDR R2, [B] → ADD R3, R1, R2</code>	1 instrução: <code>ADD R1, [B]</code>

1.4.2 Por que isso importa para processadores modernos?

Não obstante exigir um número maior de instruções de *load/store* no código, a arquitetura Load-Store proporciona vantagens fundamentais para o projeto de CPUs modernas:

- **Facilidade de *pipelining*:** uma vez que praticamente todas as instruções demandam tempo de execução aproximadamente equivalente, torna-se substancialmente mais simples dividir o processamento em estágios (*pipeline*), permitindo a execução simultânea de diversas instruções em fases distintas.
- **Velocidade e frequência:** o hardware de execução (ALU) e o hardware de acesso à memória permanecem separados; instruções mais simples possibilitam frequências de *clock* mais elevadas.
- **Compiladores mais eficientes:** um conjunto de instruções reduzido e simples facilita a otimização por parte do compilador, que passa a planejar de modo mais eficaz o uso dos registradores e a minimizar acessos lentos à memória.

1.5 Memória e Registradores

Registradores e memória já foram mencionados diversas vezes ao longo deste capítulo — é chegado o momento de examinar esses conceitos com maior profundidade. Trata-se de noções genéricas, válidas para qualquer arquitetura; no capítulo dedicado ao Cortex-M, será examinada a versão específica correspondente (registradores R0-R15, SP, LR, PC).



Tip

Recomenda-se atenção especial às seções sobre **memória volátil/não-volátil** e **pilha**: elas explicam diretamente por que razão o `startup.c`, a ser escrito no capítulo dedicado ao código de inicialização, precisa copiar dados da Flash para a RAM antes de invocar `main()`, e por que motivo `STACK_START` aponta para o final da SRAM.

1.5.1 Hierarquia de Memória

Nenhum processador emprega um único tipo de memória para todas as finalidades. Existe, ao contrário, uma **hierarquia**: quanto mais rápida a memória, menor e mais dispendiosa ela se revela por byte; quanto mais lenta, maior e mais barata. Do nível mais próximo à CPU ao mais distante, tem-se:

1. **Registradores** — situados no interior da própria CPU. O acesso ocorre em um único ciclo de *clock*, ainda que em quantidade bastante limitada (dezenas de posições, cada uma com 32 ou 64 bits).
2. **Cache (L1, L2, ...)** — memória rápida interposta entre a CPU e a RAM, na qual são armazenadas cópias de dados utilizados recentemente. Processadores de alto desempenho (como os Cortex-A) dispõem de *cache*; a maioria dos microcontroladores Cortex-M, contudo, não a possui — mais uma razão pela qual o tempo de execução em sistemas embarcados se revela mais previsível.
3. **RAM (memória principal)** — capacidade substancialmente maior que a de registradores e *cache*, embora ordens de magnitude mais lenta. É nela que residem as variáveis e os dados em uso durante a execução do programa.
4. **Armazenamento não-volátil (Flash, SSD, HD)** — a maior capacidade e, simultaneamente, a mais lenta. Nele são armazenados o programa e os dados que necessitam sobreviver a um desligamento.

Em um microcontrolador como o STM32F411, a ser utilizado neste curso, essa hierarquia se apresenta de forma bem mais reduzida do que em um PC: não há *cache* nem SSD — apenas alguns KB de **SRAM** (que cumpre o papel de registradores mais memória principal) e algumas centenas de KB de **Flash** (o armazenamento não-volátil).

1.5.2 Memória Volátil vs. Não-Volátil

Essa distinção figura entre as mais relevantes para sistemas embarcados:

- **Memória volátil** (RAM/SRAM): perde integralmente o seu conteúdo no instante em que a energia é desligada. É rápida tanto para leitura quanto para escrita, e nela residem variáveis, pilha e dados em uso durante a execução.
- **Memória não-volátil** (Flash, ROM, EEPROM): preserva o seu conteúdo mesmo na ausência de energia. É mais lenta para escrita (por vezes exigindo o apagamento de um bloco inteiro antes de uma nova gravação), e nela permanecem armazenados o código do programa e as constantes, entre uma execução e outra.

Disso decorre uma consequência direta e de grande relevância prática: ao ligar-se o microcontrolador, o programa (código somado aos valores iniciais das variáveis globais) já se encontra gravado na Flash — entretanto, as variáveis precisam existir na RAM para que possam ser lidas e escritas durante a execução. Torna-se necessário, portanto, que alguém copie os valores iniciais da Flash para a RAM antes de o programa efetivamente começar.

Esse “alguém” corresponde exatamente ao código de inicialização (`startup.c`) que será escrito mais adiante no curso: ele copia a seção `.data` (variáveis globais inicializadas) da Flash para a RAM, e zera a seção `.bss` (variáveis globais não-inicializadas) diretamente na RAM — tudo isso **antes** de invocar `main()`.

1.5.3 Tecnologias de memória não-volátil

A enumeração “Flash, ROM, EEPROM”, apresentada no parágrafo anterior, oculta tecnologias substancialmente distintas entre si, sendo oportuno esclarecê-las neste momento, uma vez que voltarão a surgir, com nomes de sinais e procedimentos concretos, no estudo da Flash do STM32F411.

A primeira geração de memória não-volátil era, em sentido literal, fixa: a **ROM** (*Read-Only Memory*) de máscara é gravada na fábrica, durante o próprio processo de fabricação do chip, e jamais se altera posteriormente — qualquer correção exige um novo lote de silício. Surgiram, posteriormente, formas de gravação de conteúdo realizadas fora do ambiente de fábrica:

- **PROM** (*Programmable ROM*): gravável **uma única vez**, pelo próprio usuário, tipicamente mediante o rompimento de pequenos fusíveis internos. Uma vez gravada, comporta-se novamente como uma ROM.
- **EPROM** (*Erasable PROM*): admite apagamento e regravação, sendo o apagamento, contudo, um procedimento físico — a exposição do chip à luz ultravioleta por meio de uma janela de quartzo no encapsulamento, o que apaga **a totalidade** do conteúdo de uma só vez. Solução pouco prática para uso em campo.
- **EEPROM** (*Electrically Erasable PROM*): o apagamento passa a ser **elétrico**, dispensando luz ultravioleta ou a abertura do chip. Trata-se da primeira dessa família que um programa é capaz de regravar autonomamente, durante a própria operação, tipicamente byte a byte.
- **Flash**: evolução da EEPROM, otimizada em termos de custo e densidade. A diferença central reside na granularidade: a Flash grava byte a byte (ou palavra a palavra), mas apenas **apaga em blocos**, nunca um único byte isoladamente. É precisamente essa restrição que explica a necessidade de “apagar um bloco inteiro antes de regravar”, mencionada anteriormente.

Figura: linha do tempo ROM (de máscara) → PROM → EPROM → EEPROM → Flash, com um ícone e uma frase curta por tecnologia (gravação de fábrica; fusíveis, uma vez; luz UV, tudo de uma vez; elétrico, byte a byte; elétrico, apaga em blocos).

i Por que apagar é diferente de gravar?

Cada bit de uma EEPROM ou de uma Flash é armazenado em uma célula equipada com um **transistor de porta flutuante** (*floating-gate transistor*): uma porta adicional, isolada por uma camada de óxido, na qual elétrons permanecem aprisionados (ou não) indefinidamente, mesmo sem energia, alterando a forma como o transistor se comporta durante a leitura. **Gravar** consiste em introduzir elétrons no interior dessa porta, célula por célula — o que permite realizar a operação bit a bit (ou byte a byte). **Apagar** realiza o percurso inverso, retirando os elétrons — sendo que, na Flash, o circuito responsável por aplicar essa tensão de apagamento é compartilhado por um bloco inteiro de células, de modo a economizar espaço no chip. Origina-se, assim, a regra: grava-se de modo fino, apaga-se de modo grosseiro.

No âmbito da própria Flash, o mercado dividiu-se em duas famílias, cada uma otimizada para usos distintos:

		Flash NOR	Flash NAND
Endereçamento		Por byte/palavra, como uma RAM comum	Por página/bloco, como um disco
Execução (XIP)	direta	Sim — o processador busca e executa instruções direto dela	Não — precisa ser copiada para RAM antes de executar
Densidade / custo por byte		Menor densidade, mais cara	Maior densidade, mais barata
Uso típico		Memória de programa em microcontroladores (é a Flash do STM32F411)	Cartões SD, <i>pen drives</i> , SSDs

Figura: duas grades lado a lado. À esquerda, Flash NOR como uma matriz endereçável por byte, com uma seta “CPU busca instrução direto daqui”; à direita, Flash NAND organizada em páginas/blocos, com uma seta “CPU → RAM → executa”, mostrando a cópia obrigatória antes de rodar.

É exatamente por essa razão que um microcontrolador executa o programa diretamente a partir de sua memória de programa, sem qualquer cópia prévia para a RAM: a Flash embarcada no STM32F411 é do tipo **NOR**. Convém reter esse nome — no capítulo dedicado ao STM32F411 será examinado o procedimento de gravação (desbloquear, apagar o setor, gravar) e a organização em setores, que representam, para aquele chip específico, a versão concreta das mesmas duas ideias aqui apresentadas: o apagamento constitui uma operação de bloco, e é de natureza elétrica, sem recurso a luz ultravioleta ou a fusíveis.

1.5.4 Registradores

Um **registrador** consiste em uma pequena unidade de memória de altíssima velocidade, embutida diretamente na CPU. Diferentemente da RAM, os registradores não possuem um “endereço de memória” no sentido tradicional do termo — são acessados diretamente pelo nome (R0, R1, ...) no interior das instruções.

Todo processador dispõe de um **banco de registradores**, tipicamente dividido em duas categorias:

- **Registradores de propósito geral:** empregados livremente pelo programa (ou pelo compilador) para o armazenamento de valores temporários no decorrer de cálculos.
- **Registradores de propósito especial:** possuem um papel fixo, definido pela própria arquitetura — como, por exemplo, um registrador que aponta permanentemente para o topo da pilha, ou outro que armazena o endereço da próxima instrução a ser executada.

Retomando o que foi apresentado na seção anterior: na arquitetura Load-Store, toda operação aritmética ou lógica ocorre exclusivamente entre registradores, jamais diretamente sobre a memória. Isso confere ao banco de registradores o papel de “bancada de trabalho” da ALU: um dado precisa ser carregado (**LOAD**) da memória para um registrador antes de ser processado, e o resultado, quando necessário, precisa ser salvo (**STORE**) novamente na memória.

1.5.5 A Pilha (Stack)

A **pilha** (*stack*) corresponde a uma região da memória RAM organizada como uma estrutura de dados **LIFO** (*Last In, First Out* — o último dado inserido é o primeiro a ser retirado). Duas operações a definem:

- **PUSH:** empilha um novo valor no topo da pilha.
- **POP:** remove o valor situado no topo da pilha e o retorna.

Um registrador especial, o **Stack Pointer (SP)**, aponta permanentemente para o topo corrente da pilha, sendo atualizado de forma automática a cada operação de **PUSH** ou **POP**.

Quais são, então, as finalidades da pilha? Três usos centrais podem ser destacados:

- **Variáveis locais:** quando uma função é chamada, suas variáveis locais costumam residir na pilha, sendo automaticamente liberadas no momento em que a função retorna.
- **Endereço de retorno:** quando uma função **A()** chama uma função **B()**, o endereço a partir do qual **A()** deverá prosseguir, após a conclusão de **B()**, é empilhado antes de o fluxo de execução ser desviado para **B()**.
- **Preservação de registradores:** caso **B()** necessite utilizar um registrador já em uso por **A()**, o valor anteriormente armazenado pode ser empilhado no início de **B()** e recuperado (**POP**) antes do retorno, de modo que **A()** não perceba qualquer alteração.

Isso explica por que, no `startup.c`, a primeira definição estabelecida é justamente o endereço inicial da pilha — antes mesmo de qualquer código ser executado, o processador precisa saber onde a pilha tem início. Nos processadores ARM Cortex-M, essa área é normalmente posicionada ao final da SRAM (a razão para tanto: a pilha cresce “para baixo”, em direção a endereços menores; iniciar, portanto, no topo da memória disponível maximiza o espaço utilizável antes de uma eventual colisão com os dados globais, que crescem “para cima” a partir do início da RAM).

1.5.6 Modos de Endereçamento

O **modo de endereçamento** corresponde à forma como uma instrução especifica a localização do dado a ser processado. Entre os mais comuns, destacam-se:

Modo	O que significa	Exemplo (sintaxe ilustrativa)
Imediato	O valor está embutido na própria instrução	<code>MOV R0, #10</code> (coloca o número 10 em R0)
Registrador	O valor está em outro registrador	<code>MOV R0, R1</code> (copia o valor de R1 para R0)
Direto (absoluto)	O endereço de memória está embutido na instrução	<code>LDR R0, [0x2000_0000]</code>
Indireto (por registrador)	Um registrador contém o <i>endereço</i> onde está o dado	<code>LDR R0, [R1]</code> (R1 guarda o endereço; carrega o dado que está lá)
Indexado (com deslocamento)	Um registrador contém uma base, somada a um deslocamento fixo	<code>LDR R0, [R1, #4]</code> (lê o endereço R1+4)

O modo indexado é particularmente comum no acesso a campos de uma estrutura de dados ou a elementos de um vetor — e, como se verá adiante, é exatamente essa técnica que se emprega para acessar registradores de periféricos a partir de um endereço-base somado a um deslocamento (por exemplo, `STM32_GPIOC_BASE + STM32_GPIO_MODER_OFFSET`).

1.6 Exemplos de Arquiteturas de Conjunto de Instruções

Os dois modelos examinados — Load-Store (RISC) e Register-Memory (CISC) — não se restringem à teoria: definem famílias inteiras de processadores atualmente em uso:

- **x86/x86-64 (Intel, AMD):** CISC, Register-Memory. Domina o mercado de *desktops*, *notebooks* e servidores. Mantém retrocompatibilidade desde a década de 1970, fato que explica boa parte de sua complexidade interna.
- **ARM:** RISC, Load-Store. Domina o segmento de dispositivos móveis e embarcados — constitui o foco deste curso e o assunto dos capítulos subsequentes.
- **RISC-V:** RISC, Load-Store, e de natureza **aberta** (sem custo de licenciamento) — em rápido crescimento como alternativa à ARM.
- **MIPS:** RISC, Load-Store. Com histórico relevante em roteadores e sistemas embarcados, encontra-se hoje menos dominante, mas exerceu influência sobre o projeto de praticamente toda arquitetura RISC posterior, incluindo a própria ARM.

Esse panorama já foi apresentado sob um ângulo distinto — mais voltado a plataformas de desenvolvimento do que à ISA propriamente dita — no [gráfico de posicionamento da Apresentação](#). Convém revisitar aquela imagem à luz desse novo vocabulário técnico: RISC-V e ARM Cortex-M situam-se no mesmo “lado” do espectro Load-Store, o que também se verifica para o AVR (utilizado no Arduino), como será examinado a seguir.

Ao longo da história da computação, fabricantes distintos responderam a essas questões de maneiras bastante diversas:

Ano	Processador	Fabricante	Observação
1974	Intel 8080	Intel	8 bits, base do que viria a ser o x86
1975	MOS 6502	MOS Technology	8 bits, simples e barato — Apple II, NES, Commodore 64

Ano	Processador	Fabricante	Observação
1978	Intel 8086	Intel	16 bits, deu origem à família x86 (CISC), ainda em uso hoje
1979	Motorola 68000	Motorola	16/32 bits — Macintosh original, Amiga, Sega Genesis
1985	ARM1	Acorn Computers	primeiro chip comercial RISC — origem da arquitetura ARM
1991	MIPS R4000	MIPS Technologies	RISC de 64 bits — estações de trabalho, e o Nintendo 64
1994	MIPS R3000 (custom)	LSI Logic / Sony	RISC — o processador do PlayStation original
2005	IBM PowerPC “Xenon”	IBM	RISC — o processador do Xbox 360
2008	Intel Core i7	Intel	x86-64 (CISC) — ainda hoje a linha “topo de linha” da Intel para desktops/notebooks
2010	ARM Cortex-M	ARM Holdings	RISC para microcontroladores — o foco deste curso
2017	NVIDIA Tegra X1 (núcleos ARM Cortex-A57)	NVIDIA	RISC — o processador do Nintendo Switch
2019	RISC-V (ex.: SiFive)	Diversos fabricantes	RISC, ISA aberta — sem custo de licenciamento
2020	Apple M1	Apple	RISC (ARM) — os Macs abandonaram o x86 (Intel) por processadores ARM próprios
2024	Qualcomm Snapdragon X Elite	Qualcomm	RISC (ARM) — primeiros notebooks Windows de peso rodando nativamente em ARM

Merece destaque a trajetória que essa tabela revela: os consoles de videogame migraram de RISC (N64, PlayStation, Xbox 360) para CISC ao longo da década de 2010 (o PlayStation 4/5 e o Xbox One/Series utilizam x86-64 da AMD, por razões de compatibilidade com o PC) — ao passo que os computadores pessoais percorreram, mais recentemente, o caminho inverso: após décadas de domínio do x86, tanto a Apple (nos Macs equipados com o chip M1 em diante) quanto a Microsoft (em *notebooks* com Snapdragon) apostaram fortemente em **ARM** na década de 2020, em busca de maior eficiência energética — a mesma vantagem que torna o ARM dominante em sistemas embarcados, como será examinado ao longo do curso.

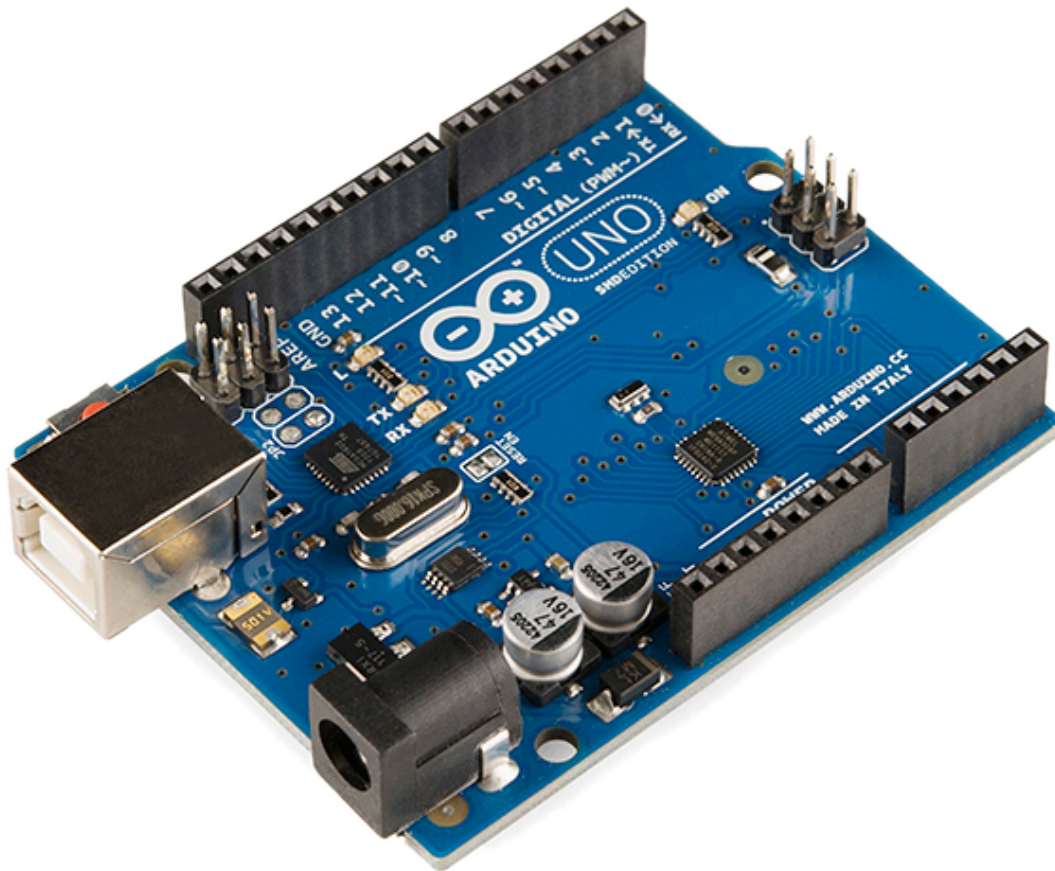


Figure 1.4: Arduino Uno (chip AVR ATmega328P sob o dissipador central). Foto: SparkFun Electronics, CC BY 2.0.

Um exemplo bem mais próximo: o AVR

Quem já utilizou um **Arduino Uno** já programou, ainda que sem sabê-lo, uma ISA **RISC** de 8 bits: a família **AVR**, lançada pela Atmel (atualmente parte da Microchip) em 1996, presente no chip **ATmega328P** situado sob o dissipador retangular no centro da placa.

O AVR também emprega arquitetura **Harvard** — memória de programa (Flash) e memória de dados (SRAM) fisicamente separadas, exatamente o conceito examinado anteriormente. Constitui um bom recordatório de que essas ideias, por vezes tidas como “acadêmicas”, não são exclusividade de processadores de alto desempenho: encontram-se presentes na placa de introdução à eletrônica mais amplamente utilizada que existe.

Nos capítulos subsequentes, será examinado de que modo esses princípios se concretizam especificamente na arquitetura **ARM** — sua evolução histórica e o percurso que a conduziu ao Cortex-M, a ser programado ao longo de todo o curso.

1.7 Questionário de revisão

1. As três subáreas da Arquitetura de Computadores são: conjunto de instruções (ISA), organização do computador, e .

2. No modelo de Von Neumann, instruções e dados ficam em memórias fisicamente separadas. Verdadeiro/Falso
3. No ciclo Fetch-Decode-Execute, qual registrador guarda o endereço da próxima instrução a ser buscada? Stack Pointer (SP) Program Counter (PC) Registrador de Status
4. O Gargalo de Von Neumann acontece porque busca de instrução e acesso a dado competem pelo mesmo .
5. A arquitetura que usa barramentos separados para instrução e dados, permitindo os dois acontecerem em paralelo, se chama .
6. Em uma arquitetura Load-Store (RISC), uma instrução **ADD** pode somar um registrador diretamente com um valor da memória. Verdadeiro/Falso
7. A filosofia RISC nasceu, nos anos 1980, como reação a arquiteturas CISC, com instruções cada vez mais complexas de 8 bits, consideradas fracas demais sem sistema de memória.
8. O AVR, usado no Arduino Uno, segue a filosofia CISC/RISC.
9. Do mais rápido/menor para o mais lento/maior, a hierarquia de memória é: registradores, cache, , armazenamento não-volátil.
10. Memória volátil perde seu conteúdo quando a energia é desligada. Verdadeiro/Falso
11. O código do programa fica gravado na Flash (não-volátil) RAM (volátil), enquanto variáveis em uso durante a execução ficam na Flash (não-volátil) RAM (volátil).
12. A pilha (stack) segue uma disciplina de acesso chamada .
13. Qual registrador especial sempre aponta para o topo atual da pilha? Program Counter (PC) Stack Pointer (SP) Registrador de Status
14. No modo de endereçamento imediato/indireto/indexado, o valor a ser usado já está embutido diretamente na instrução (ex.: **MOV R0, #10**).
15. Por que a pilha, no Cortex-M, costuma começar no final da SRAM e crescer “para baixo”? Porque assim ela maximiza o espaço disponível antes de colidir com os dados globais, que crescem do início da RAM para cima e é mais rápida de acessar obrigatória por lei da arquitetura ARM.

1.8 Referências

- [1] [ARMv7-M Architecture Reference Manual](#)
- [2] Patterson, D. A.; Hennessy, J. L. *Computer Organization and Design: The Hardware/Software Interface*.
- [3] Tanenbaum, A. S. *Structured Computer Organization*.

2. Prática: Fundamentos de Linux

i Note

Esta aula cobre o terminal Linux que será utilizado em todas as atividades práticas do curso. Ainda que você já possua familiaridade com ele, vale a pena percorrer rapidamente cada seção — cada uma delas termina com uma atividade de fixação, que pode servir como checagem rápida antes de avançar.

2.1 Por que terminal antes de interface gráfica?

Pode parecer estranho iniciar um curso de 2026 aprendendo uma interface de texto existente desde os anos 1970. Há, contudo, uma razão prática e direta para essa escolha — e ela corresponde exatamente à razão pela qual este curso adota a abordagem *bottom-up*: **a interface gráfica oculta o que efetivamente ocorre por baixo dela, ao passo que o terminal não o faz.**

Ao clicar em um botão numa IDE gráfica, diversas operações ocorrem sem que você jamais as veja. Ao digitar um comando no terminal, por outro lado, você observa exatamente o que foi executado, com quais parâmetros, sendo capaz de repetir, automatizar ou depurar esse passo com precisão. Essa distinção importa particularmente em sistemas embarcados, nos quais as ferramentas de linha de comando (compilador, *linker*, gravador, depurador) **constituem** a própria ferramenta — não existe, por trás delas, nenhuma “versão gráfica” a esconder a complexidade.

Cabem, ainda, outros motivos de ordem prática:

- **Portabilidade:** os mesmos comandos funcionam em qualquer distribuição Linux, em qualquer servidor remoto, em qualquer *pipeline* de integração contínua (CI/CD) — uma sequência de cliques numa GUI não é reproduzível dessa forma.
- **Automação:** comandos podem ser armazenados em *scripts* e repetidos identicamente quantas vezes for necessário, constituindo a base de tudo o que faremos com **Makefile** mais adiante no curso.
- **Precisão:** o terminal não admite ambiguidade — um comando faz exatamente o que foi escrito, sempre.

2.2 Um pouco de história

O Unix surgiu ao final dos anos 1960, em um contexto de hardware extremamente limitado, no qual não havia recursos computacionais disponíveis para telas gráficas elaboradas. A interface de texto não constituía, portanto, uma limitação estética, mas sim a forma mais eficiente de interagir com a máquina naquele momento. O programa responsável por interpretar aquilo que se digita e traduzi-lo em ações do sistema operacional denomina-se **shell** — é ele quem “dialoga” com o usuário no terminal.

Décadas mais tarde, mesmo diante da abundância de hardware gráfico, o *shell* permanece como a interface preferida para o desenvolvimento profissional — não por nostalgia, mas porque continua sendo a forma mais direta de operar um sistema, além de ser a mais simples de combinar na construção de soluções novas, como será demonstrado ao longo desta aula.

2.3 Sistemas de arquivos: Windows vs. Linux

Antes de digitar o primeiro comando, é importante compreender a árvore de diretórios que será percorrida ao longo do curso — sobretudo porque ela é estruturalmente distinta daquela já conhecida no Windows.

Múltiplas unidades vs. uma única árvore. No Windows, cada disco ou partição recebe uma letra própria (C:\, D:\...), cada uma constituindo sua própria árvore independente. No Linux, existe **uma única árvore**, com início na **raiz /**. Qualquer outro disco ou partição é “montado” como uma subpasta dentro dessa mesma árvore — não existe um “D:” separado; existe, isso sim, um diretório adicional pendurado em algum ponto de /. Os caminhos também empregam a barra normal /, e não a barra invertida \ característica do Windows.

Pasta pessoal. O equivalente ao `C:\Users\<usuário>` do Windows é, no Linux, `/home/<usuário>` — é justamente para lá que o comando `cd`, quando executado sem argumentos, sempre conduz.

A ponte do WSL. Como estamos executando o Ubuntu dentro do WSL, os dois mundos coexistem: os discos do Windows permanecem acessíveis de dentro do Linux em `/mnt/c/`, `/mnt/d/`, e assim por diante. Os arquivos baixados pelo navegador do Windows, por exemplo, costumam encontrar-se em `/mnt/c/Users/<usuário>/Downloads`. É dessa forma que se transita entre os dois sistemas de arquivos sem a necessidade de sair do terminal.

Atenção a maiúsculas e minúsculas. O Windows não distingue `Sample.txt` de `sample.txt` — para ele, tratam-se do mesmo arquivo. O Linux, por sua vez, **distingue** ambos: `Sample.txt` e `sample.txt` constituem dois arquivos diferentes, capazes de coexistir na mesma pasta. Um erro de digitação envolvendo maiúsculas e minúsculas é, aliás, uma causa recorrente de mensagens de “arquivo não encontrado” para quem está começando.

Quase tudo é um arquivo. Um traço bastante particular do Linux consiste no fato de que até mesmo dispositivos de hardware (discos, portas seriais, gravadores USB) aparecem como arquivos dentro de `/dev/`. Isso não constitui mera curiosidade — é exatamente o que será empregado mais adiante: o gravador **ST-LINK**, responsável por programar o microcontrolador, aparecerá como um arquivo do tipo `/dev/bus/usb/001/00X`, e as permissões desse “arquivo” (examinadas na próxima seção) determinam se será ou não possível gravar o *firmware*.

Alguns diretórios padrão que você encontrará ao longo do curso: `/home` (arquivos pessoais dos usuários), `/etc` (arquivos de configuração do sistema), `/usr` (a maior parte dos programas instalados) e `/tmp` (arquivos temporários, apagados a cada reinicialização).

2.3.1 Caminhos absolutos e relativos

Um **caminho absoluto** tem início em `/` e descreve a localização de um arquivo ou diretório de forma completa, independentemente de onde o usuário se encontre no momento:

```
foo@bar$ cd /home/usuario/seml-workspace
```

Um **caminho relativo**, por sua vez, é descrito a partir do diretório atual — sem `/` no início:

```
foo@bar$ cd seml-workspace      # entra em uma subpasta do diretório atual
foo@bar$ cd ..                  # sobe um nível (diretório pai)
foo@bar$ cd ../outro-diretorio  # sobe um nível e entra em outro
```

Note

Para saber mais: o que foi apresentado aqui é suficiente para o dia a dia. Caso deseje compreender o que realmente ocorre por baixo quando um arquivo é salvo (*journaling*, *cache* de escrita, *fsync*...), consulte [Internals of Windows and Linux File Systems](#) — leitura opcional, de nível consideravelmente mais avançado.

2.4 Abrindo o terminal e o primeiro comando

Coloquemos isso em prática. Abra o terminal do Ubuntu (WSL); deve aparecer uma linha semelhante à seguinte, aguardando que algo seja digitado:

```
foo@bar:~$
```

Essa linha corresponde ao **prompt** — a forma pela qual o *shell* comunica “estou pronto, pode digitar um comando”. `foo` corresponde ao nome do usuário, `bar` ao nome da máquina, `~` indica que se está na pasta *home*, e `$` marca o fim do prompt (por convenção, indicando um usuário comum; um `#` no mesmo lugar indicaria uma sessão como *root*, o superusuário). Ao longo desta aula, os blocos de código empregam `foo@bar$` de forma simplificada, apenas para indicar “isto é um comando digitado no terminal” — na tela de cada estudante, `foo@bar` será substituído pelo respectivo usuário e máquina.

Digite o seu primeiro comando: `pwd` (*Print Working Directory*), responsável por exibir o caminho completo do diretório em que você se encontra no momento.

```
foo@bar$ pwd
```

💡 Atividade de fixação

Abra o terminal e execute `pwd`. Qual foi o caminho retornado? Ele deveria ter início em `/home/`, seguido do seu nome de usuário — essa é, precisamente, a sua pasta *home*.

2.5 Anatomia de um comando

Antes de examinar mais comandos, é importante compreender o padrão seguido por praticamente todos eles:

```
foo@bar$ comando [opções] [argumentos]
```

- O **comando** corresponde ao programa que se deseja executar (`pwd`, `ls`, `cd`...).
- As **opções** (ou *flags*) modificam o comportamento do comando. São precedidas de `-` (forma curta, geralmente uma única letra, como `-l`) ou de `--` (forma longa, mais descritiva, como `--all`). Muitos comandos aceitam ambas as formas para a mesma opção.
- Os **argumentos** constituem o “alvo” do comando — geralmente um arquivo, um diretório ou um texto.

Em `ls -l /home`, por exemplo, `ls` corresponde ao comando, `-l` a uma opção (formato de listagem detalhado) e `/home` ao argumento (o diretório a ser listado). Nem todo comando exige as três partes: `pwd`, isoladamente, já constitui um comando completo, sem opções nem argumentos.

Esse padrão se repetirá constantemente ao longo do curso — inclusive nas ferramentas ainda a serem instaladas, como em `arm-none-eabi-gcc -c -mcpu=cortex-m4 -mthumb -O0 -Wall main.c -o main.o`: por mais complexo que possa parecer à primeira vista, trata-se exatamente da mesma estrutura `comando [opções] [argumentos]`, apenas com um número maior de opções e argumentos.

2.6 Navegação básica

Além de `pwd`, os comandos mais empregados para fins de navegação são `cd`, `mkdir` e `ls`.

cd (*Change Directory*) altera o diretório atual. Quando executado sem argumentos, conduz à pasta *home*:

```
foo@bar$ cd
```

Para se deslocar, `cd` aceita tanto caminhos absolutos ou relativos (conforme apresentado na seção anterior) quanto dois atalhos especiais, válidos em **qualquer** contexto do sistema, e não apenas no `cd`:

- `.` (um ponto) representa “o diretório atual”;
- `..` (dois pontos) representa “o diretório pai” (um nível acima na árvore).

```
foo@bar$ cd ..          # sobe um nível
foo@bar$ cd ../..       # sobe dois níveis
foo@bar$ cd ./seml-workspace # equivalente a "cd seml-workspace"
```

mkdir (*Make Directory*) é responsável pela criação de um novo diretório:

```
foo@bar$ mkdir seml-workspace
```

ls lista o conteúdo de um diretório. Quando empregado isoladamente, exibe apenas os nomes; combinado a opções, revela uma quantidade de informação consideravelmente maior. As opções mais utilizadas são:

Opção	O que faz
<code>-l</code>	Formato detalhado: permissões, dono, tamanho, data de modificação
<code>-a</code>	Inclui arquivos ocultos (nomes começando com <code>.</code>) — veremos isso em detalhe adiante
<code>-h</code>	Tamanhos em formato legível (KB/MB/GB), combine com <code>-l</code>
<code>-t</code>	Ordena pela data de modificação (mais recente primeiro)
<code>-R</code>	Lista recursivamente, entrando em subdiretórios

As opções podem ser combinadas em uma única *flag*: `ls -lah` equivale a `ls -l -a -h`.

```
foo@bar$ ls -l
foo@bar$ ls -lah
```

Para consultar o manual de qualquer comando — incluindo a totalidade das opções por ele aceitas —, utilize `man`:

```
foo@bar$ man ls
```

O `man` abre o manual em um **paginador de texto** (o mesmo `less` a ser empregado mais adiante na leitura de arquivos grandes) — a tela permanece “tomada” pelo manual até que se saia dele, o que costuma gerar certa confusão em quem nunca o utilizou antes. A navegação básica dentro do `man` ocorre da seguinte forma:

- **setas** ↑/↓ ou **j/k** — rola uma linha por vez;
- **barra de espaço** — avança uma tela inteira; **b** — volta uma tela;
- **/palavra** seguido de **Enter** — busca **palavra** no texto; **n** repete a busca para a próxima ocorrência;
- **q** — sai do manual e volta ao prompt.

2.6.1 Repetindo comandos com history

O terminal armazena um histórico de tudo o que já foi digitado durante a sessão. Há duas formas de reaproveitar esse histórico:

- **seta** ↑ — traz de volta o último comando digitado (e ↓ avança novamente); trata-se da forma mais rápida de corrigir um comando que falhou por um pequeno detalhe;
- **history** — lista a totalidade do histórico de comandos, de forma numerada.

```
foo@bar$ history
```

💡 Atividade de fixação

A partir da sua pasta *home*, crie a estrutura `seml-workspace/lab-01/atividade-01` empregando três comandos `mkdir` separados (dica: `mkdir -p` cria todos os níveis de uma só vez — pesquise no `man mkdir` o que a opção `-p` realiza). Em seguida, utilizando **apenas caminhos relativos** (incluindo `./.`), navegue até `atividade-01` e retorne à sua pasta *home* por meio de um único comando `cd`. Por fim, execute `ls -lah` na sua pasta *home* e identifique: qual arquivo foi modificado mais recentemente?

2.7 Criando, movendo e removendo arquivos

Retomando o `mkdir`: quando empregado isoladamente, ele cria apenas um nível de diretório por vez, falhando caso os diretórios “pais” ainda não existam. A opção `-p` resolve essa limitação, criando toda a cadeia de diretórios necessária de uma só vez:

```
foo@bar$ mkdir sembl-workspace # cria só este nível
foo@bar$ mkdir -p sembl-workspace/lab-01/atividade-02 # cria os três níveis de uma vez
```

Para copiar, mover e remover arquivos, serão empregados três comandos, todos seguindo o padrão `comando origem destino` (com exceção de `rm`, que não possui “destino”):

cp (*copy*) copia um arquivo, preservando o original em seu lugar de origem:

```
foo@bar$ cp main.c main.c.bak
```

mv (*move*) move um arquivo para outro local — sendo, igualmente, a forma pela qual se renomeia um arquivo no Linux (não existe um comando `rename` separado: “renomear” corresponde, tão somente, a “mover para o mesmo diretório sob outro nome”):

```
foo@bar$ mv main.c.bak backup/
foo@bar$ mv relatorio-rascunho.txt relatorio-final.txt # isso é um "rename"
```

rm (*remove*) apaga um arquivo. Diferentemente de `cp/mv`, não recebe destino, apenas aquilo que deve ser apagado:

```
foo@bar$ rm backup/main.c.bak
```

Os três comandos aceitam opções em comum, capazes de alterar consideravelmente o comportamento de cada um:

Opção	Comando(s)	O que faz
-r	cp, rm	Recursivo: opera também dentro de subdiretórios. Necessário para copiar/remover um diretório inteiro
-i	cp, mv, rm	Interativo: pede confirmação antes de sobrescrever ou apagar algo
-f	rm	Força: remove sem pedir confirmação, e não reclama se o arquivo já não existir
-v	cp, mv, rm	Verboso: mostra na tela cada arquivo conforme é copiado/movido/removido

```
foo@bar$ cp -r projeto/ projeto-backup/ # copia o diretório projeto inteiro
foo@bar$ rm -rf build/                  # remove build/ e tudo dentro, sem perguntar
```

⚠ Warning

Diferentemente da lixeira do Windows/macOS, **o rm no Linux é permanente** — não existe, por padrão, qualquer forma de “restaurar”. O `rm -rf`, em particular, apaga sem solicitar confirmação e sem avisar caso algo não exista — é rápido de digitar e igualmente rápido de causar estrago, caso o caminho esteja incorreto. Na ausência de certeza absoluta, utilize `-i` para forçar uma confirmação antes de cada arquivo.

Para visualizar rapidamente o conteúdo de um arquivo extenso sem a necessidade de abrir um editor, utilize o paginador `less` (o mesmo empregado internamente pelo `man`):

```
foo@bar$ less arquivo-grande.log
```

(pressione q para sair)

💡 Atividade de fixação

Dentro de `atividade-01`, utilize `mkdir -p` para criar `rascunho/v1` e `rascunho/v2` em um único comando. Copie `arquivo1.c` para dentro de `rascunho/v1` empregando `-v` e observe a mensagem exibida. Por fim, remova o diretório `rascunho` inteiro com `rm -r`, primeiramente acompanhado de `-i` (confirmando cada remoção individualmente) — recrie, então, a estrutura e remova-a novamente, desta vez com `-rf`, sem qualquer confirmação.

2.7.1 Wildcards (curingas)

O caractere `*` substitui qualquer sequência de caracteres, ao passo que `?` substitui exatamente um único caractere. Ambos revelam-se extremamente úteis para operar sobre diversos arquivos simultaneamente:

```
foo@bar$ rm *.o          # remove todos os arquivos objeto
foo@bar$ ls *.c          # lista todos os arquivos .c
```

💡 Atividade de fixação

Dentro de `atividade-01`, crie três arquivos vazios com `touch arquivo1.c arquivo2.c notas.txt`. Em seguida, utilize um único comando com *wildcard* para listar apenas os arquivos `.c`, e outro comando com *wildcard* para copiar todos eles (`.c` e `.txt`) para um novo subdiretório `backup/`.

2.8 Editores de texto no terminal

Até este ponto, criamos apenas arquivos vazios (`touch`) ou copiamos conteúdo já existente — em algum momento, contudo, será necessário **editar** um arquivo sem sair do terminal. Essa necessidade surge com frequência maior do que se poderia supor: editar um arquivo de configuração que exige `sudo`, ajustar rapidamente um *script*, ou escrever a mensagem de um *commit* do Git.

Ao longo do curso, o **VS Code** (conectado ao WSL por meio do Remote-WSL, conforme apresentado no roteiro de [Ambiente e Ferramentas](#)) constituirá o editor principal para a escrita de código. Vale a pena, entretanto, conhecer dois editores capazes de operar **dentro do próprio terminal**, sem depender de interface gráfica alguma — essenciais tanto para editar algo diretamente de dentro do WSL quanto para o dia em que se estiver trabalhando em um servidor remoto totalmente carente de interface gráfica.

2.8.1 nano: simples e direto

O **nano** é o editor recomendado para quem está começando: não possui modos distintos (basta digitar para que o texto apareça, sem a necessidade de “entrar” ou “sair” de qualquer estado), e exibe os atalhos disponíveis fixados na parte inferior da tela.

```
foo@bar$ nano arquivo.txt
```

Os atalhos mais usados (o símbolo ^ significa **Ctrl**):

Atalho	O que faz
Ctrl+O	Salva o arquivo (<i>write out</i>) — pede para confirmar o nome, pressione Enter
Ctrl+X	Sai do nano (pede para salvar antes, se houver alterações não salvas)
Ctrl+K	Recorta a linha atual
Ctrl+U	Cola a última linha recortada
Ctrl+W	Busca um trecho de texto no arquivo

2.8.2 vim: onipresente, porém exigente

O **vim** encontra-se instalado por padrão em praticamente qualquer sistema Linux/Unix que se venha a encontrar — inclusive em servidores remotos nos quais não haja a possibilidade de instalar o **nano**. Por essa razão, ainda que não venha a ser adotado como editor principal, vale a pena conhecer o mínimo necessário para **sobreviver** dentro dele.

A diferença fundamental reside no fato de que o **vim** é **modal**: existe um modo destinado à navegação e a comandos (*Normal*) e outro destinado à digitação de texto (*Insert*), sendo necessário alternar entre eles de forma consciente — diferentemente do **nano**, no qual basta digitar.

```
foo@bar$ vim arquivo.txt
```

O roteiro mínimo de sobrevivência é o seguinte:

1. Ao abrir o arquivo, você se encontra em modo **Normal** (nenhum texto é digitado ainda; cada tecla corresponde a um comando);
2. Pressione **i** para ingressar no modo **Insert** — a partir desse momento, o que for digitado passa a constituir texto no arquivo;
3. Pressione **Esc** para retornar ao modo Normal;
4. Em modo Normal, digite **:wq** e pressione Enter para **salvar e sair** (*write + quit*);
5. Caso se deseje sair **sem salvar**, em modo Normal digite **:q!** e pressione Enter.

i Note

“Como eu saio do vim?” constitui piada recorrente entre programadores — é fácil permanecer presos nele na primeira tentativa. Caso isso ocorra, pressione **Esc** algumas vezes para garantir que se está em modo Normal, e digite, em seguida, **:q!** seguido de Enter.

Para os propósitos deste curso, o **nano** é suficiente para qualquer edição rápida realizada no terminal — o **vim** é apresentado aqui apenas para o dia em que se esbarrar nele (o Git, por exemplo, abre o **vim** por padrão para a escrita de mensagens de *commit* mais longas) e for necessário sair dele com vida.

💡 Atividade de fixação

Dentro de `atividade-01`, abra `notas.txt` com **nano**, escreva uma linha de texto qualquer, salve e saia (**Ctrl+O**, **Enter**, **Ctrl+X**). Confirme o conteúdo com `cat notas.txt`. Em seguida, abra o mesmo arquivo com **vim**, ingresse no modo Insert (**i**), acrescente uma segunda linha, retorne ao modo Normal (**Esc**) e salve e saia com `:wq`.

2.9 Buscando arquivos e conteúdo

Os *wildcards* resolvem bem a situação em que já se sabe *onde* o arquivo está e apenas se precisa filtrá-lo por nome, dentro do diretório atual. Duas ferramentas resolvem problemas de escopo mais amplo: localizar um arquivo em qualquer ponto da árvore, ou localizar um trecho de texto dentro do conteúdo dos arquivos.

2.9.1 find: buscando arquivos pela árvore

O **find** procura arquivos de forma recursiva, percorrendo todos os subdiretórios a partir de um ponto de partida — ao contrário do *wildcard*, que enxerga apenas o diretório atual.

```
foo@bar$ find . -name "*.c"          # busca arquivos .c a partir do diretório atual
foo@bar$ find /home -name "main.c"  # busca "main.c" a partir de /home
```

O primeiro argumento indica **onde** procurar (`.` para “aqui”, ou um caminho absoluto), enquanto `-name` filtra pelo nome (aceitando *wildcards*). O **find** dispõe de dezenas de outros filtros (por tamanho, por data de modificação, por tipo) — consulte `man find` sempre que algo mais específico se fizer necessário.

2.9.2 grep: buscando texto dentro dos arquivos

Enquanto o **find** localiza **arquivos**, o **grep** localiza **texto dentro do conteúdo** de um ou mais arquivos — recurso extremamente útil, por exemplo, para descobrir em qual arquivo `.c` do projeto uma determinada função foi declarada.

```
foo@bar$ grep "main" arquivo.c      # procura a palavra "main" em arquivo.c
foo@bar$ grep -r "main" .            # -r: busca recursivamente em todos os
arquivos a partir daqui
foo@bar$ grep -n "main" arquivo.c    # -n: mostra o número da linha de
cada ocorrência
```

O **grep** constitui, também, o exemplo clássico de ferramenta concebida para ser **combinada com outras por meio de pipe**, como será demonstrado a seguir.

💡 Atividade de fixação

Dentro de `atividade-01`, escreva a palavra `TODO` em algum ponto do conteúdo de `arquivo1.c` (dica: `echo "// TODO: revisar" >> arquivo1.c` acrescenta essa linha ao arquivo). Em seguida, utilize **grep** para confirmar em qual arquivo aquela palavra aparece, e utilize **find** para localizar o arquivo `notas.txt` a partir da sua pasta *home*, sem saber de antemão que ele se encontrava dentro de `atividade-01`.

2.10 Superusuário e gerenciamento de pacotes

Sistemas Unix (dos quais o Linux faz parte) controlam de forma minuciosa quem está autorizado a acessar cada arquivo, dispositivo ou recurso do sistema. Ações administrativas — como a instalação de programas — exigem privilégio de superusuário (administrador). Para tanto, emprega-se o comando **sudo** (*super user do*), que solicita a senha do usuário e executa **apenas o comando imediatamente seguinte** com

privilégios elevados — em contraposição à abertura de uma sessão inteira como *root* (*su*), o que se revela desnecessário e mais arriscado para o uso cotidiano.

O gerenciador de pacotes do Ubuntu é o **apt** (*Advanced Packaging Tool*). Os comandos mais empregados são:

```
foo@bar$ sudo apt update    # atualiza a lista de pacotes disponíveis
foo@bar$ sudo apt upgrade   # baixa e instala atualizações dos programas já instalados
foo@bar$ sudo apt install <pacote> # instala um novo programa
```

2.11 Tipos de arquivo e permissões

No Linux, praticamente tudo é representado como um arquivo — inclusive dispositivos de hardware, a exemplo do gravador ST-LINK, que será empregado na programação do microcontrolador. Ao listar um diretório com `ls -l`, a primeira coluna exibe o tipo do arquivo e suas permissões, como no exemplo a seguir:

```
crw----- 1 root root 189, 2 jan 1 00:00 /dev/bus/usb/001/003
```

O **primeiro caractere** indica o tipo do arquivo:

- **-** — arquivo comum
- **d** — diretório
- **l** — link simbólico
- **c** — dispositivo do tipo caractere (como o ST-LINK)
- **b** — dispositivo do tipo bloco
- **p** — *pipe* (FIFO)
- **s** — *socket*

Os **9 caracteres seguintes** organizam-se em três grupos de três (*rwX*), indicando permissão de leitura (*read*), escrita (*write*) e execução (*execute*) — respectivamente para o **dono** do arquivo, o **grupo** ao qual ele pertence, e **todos os demais** usuários. Um **-** no lugar da letra indica que aquela permissão específica não foi concedida.

No exemplo acima, `crw-----` significa o seguinte: trata-se de um dispositivo de caractere, e **apenas o usuário root** possui permissão de leitura e escrita — nenhum outro usuário, seja do grupo, seja de “todos os demais”, possui qualquer acesso. É exatamente esse tipo de restrição que precisaremos contornar (por meio de regras *udev*) para que o usuário comum consiga acessar o gravador ST-LINK, conforme será demonstrado no roteiro de [Ambiente e Ferramentas de Desenvolvimento](#).

2.11.1 Arquivos ocultos

Arquivos e diretórios cujo nome se inicia por **.** (ponto) permanecem ocultos por padrão — trata-se, geralmente, de arquivos de configuração que não devem poluir uma listagem comum. Utilize a opção `-a` do `ls` para revelá-los:

```
foo@bar$ ls -a
foo@bar$ ls -la    # combina com -l para ver também tipo/permissões
```

Atividade de fixação

Execute `ls -la` na sua pasta *home*. Localize um arquivo oculto (nome iniciado por **.**) e identifique, a partir das letras *rwX*, se você (o dono) possui permissão de escrita nele. Em seguida, execute `ls -l /dev/null` e interprete o resultado: de que tipo de arquivo se trata, e quem possui permissão de leitura e escrita?

2.12 Compondo comandos com pipes

Um dos recursos mais poderosos do terminal é o **pipe** (`|`): ele conecta a saída de um comando diretamente à entrada do comando seguinte, permitindo combinar ferramentas simples na resolução de tarefas complexas — sem que seja necessário escrever um programa novo para cada combinação.

```
foo@bar$ ls -la | sort | less
```

O comando acima lista os arquivos (`ls -la`), ordena o resultado (`sort`) e o envia ao paginador (`less`) — três ferramentas independentes, cada uma cumprindo bem uma única função, combinadas em uma única linha. Essa filosofia — programas pequenos, combinados entre si, em contraposição a uma ferramenta monolítica que pretende fazer tudo — é exatamente a mesma que fundamenta a preferência, neste curso, por escrever nosso próprio `Makefile` em vez de recorrer a uma IDE que oculta os passos de compilação: compreender e combinar as peças, em vez de depender de uma caixa-preta que realiza tudo de uma só vez.

O `grep`, apresentado na seção anterior, constitui o exemplo mais comum de comando concebido para figurar do lado direito de um *pipe* — filtrando a saída de outro comando:

```
foo@bar$ history | grep gcc      # entre tudo que já digitei, o que envolveu "gcc"?
foo@bar$ ls -la | grep ".c"     # entre os arquivos listados, quais têm ".c"
no nome?
```

💡 Atividade de fixação

Na sua pasta `atividade-01`, utilize um *pipe* para contar quantos arquivos `.c` existem no diretório, combinando `ls *.c` com o comando `wc -l` (dica: `wc -l` conta as linhas da entrada que recebe). Em seguida, reproduza o mesmo resultado por outro caminho, combinando `ls -la` com `grep` e `wc -l` em um único *pipe* de três comandos.

2.13 Redirecionamento e encadeamento de comandos

O *pipe* conecta a saída de um comando à entrada do seguinte. Existem, porém, outras formas de direcionar a saída de um programa e de combinar comandos numa mesma linha — recursos empregados com frequência nas práticas a partir da próxima semana.

2.13.1 Redirecionando a saída para um arquivo

O operador `>` envia a saída de um comando para um arquivo, em vez de exibi-la na tela. Se o arquivo já existir, o conteúdo anterior será substituído; se não existir, será criado:

```
foo@bar$ ls -la > listagem.txt
foo@bar$ cat listagem.txt
```

O operador `>>` faz o mesmo, porém **acrescenta** ao final do arquivo, sem apagar o que já existia:

```
foo@bar$ echo "// TODO: revisar" >> arquivo1.c
```

2.13.2 Saída padrão, saída de erro e `2>&1`

Todo programa no Linux possui duas saídas distintas: a **saída padrão** (`stdout`, identificada internamente pelo número 1), por onde saem os resultados normais, e a **saída de erro** (`stderr`, identificada pelo número 2), por onde saem mensagens de erro e avisos. Quando se digita um comando e as duas saídas aparecem misturadas na tela, é apenas porque ambas estão sendo exibidas no mesmo terminal — mas são canais separados.

Essa distinção se torna visível ao redirecionar: `comando > arquivo.txt` captura apenas o `stdout`; mensagens de erro continuam aparecendo na tela, porque o `stderr` não foi redirecionado. Para capturar **tudo** — resultados e erros — em um único destino, emprega-se a notação `2>&1`, que significa “redirecione o canal 2 (erro) para onde quer que o canal 1 (saída padrão) esteja indo”:

```
foo@bar$ gcc programa.c -o programa 2>&1 | less
```

No exemplo acima, tanto os avisos do compilador (`stderr`) quanto qualquer saída normal (`stdout`) são enviados ao `less`, permitindo inspecionar tudo com calma. Sem o `2>&1`, apenas a saída padrão passaria pelo *pipe*, e os avisos continuariam rolando diretamente na tela.

2.13.3 Encadeando comandos com ; e o código de saída \$?

O **ponto e vírgula (;)** permite escrever dois ou mais comandos na mesma linha. Cada comando é executado **sequencialmente**, um após o outro, independentemente de o anterior ter sido bem-sucedido ou não:

```
foo@bar$ mkdir teste; cd teste; pwd
```

A utilidade mais imediata do ; é combinar um programa com a inspeção do seu resultado. Todo programa, ao encerrar, devolve ao *shell* um número inteiro chamado **código de saída** (*exit status*): **0** indica sucesso, e qualquer valor diferente de zero indica algum tipo de falha. Esse valor fica armazenado no **parâmetro especial \$?** — não se trata de uma variável de ambiente (assunto da próxima seção), e sim de um valor mantido internamente pelo próprio *shell*, acessível apenas por leitura:

```
foo@bar$ ls /tmp; echo $?
0
foo@bar$ ls /pasta-inexistente; echo $?
ls: cannot access '/pasta-inexistente': No such file or directory
2
```

No primeiro caso, o `ls` encontrou o diretório e retornou 0 (sucesso); no segundo, falhou e retornou 2. O ; garante que o `echo $?` seja executado em seguida, exibindo o código de saída do comando anterior. Esse recurso será empregado nas práticas para verificar se um programa compilado no PC terminou corretamente — por exemplo, `./programa; echo $?` — e, mais adiante, para entender o papel do código de retorno do `main()` em C.

💡 Atividade de fixação

Execute `ls /tmp > /dev/null; echo $?` e, em seguida, `ls /pasta-que-nao-existe > /dev/null 2>&1; echo $?`. Explique por que o primeiro retorna 0 e o segundo retorna um valor diferente de zero. Por que foi necessário acrescentar `2>&1` no segundo caso para que a mensagem de erro também fosse redirecionada?

2.14 Filtros de texto

O terminal do Linux segue uma filosofia de ferramentas pequenas e especializadas, cada uma realizando bem uma única tarefa, combinadas por *pipes* para resolver problemas maiores. Os comandos a seguir constituem os filtros de texto mais empregados nas práticas deste curso.

2.14.1 cat — exibir o conteúdo inteiro

O **cat** (*concatenate*) exibe na tela o conteúdo completo de um ou mais arquivos. É a forma mais rápida de verificar o que há dentro de um arquivo curto:

```
foo@bar$ cat notas.txt
foo@bar$ cat arquivo1.c arquivo2.c      # exibe os dois, um após o outro
```

2.14.2 head e tail — ver o início ou o final

Quando o arquivo é extenso, examinar apenas as primeiras ou as últimas linhas é mais prático do que abri-lo inteiro. O **head** exibe as primeiras linhas e o **tail** exibe as últimas; a opção **-n** controla quantas:

```
foo@bar$ head -n 20 programa.i          # as 20 primeiras linhas do arquivo pré-processado
foo@bar$ tail -n 10 programa.s          # as 10 últimas linhas do arquivo assembly
```

Ambos aceitam *pipe*, o que permite extrair uma fatia de linhas da saída de outro comando:

```
foo@bar$ cat programa.i | head -n 30    # equivalente a "head -n 30 programa.i"
```

2.14.3 **wc** — contar linhas, palavras e bytes

O **wc** (*word count*) conta linhas, palavras e bytes. A opção **-l** restringe a contagem apenas às linhas — forma habitual de medir o tamanho de um arquivo ou de contar a quantidade de resultados produzidos por outro comando:

```
foo@bar$ wc -l programa.i           # quantas linhas tem o arquivo pré-processado?
foo@bar$ ls *.c | wc -l             # quantos arquivos .c existem neste diretório?
```

2.14.4 **tr** — substituir ou eliminar caracteres

O **tr** (*translate*) opera sobre caracteres individuais, substituindo cada ocorrência de um caractere por outro. Diferentemente do **grep** e do **sed**, o **tr** não trabalha com nomes de arquivo — ele lê exclusivamente da entrada padrão (via *pipe* ou redirecionamento):

```
foo@bar$ echo "Hello, World!" | tr 'A-Z' 'a-z'
hello, world!
foo@bar$ echo "linha1:linha2:linha3" | tr ':' '\n'
linha1
linha2
linha3
```

No segundo exemplo, cada **:** é substituída por uma quebra de linha (**\n**), separando os campos em linhas individuais — recurso útil para formatar saídas de ferramentas que separam dados por dois-pontos.

2.14.5 **diff** — comparar dois arquivos

O **diff** exibe as diferenças entre dois arquivos, linha por linha. Linhas precedidas de **<** existem apenas no primeiro arquivo; linhas precedidas de **>** existem apenas no segundo:

```
foo@bar$ diff original.c modificado.c
```

Quando os dois arquivos são idênticos, o **diff** não produz saída alguma. É a forma padrão de verificar, no terminal, se duas versões de um arquivo coincidem — recurso empregado, por exemplo, para conferir se o código digitado pelo estudante corresponde ao gabarito esperado.

Atividade de fixação

Crie dois arquivos: `echo -e "linha1\nlinha2\nlinha3" > a.txt` e `echo -e "linha1\nlinha2-modificada\nlinha3" > b.txt`. Execute `diff a.txt b.txt` e identifique qual linha difere. Em seguida, conte quantas linhas tem cada arquivo com `wc -l` e exiba apenas a última linha de `b.txt` com `tail -n 1`.

2.15 Identificando e inspecionando arquivos

Os comandos anteriores operam sobre texto. No desenvolvimento de *firmware*, contudo, boa parte dos arquivos produzidos pelo compilador e pelo *linker* são **binários** — sequências de bytes sem representação textual legível. Dois utilitários do sistema (não das ferramentas de compilação) permitem investigar esses arquivos por fora, sem precisar conhecer a fundo o seu formato interno.

2.15.1 **file** — descobrir o tipo real de um arquivo

O **file** examina os primeiros bytes de um arquivo — os chamados *magic numbers*, uma sequência fixa de bytes que identifica o formato — e informa o tipo real do conteúdo, independentemente do nome ou da extensão:

```
foo@bar$ file notas.txt
notas.txt: ASCII text
foo@bar$ file /bin/ls
/bin/ls: ELF 64-bit LSB pie executable, x86-64, ...
```

Nas práticas de compilação, o `file` será empregado para confirmar o tipo de cada arquivo produzido em cada etapa — por exemplo, distinguir um objeto relocável de um executável, ou verificar se o compilador realmente gerou código para ARM em vez de x86:

```
foo@bar$ file programa.o
programa.o: ELF 64-bit LSB relocatable, x86-64, ...
foo@bar$ file programa
programa: ELF 64-bit LSB executable, x86-64, ...
```

A diferença entre `relocatable` e `executable` na saída do `file` reflete exatamente a diferença entre um arquivo objeto (que ainda precisa ser ligado) e um executável pronto — a mesma distinção que será examinada em detalhe no capítulo sobre o fluxo de compilação.

2.15.2 `od` — inspecionar o conteúdo binário em hexadecimal

O `od` (*octal dump*) exibe o conteúdo bruto de qualquer arquivo em formato numérico — por padrão em octal, mas a combinação de opções mais útil para o nosso curso produz uma saída em **hexadecimal com palavras de 32 bits**, que corresponde exatamente à largura de dado do processador ARM Cortex-M:

```
foo@bar$ od -A x -t x4 -N 8 arquivo.bin
000000 20020000 08000041
000008
```

As opções, uma a uma:

Opção	Significado
<code>-A x</code>	Exibe os endereços (coluna da esquerda) em hexadecimal, em vez de octal
<code>-t x4</code>	Formata os dados como palavras de 4 bytes (32 bits) em hexadecimal
<code>-N 8</code>	Lê apenas os primeiros 8 bytes do arquivo
<code>-j 0x100</code>	(opcional) Pula os primeiros 0x100 bytes antes de começar a ler

Na prática de código de inicialização e *linker script*, esse comando será empregado para inspecionar a imagem binária que será gravada na Flash do microcontrolador — por exemplo, para confirmar que as duas primeiras palavras da tabela de vetores contêm o valor inicial do ponteiro de pilha e o endereço do tratador de reset. Um exemplo real dessa inspeção:

```
foo@bar$ od -A x -t x4 -N 8 embedded.bin
000000 20020000 08000041
000008
```

A primeira palavra (`20020000`) é o topo da pilha (SRAM do STM32F411) e a segunda (`08000041`) é o endereço do tratador de reset na Flash, com o bit 0 ligado para indicar código Thumb — exatamente o que o processador espera encontrar ali ao sair do estado de reset.

💡 Atividade de fixação

Execute `file /bin/ls` e interprete a saída: de que tipo é o arquivo, para qual arquitetura, e é executável ou relocável? Em seguida, crie um pequeno arquivo binário com `echo -ne '\x7fELF' > teste.bin` e execute `file teste.bin` — o que o `file` identifica, e de onde vem essa informação (dica: são os *magic numbers*)? Por fim, execute `od -A x -t x1 -N 4 teste.bin` e confira que os quatro bytes correspondem à sequência `7f 45 4c 46` (as letras `ELF` em ASCII, precedidas pelo byte `0x7f`).

2.16 Variáveis de ambiente e o `PATH`

Todo processo em execução no Linux carrega consigo um conjunto de **variáveis de ambiente** — pares `NOME=valor` que o sistema operacional e os programas utilizam para configurar seu comportamento. Diferentemente dos parâmetros especiais do *shell* (como o `$?` da seção anterior, mantidos internamente pelo interpretador), as variáveis de ambiente são herdadas por todos os programas que o *shell* vier a lançar.

2.16.1 Consultando variáveis de ambiente

Para exibir o valor de uma variável, emprega-se `echo` com o prefixo `$`:

```
foo@bar$ echo $HOME
/home/usuario
foo@bar$ echo $USER
usuario
```

Para listar todas as variáveis de ambiente definidas na sessão atual, utilize `env` ou `printenv`:

```
foo@bar$ env
```

A saída é extensa — dezenas de variáveis — mas algumas aparecem com frequência:

Variável	O que contém
HOME	Caminho da pasta pessoal do usuário (<code>/home/usuario</code>)
USER	Nome do usuário atual
PATH	Lista de diretórios onde o <i>shell</i> procura os programas executáveis
SHELL	Caminho do <i>shell</i> em uso (geralmente <code>/bin/bash</code>)
PWD	Diretório atual de trabalho (o mesmo que <code>pwd</code> exibe)

2.16.2 O PATH: onde o *shell* encontra os programas

Quando se digita um comando como `gcc` ou `arm-none-eabi-gcc`, o *shell* não sabe, de antemão, onde esse programa está instalado no disco. Ele consulta a variável **PATH**, que contém uma lista de diretórios separados por dois-pontos (`:`), e percorre cada um deles, na ordem, até encontrar um executável com o nome correspondente:

```
foo@bar$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
```

Se o programa não se encontra em nenhum dos diretórios listados, o *shell* exibe a mensagem `command not found` — mesmo que o programa exista em outra pasta do sistema. É por essa razão que, no roteiro de [Ambiente e Ferramentas de Desenvolvimento](#), a instalação do *toolchain* ARM deve colocar os executáveis em um diretório que já pertença ao **PATH** (como `/usr/bin`), ou então acrescentar o diretório onde eles foram instalados ao **PATH**.

2.16.3 Criando e exportando variáveis

Uma variável comum, criada por atribuição direta, existe apenas no *shell* atual — ela **não** é transmitida aos programas lançados a partir dele:

```
foo@bar$ MINHA_VAR="teste"
foo@bar$ echo $MINHA_VAR
teste
```

Para que a variável seja herdada pelos programas filhos (inclusive por outro *shell* aberto a partir deste), emprega-se **export**:

```
foo@bar$ export MINHA_VAR="teste"
```

Na prática, a combinação mais comum é acrescentar um diretório ao **PATH**:

```
foo@bar$ export PATH="$PATH:/caminho/para/novas/ferramentas"
```

Essa alteração vale apenas para a sessão atual do terminal. Para torná-la permanente, o comando `export` deve ser acrescentado ao arquivo de configuração do *shell* — geralmente `~/.bashrc` — que é lido automaticamente cada vez que um novo terminal é aberto.

i Note

Nas práticas deste curso, as variáveis de ambiente aparecem em dois contextos recorrentes: o `PATH`, que precisa incluir o diretório do *toolchain* ARM, e a explicação do *runtime* em C, na qual o `main()` recebe, além de `argc` e `argv`, o ponteiro para o bloco de variáveis de ambiente preparado pelo código de inicialização (`crt0`). No microcontrolador *bare-metal* não existe sistema operacional para fornecer essas variáveis — razão pela qual o `main()` do *firmware* é declarado simplesmente como `int main(void)`.

? Atividade de fixação

Execute `echo $PATH` e identifique os diretórios listados. Em seguida, descubra onde está instalado o programa `gcc` com o comando `which gcc` — o diretório que ele indica deveria constar na lista do `PATH`. Por fim, crie uma variável `CURSO="SEMB1"`, exiba-a com `echo $CURSO`, e verifique (com `env | grep CURSO`) se ela aparece entre as variáveis de ambiente. Repita após `export CURSO` e observe a diferença.

2.17 Onde isso se aplica no curso

Esses conceitos não constituem mera “cultura geral” — serão empregados diretamente ao longo do curso:

- `cd/mkdir/ls`/caminhos relativos, para organizar cada atividade em seu próprio diretório dentro de `sembl-workspace`;
- `cp/mv/rm/wildcards`, para gerenciar os arquivos de código-fonte e objeto gerados durante a compilação;
- `sudo apt install`, para instalar cada ferramenta do roteiro de ambiente;
- `nano`, para editar rapidamente as regras `udev` a serem copiadas para `/etc/udev/rules.d`, exatamente no roteiro de [Ambiente e Ferramentas de Desenvolvimento](#);
- permissões de arquivo, para compreender por que o ST-LINK “não aparece” até que essas regras sejam configuradas corretamente;
- `find/grep`, para localizar arquivos e trechos de código em projetos com dezenas de arquivos-fonte — recursos cada vez mais úteis conforme os laboratórios crescem em tamanho;
- `>, 2>&1` e `;`, para redirecionar saídas do compilador, combinar um programa com `echo $?` e inspecionar erros de ligação;
- `head/tail/wc -l`, para examinar trechos de arquivos pré-processados e *assembly* com milhares de linhas;
- `file`, para confirmar o tipo e a arquitetura de cada arquivo objeto e executável produzido pelo *toolchain*;
- `od`, para inspecionar diretamente os bytes da imagem binária gravada na Flash do microcontrolador;
- `PATH` e variáveis de ambiente, para entender por que o *shell* encontra (ou não encontra) o `arm-none-eabi-gcc` e como o *runtime* em C prepara o ambiente antes do `main()`.

2.18 Leitura complementar

The Linux Command Line, de William Shotts

linuxcommand.org/tlcl.php — livro gratuito (PDF, licença Creative Commons BY-NC-ND), em inglês.

Trata-se de um curso completo de linha de comando, escrito para quem está começando e organizado como uma sequência de experimentos no terminal. O capítulo *Redirection* aprofunda os operadores de redirecionamento e os *pipes* apresentados aqui, o capítulo *The Environment* detalha o funcionamento interno das variáveis de ambiente e do `~/ .bashrc`, o capítulo *Regular Expressions* explora o `grep` com expressões regulares avançadas e o capítulo *Processing Text* apresenta ferramentas adicionais como `sort`, `uniq`, `cut` e `sed`. Não deve ser lido de uma só vez: utilize-o como fonte de consulta sempre que um comando das práticas não estiver suficientemente claro.

Expressões Regulares — Guia de Consulta Rápida, de Aurelio Marinho Jargas

aurelio.net/regex/guia — em português, gratuito e completo na internet (Novatec).

Guia curto e direto acerca de expressões regulares, os padrões de busca empregados pelo `grep`, pelo `sed` e pelos editores de código, contendo tabelas com a totalidade dos metacaracteres.

2.19 Questionário de revisão

Verifique o que foi retido antes de seguir para o próximo capítulo. Preencha ou selecione cada resposta e clique em “**Show Answers**” ao final para conferir.

1. O comando que mostra o caminho completo do diretório atual é
2. Qual opção do `ls` exibe também os arquivos ocultos? `-l-a-R`
3. `Sample.txt` e `sample.txt` são o mesmo arquivo no Linux. VerdadeiroFalso
4. O atalho que representa “o diretório pai” (um nível acima) é
5. Qual comando busca um trecho de **texto dentro do conteúdo** de arquivos? `findgrep`
6. Qual opção do `rm` remove sem pedir confirmação, mesmo que o arquivo não exista? `-i-f-v`
7. O `sudo` abre uma sessão permanente como superusuário, válida para todos os comandos seguintes até você sair do terminal. VerdadeiroFalso
8. Para sair do `vim` **sem salvar** as alterações, em modo Normal você digita
9. Dentro do paginador do `man` (ou do `less`), qual tecla você pressiona para sair? `qxEsc`
10. O caractere que conecta a saída de um comando à entrada do próximo é
11. O operador `>` redireciona a saída de um comando para um arquivo. Qual operador **acrescenta** ao final do arquivo, sem apagar o conteúdo anterior? `>[‘ size=’4’/>`
12. A notação `2>&1` redireciona a entrada padrão para o canal 2a saída de erro (`stderr`) para onde a saída padrão (`stdout`) está indo a saída padrão para um arquivo de erro
13. Quando um programa termina com sucesso, o código de saída armazenado em `$?` é
14. O comando que exibe o conteúdo bruto de um arquivo em formato hexadecimal é `filehexdump`
15. A variável de ambiente que contém a lista de diretórios onde o *shell* procura programas executáveis é

II

Semana 2 — Arquitetura ARM Cortex-M e Fluxo de Desenvolvimento

3	Introdução aos Processadores ARM Cortex-M	53
3.1	A ARM e a evolução da sua arquitetura	54
3.2	Processador, núcleo e microcontrolador	56
3.3	Harvard ou Von Neumann?	56
3.4	RISC ou CISC?	57
3.5	Os conjuntos de instruções A32, T16 e T32	57
3.6	O modelo de programação	60
3.7	O sistema de memória	65
3.8	Exceções e interrupções	67
3.9	Depuração: JTAG e SWD	70
3.10	A sequência de reset	70
3.11	Questionário de revisão	70
3.12	Referências	71
4	Prática: Do C ao Binário	73
4.1	Do código-fonte ao executável	74
4.2	Parte 1 — Quanto vale <code>g_a + g_b</code> ?	76
4.3	Parte 2 — E no Cortex-M4?	77
4.4	Referências	84

3. Introdução aos Processadores ARM Cortex-M

i Objetivos de aprendizagem

Ao final deste capítulo, você deverá ser capaz de:

- situar o Cortex-M na evolução da arquitetura ARM e entre os perfis A, R e M;
- explicar por que o Cortex-M é uma arquitetura **Harvard modificada** — barramentos separados, mas um único mapa de memória, como na de Von Neumann;
- classificar o conjunto de instruções do Cortex-M como RISC e reconhecer as concessões que ele faz em nome da densidade de código;
- comparar os conjuntos de instruções **A32**, **T16** e **T32** (Thumb-2), com vantagens e desvantagens de cada um;
- descrever os registradores de uso geral e de uso específico, os registradores especiais, os modos de operação e os níveis de privilégio;
- descrever o mapa de memória, a pilha, o mecanismo de exceções e a sequência de reset do Cortex-M.

No capítulo anterior, estudamos a arquitetura de computadores de forma genérica: o modelo de Von Neumann e a arquitetura Harvard, a distinção entre CISC e RISC, a regra *Load-Store*, a hierarquia de memória e o papel dos registradores e da pilha. Neste capítulo, aplicamos esses conceitos a um processador real — o **ARM Cortex-M4**, núcleo do STM32F411 que será utilizado durante todo o curso. Começamos pela história da arquitetura ARM, que explica boa parte das escolhas de projeto do Cortex-M, e em seguida percorremos o seu **modelo de programação**: a forma como o processador se apresenta a quem escreve software — seus modos de operação, registradores, organização de memória, tratamento de exceções e sequência de reset.

Não se trata de memorizar o manual da arquitetura (o *ARMv7-M Architecture Reference Manual* possui mais de mil páginas e destina-se a projetistas de processadores e de compiladores). Para programar um microcontrolador Cortex-M, basta um entendimento sólido do modelo de programação, do mapa de memória e do mecanismo de exceções — e é exatamente isso que este capítulo se propõe a oferecer. Cada um dos tópicos aqui apresentados reaparecerá, em código, mais adiante no livro, no momento em que escrevermos o nosso próprio `startup.c`.

🔗 Livro-Texto Oficial: Leitura de Aprofundamento no Joseph Yiu

Este capítulo baseia-se diretamente nos conceitos fundamentais apresentados no livro-texto da disciplina: **Joseph Yiu — *The Definitive Guide to ARM® Cortex®-M3 and Cortex®-M4 Processors*, 3ª edição (Newnes/Elsevier, 2013).**

Para consolidar o aprendizado, consultar os diagramas completos de barramentos internos e estudar o detalhamento de cada registrador, recomendamos fortemente a leitura das seguintes seções:

- **Capítulo 3 (Technical Overview):** Seções 3.1 (*General information*) e 3.2 (*Features of the Cortex-M3 and Cortex-M4 processors*), pp. 57–74 — Arquitetura de barramentos (I-Code, D-Code, System), matriz de barramentos e diferenciais do Cortex-M4.
- **Capítulo 4 (Architecture):** Seções 4.1 (*Introduction*), 4.2 (*Programmer's model*), 4.3 (*Behavior of the APSR*), 4.4 (*Memory system*), 4.5 (*Exceptions and interrupts*) e 4.8 (*Reset and reset sequence*), pp. 75–108 e 113–116 — Registradores R0–R15, dual stack (MSP/PSP), flags de status, mapa de memória, tabela de vetores e sequência física de inicialização.
- **Capítulo 5 (Instruction Set):** Seção 5.6.2 (*Memory access instructions*), pp. 134–146 — modos de endereçamento, acessos de byte e meia-palavra e *literal pool*.
- **Capítulo 6 (Memory System):** Seções 6.8 (*Default memory access permissions*) e 6.9 (*Memory access attributes*), pp. 217–220 — tipos de memória Normal, Device e Strongly-ordered, *buffer* de escrita e *Execute Never*.

3.1 A ARM e a evolução da sua arquitetura

A ARM é uma empresa britânica, fundada em 1990, reconhecida pelo desenvolvimento de arquiteturas de processadores eficientes e de baixo consumo de energia. Conforme já discutido na Apresentação do curso, a ARM não fabrica chips: ela **licencia** suas arquiteturas e núcleos a fabricantes de semicondutores — ST, NXP, Texas Instruments, Nordic, Microchip, entre dezenas de outros —, que os incorporam a seus próprios produtos. Essa estratégia tornou a arquitetura ARM a mais amplamente utilizada do mundo em dispositivos móveis e sistemas embarcados.

As origens da arquitetura, contudo, são anteriores à própria empresa. O primeiro processador ARM, o **ARM1** (arquitetura ARMv1), foi lançado em 1985 pela Acorn Computers, com foco em computadores pessoais como o Acorn Archimedes — e não em sistemas embarcados. As versões subsequentes, **ARMv2** e **ARMv3**, acrescentaram instruções de multiplicação e de multiplicação-acumulação, suporte a coprocessadores e um espaço de endereçamento completo de 32 bits.

3.1.1 ARMv4 e o conjunto de instruções Thumb

A **ARMv4** representou o ponto de inflexão da arquitetura em direção aos sistemas embarcados e dispositivos móveis. Além de operações de leitura e escrita de meias-palavras (*halfword*, 16 bits), essa versão introduziu **modos de execução privilegiados**, fundamentais para a operação de sistemas operacionais: o *kernel* passa a exercer controle total sobre os recursos do sistema, ao passo que os aplicativos do usuário são executados com privilégios reduzidos.

Sua variante **ARMv4T** introduziu o conjunto de instruções **Thumb**, marco que tornou a arquitetura ARM dominante em sistemas embarcados. O Thumb foi concebido com o propósito de maximizar a **densidade de código**: em lugar das instruções de 32 bits do conjunto ARM tradicional (ARM32), as instruções Thumb possuem 16 bits, o que reduz o tamanho do programa em algo entre 30% e 65%. Programas menores implicam menor consumo de memória Flash — e, conseqüentemente, chips mais baratos —, menos falhas de *cache* e menor gasto de energia na busca de instruções. Em barramentos de 16 bits, esse ganho é ainda mais expressivo, uma vez que cada instrução pode ser buscada em um único ciclo.

Essa densidade, contudo, tinha um preço. No ARMv4T, o Thumb constituía um subconjunto do ARM32, sujeito a limitações importantes: acesso restrito aos registradores inferiores e ausência de execução condicional. Operações mais complexas exigiam um número maior de instruções, e determinadas tarefas — manipulação de coprocessadores e tratamento de exceções — só podiam ser realizadas no conjunto ARM32. Por essa razão, o processador precisava **alternar entre dois estados de execução**, ARM e Thumb, conforme o trecho de código em questão: Thumb nos pontos em que o tamanho era prioritário, ARM32 onde o desempenho se mostrava crítico.

Os processadores mais representativos dessa geração foram o **ARM7TDMI** e o **ARM9TDMI**, comercializados aos bilhões de unidades. O ARM7TDMI tornou-se onipresente em sistemas embarcados em

razão de sua baixa complexidade, eficiência energética e depuração integrada via JTAG; já o ARM9TDMI trouxe frequências mais elevadas e uma arquitetura Harvard modificada, atendendo a aplicações que demandavam maior capacidade de processamento. Diversos termos que ainda figuram na documentação do Cortex-M — como “*User state*” para o nível não-privilegiado — constituem herança direta desses processadores.

3.1.2 ARMv5 e ARMv6: interoperabilidade e Thumb-2

A **ARMv5** aprimorou a convivência entre os dois estados de execução. No ARMv4T, a única instrução capaz de alternar entre ARM e Thumb era a **BX** (*Branch and Exchange*); a ARMv5 acrescentou a **BLX**, que facilitava a chamada de funções escritas no outro conjunto de instruções, e passou a permitir que instruções de carga no contador de programa (**LDR**, **LDM**, **POP**) também promovessem essa troca de estado. O mecanismo então adotado é de especial relevância para nós, pois permanece em vigor até hoje: **o bit menos significativo do endereço carregado no PC indica o estado de destino** — 1 para Thumb, 0 para ARM. A ARMv5 introduziu, ainda, aritmética saturada e extensões para processamento digital de sinais (DSP), que vieram a acelerar aplicações de áudio e vídeo.

A **ARMv6**, lançada por volta do ano 2000, trouxe mudanças profundas na arquitetura de memória (acessos não alinhados, acessos exclusivos para operações atômicas, suporte a múltiplos núcleos), instruções **SIMD** (*Single Instruction, Multiple Data*) destinadas a multimídia, as extensões de segurança **TrustZone** e, principalmente, o **Thumb-2**. O Thumb-2 estende o Thumb original com instruções de 32 bits que podem ser **livremente combinadas** com as de 16 bits no mesmo programa, sem qualquer troca de estado. Preserva a maior parte da densidade de código do Thumb e se aproxima da funcionalidade completa do ARM32 — inclusive no que se refere à execução condicional, viabilizada pela instrução **IT** (*If-Then*), capaz de tornar condicionais até quatro instruções subsequentes. Operar em um único estado simplifica sobremaneira o desenvolvimento de software e elimina a sobrecarga inerente às trocas de estado.

Processadores ARMv6 equiparam os primeiros modelos do Raspberry Pi. A variante **ARMv6-M**, por sua vez, deu origem aos microcontroladores mais simples da família Cortex-M, a saber, o Cortex-M0 e o Cortex-M0+.

3.1.3 ARMv7 e os perfis A, R e M

Lançada em 2009, a **ARMv7** foi a arquitetura responsável por retirar a ARM do nicho de “baixo consumo” e conduzi-la ao domínio dos *smartphones*, dos *tablets* e de boa parte da eletrônica embarcada. Sua principal mudança conceitual consistiu em abandonar a ideia de uma arquitetura única e genérica, formalizando, em seu lugar, três **perfis**, cada um deles otimizado para um tipo específico de aplicação — e tornando o Thumb-2 obrigatório em todos eles.

O **perfil A** (*Application*) constitui o mais completo dos três: destinado a dispositivos que executam sistemas operacionais complexos, como Linux e Android, oferece múltiplos núcleos, unidade de gerenciamento de memória (MMU), virtualização e o conjunto de instruções vetoriais **NEON**, voltado a multimídia. A despeito do alto desempenho, incorpora também mecanismos de gerenciamento de energia — modos de economia e ajuste dinâmico de frequência.

O **perfil R** (*Real-time*) é concebido para aplicações nas quais previsibilidade e baixa latência revelam-se essenciais: controladores de discos rígidos, sistemas de freio, controle de movimento, automação industrial. Dispõe de recursos de hardware mais enxutos que os do perfil A, eliminando aquilo que não é estritamente necessário à resposta em tempo real, além de um tratamento de interrupções concebido para atender tarefas críticas de forma imediata.

O **perfil M** (*Microcontroller*), por sua vez, atende microcontroladores de baixo custo e baixo consumo: sistemas embarcados, IoT, dispositivos vestíveis, automação. Prescinde integralmente do conjunto de instruções ARM32 — existe apenas o Thumb-2 —, possui memória e recursos de hardware reduzidos, o que resulta em fabricação mais econômica, e mantém resposta rápida e determinística a eventos, característica fundamental em aplicações de controle. É precisamente este o perfil de nosso interesse: o **Cortex-M3** implementa a arquitetura **ARMv7-M**, ao passo que o **Cortex-M4** implementa a **ARMv7E-M**, uma extensão que incorpora instruções DSP, SIMD e, opcionalmente, unidade de ponto flutuante.

Perfil	Foco	Recursos característicos	Exemplos de uso
A	Alto desempenho, sistemas operacionais complexos	MMU, múltiplos núcleos, NEON, virtualização	Smartphones, tablets, Raspberry Pi

Perfil	Foco	Recursos característicos	Exemplos de uso
R	Tempo real com alto desempenho	Latência baixa e previsível, recursos enxutos	Controladores de disco, freios, controle de motores
M	Baixo custo, baixo consumo, determinismo	Apenas Thumb-2, sem MMU, controlador de interrupções integrado	Microcontroladores em geral — nosso foco

A história, entretanto, não se encerra aí: a **ARMv8** introduziu o modo de 64 bits (AArch64), preservando a compatibilidade com o modo de 32 bits (AArch32), e conduziu a arquitetura a servidores, computadores pessoais e supercomputadores. No que concerne aos microcontroladores, a ARMv8-M deu origem a núcleos como o Cortex-M33, dotados de TrustZone. Neste curso, contudo, permanecemos na ARMv7E-M do Cortex-M4.

Figura: linha do tempo da arquitetura ARM (ARMv4T → ARMv5 → ARMv6 → ARMv7-A/R/M → ARMv8), com os núcleos representativos de cada versão.

3.2 Processador, núcleo e microcontrolador

Antes de adentrarmos o modelo de programação, convém fixar uma distinção que costuma gerar confusão. O **Cortex-M4** constitui um **núcleo de processador** (*core*): a unidade responsável por buscar, decodificar e executar instruções, acompanhada de alguns componentes que a ARM fornece em conjunto — o controlador de interrupções (NVIC), o temporizador SysTick, a unidade de ponto flutuante e os blocos de depuração (FPB, DWT e ITM). É precisamente isso que a ARM projeta e licencia.

O **STM32F411**, por sua vez, é um **microcontrolador**: a ST parte do núcleo Cortex-M4 licenciado e constrói, em torno dele, a memória Flash, a SRAM, o sistema de *clock* e os periféricos (GPIO, *timers*, USART, SPI, I²C, ADC, USB). Por essa razão, a documentação apresenta-se organizada em camadas: tudo o que pertence ao núcleo encontra-se no manual de programação do Cortex-M4 (PM0214) — válido para **qualquer** microcontrolador Cortex-M4, independentemente do fabricante —, ao passo que a memória e os periféricos específicos do chip constam do manual de referência do STM32F411 (RM0383), a ser estudado no capítulo dedicado ao chip e a seus periféricos. Tudo o que será examinado a seguir pertence ao núcleo. Figura: diagrama de blocos em camadas: o núcleo Cortex-M4 (CPU, NVIC, SysTick, FPU, MPU, depuração), fornecido pela ARM, dentro do STM32F411 (Flash, SRAM, clock, periféricos), fornecido pela ST.

Em relação ao Cortex-M3, o Cortex-M4 acrescenta instruções DSP/SIMD e, na versão empregada no STM32F411, uma **unidade de ponto flutuante** (FPU) de precisão simples. Afora essas diferenças, os dois núcleos são praticamente equivalentes do ponto de vista de quem programa, de modo que tudo o que será apresentado a seguir vale para ambos.

3.3 Harvard ou Von Neumann?

No capítulo anterior, examinamos os dois modelos clássicos de organização: o de **Von Neumann**, com uma única memória e um único barramento para instruções e dados, e o **Harvard**, com barramentos separados, o que permite buscar a próxima instrução enquanto a atual acessa um dado. Constatamos, ainda, que a Harvard “pura” é rara, sendo mais comum encontrar uma **Harvard modificada**. O Cortex-M3/M4 constitui exatamente esse caso — e convém compreender em que sentido ele se aproxima de um modelo e em que sentido se aproxima do outro.

No que concerne ao hardware, trata-se de Harvard. O núcleo dispõe de três barramentos principais, dois dos quais atendem à mesma região de memória:

- o barramento **I-Code**, responsável pela busca de instruções na região de código (0x00000000–0x1FFFFFFF, onde se localiza a Flash);
- o barramento **D-Code**, responsável pelos acessos a **dados** nessa mesma região — por exemplo, a leitura de constantes armazenadas na Flash;
- o barramento de **Sistema** (*System bus*), que atende à SRAM, aos periféricos e ao restante do mapa de memória.

Como a busca de instruções e o acesso a dados utilizam barramentos distintos, ambos podem ocorrer **simultaneamente**. Enquanto uma instrução **LDR** efetua a leitura de uma variável na SRAM pelo barramento de Sistema, a instrução seguinte já está sendo buscada na Flash pelo I-Code. É esse mecanismo que mantém o *pipeline* de três estágios (busca, decodificação e execução) constantemente alimentado, sem o gargalo característico de Von Neumann.

Figura: os barramentos I-Code, D-Code e System ligando o núcleo à Flash, à SRAM e aos periféricos; destacar a busca de instrução e o acesso a dado acontecendo em paralelo.

Para quem programa, contudo, trata-se de Von Neumann. Existe um **único espaço de endereçamento** de 4 GB, no qual código, dados e periféricos coexistem, cada um em seu respectivo endereço. Um ponteiro em C pode apontar tanto para a Flash quanto para a SRAM, e as mesmas instruções **LDR/STR** acessam ambas indistintamente. O processador é até capaz de **executar código a partir da SRAM** — é exatamente o que faremos na atividade [Explorando o Cortex-M com o Depurador](#), ao escrever instruções manualmente no endereço `0x20000000`. Nessa situação, instrução e dado passam a disputar o mesmo barramento de Sistema, perdendo-se a vantagem proporcionada pela separação. Por essa razão, o código normalmente permanece na Flash, e os dados, na SRAM.

Essa combinação reúne o melhor dos dois modelos: o desempenho proporcionado pelos barramentos separados, aliado à simplicidade de programação característica de um espaço de endereços único. Nem todo núcleo Cortex-M realiza essa mesma escolha: o Cortex-M0 e o M0+, projetados visando ao menor custo possível, possuem um único barramento e configuram-se como Von Neumann de fato. No STM32F411, os barramentos do núcleo ainda passam por uma **matriz de barramentos** de responsabilidade da ST, que conecta cada um deles à Flash, à SRAM e aos periféricos, além de um acelerador de leitura da Flash. Esses detalhes serão examinados em conjunto com o mapa de memória do chip, mais adiante no livro.

3.4 RISC ou CISC?

O Cortex-M é, sem qualquer dúvida, um processador **RISC**. Segue a regra *Load-Store* — nenhuma instrução aritmética ou lógica opera diretamente sobre a memória —, dispõe de um banco de registradores de propósito geral relativamente extenso, e a maior parte de suas instruções é simples e é executada em um único ciclo. Esse padrão se manifesta em praticamente todo trecho de *assembly* gerado pelo compilador: carregar, operar entre registradores, armazenar.

Entretanto, a tabela de comparação entre CISC e RISC apresentada no capítulo anterior estabelecia que as instruções RISC possuem **tamanho fixo**, característica que não se verifica no Cortex-M, cujo conjunto mescla instruções de 16 e de 32 bits. Essa não é a única concessão realizada. Com o intuito de obter maior densidade de código e desempenho em tarefas típicas de microcontroladores, o conjunto de instruções do Cortex-M3/M4 incorpora diversas instruções que um RISC “de livro-texto” não possuiria:

- **transferências de múltiplos registradores** — **PUSH**, **POP**, **LDM** e **STM** salvam ou restauram vários registradores em uma única instrução, que leva vários ciclos para executar;
- **divisão em hardware** (**SDIV**, **UDIV**), com duração variável conforme os operandos, e **multiplicação-acumulação** (**MLA**);
- **manipulação de campos de bits** (**BFI**, **UBFX**), muito úteis para configurar registradores de periféricos;
- **execução condicional** via o bloco **IT** (*If-Then*);
- no Cortex-M4, instruções **DSP** e **SIMD**, que operam sobre vários dados de 8 ou 16 bits empacotados em um registrador.

A esses aspectos somam-se comportamentos que o próprio hardware realiza autonomamente e que, em processadores RISC mais antigos, dependiam de software — como o salvamento automático de registradores na pilha ao ingressar em uma interrupção, assunto que será examinado adiante. A classificação mais honesta, portanto, é a seguinte: **o Cortex-M constitui um RISC pragmático**. Preserva o núcleo da filosofia RISC — *Load-Store*, registradores, instruções simples e *pipeline* eficiente — e admite pequenas doses de complexidade nos pontos em que estas permitem economizar memória ou ciclos. Cabe observar, aliás, que a fronteira entre os dois modelos tornou-se difusa em praticamente todos os processadores atuais: mesmo os x86, CISC por excelência, traduzem internamente suas instruções complexas em micro-operações de natureza RISC.

3.5 Os conjuntos de instruções A32, T16 e T32

A seção anterior, sobre a evolução da ARM, evidenciou que a arquitetura acumulou, ao longo do tempo, mais de um conjunto de instruções. A nomenclatura atual da ARM designa o conjunto clássico de 32 bits como **A32** e o conjunto Thumb como **T32**. É igualmente comum empregar a denominação **T16** para o Thumb original, composto exclusivamente por instruções de 16 bits. Convém compará-los lado a lado, uma vez que as vantagens e desvantagens de cada um esclarecem a escolha adotada no Cortex-M.

	A32 (ARM)	T16 (Thumb original)	T32 (Thumb-2)
Surgiu em	ARMv1 (1985)	ARMv4T (anos 1990)	ARMv6T2 / ARMv7 (anos 2000)
Tamanho das instruções	32 bits, fixo	16 bits, fixo	16 e 32 bits, misturados
Registradores acessíveis	Todos, em quase todas as instruções	Na maioria das instruções, só R0–R7	Todos (as de 16 bits continuam restritas a R0–R7)
Execução condicional	Em quase toda instrução (campo de condição de 4 bits)	Só em desvios	Com o bloco IT (até 4 instruções)
Densidade de código	Baixa	Alta (programas cerca de 30% a 65% menores que em A32)	Próxima à do T16
Desempenho	Alto	Menor: mais instruções para o mesmo trabalho	Próximo ao do A32
Troca de estado	—	Necessária para usar recursos do A32	Desnecessária: um único estado

O **A32** oferece o máximo de flexibilidade — todos os registradores disponíveis em todas as instruções, execução condicional em praticamente tudo, constantes e deslocamentos embutidos na própria instrução —, mas cada instrução ocupa 4 bytes, inclusive as mais simples. Em um microcontrolador, no qual a Flash constitui o componente mais dispendioso do chip, essa característica pesa consideravelmente. O **T16** solucionou o problema do tamanho restringindo o que cabe em 16 bits: apenas 3 bits para indicar um registrador (daí o acesso restrito a R0–R7), constantes de pequena magnitude, e ausência de execução condicional. Os programas resultavam bem menores, porém mais lentos, e o processador precisava alternar para o estado ARM sempre que necessitasse de algum recurso que o Thumb não oferecia.

O **T32** elimina esse dilema: as operações mais comuns continuam empregando codificações de 16 bits, ao passo que aquelas que demandam mais recursos utilizam codificações de 32 bits, livremente combinadas entre si. Na prática a seguir, o estudante encontrará instruções de ambos os tamanhos combinadas. Por exemplo, `adds r3, #1` é codificada em 16 bits (0x3301), ao passo que `add.w r3, r3, #1000` demanda 32 bits (0xF503 0x737A), pois a constante 1000 não cabe nos 8 bits disponíveis na forma reduzida. A decisão sobre qual codificação empregar cabe ao *assembler*, e não ao programador. A execução condicional, que no A32 ocupava 4 bits de toda instrução, retorna sob a forma do bloco IT:

```

CMP    R0, #0
IT      EQ           @ a instrução seguinte só executa se Z = 1
ADDEQ  R1, R1, #1    @ R1 = R1 + 1, apenas se R0 era zero

```

Os processadores Cortex-M executam **exclusivamente T32**, sem dispor de estado ARM. O que varia de um núcleo para outro é a extensão do T32 implementada por cada um: o Cortex-M0 e o M0+ (ARMv6-M) implementam basicamente as instruções de 16 bits e apenas algumas de 32 bits; o Cortex-M3 (ARMv7-M) implementa o T32 em sua totalidade; e o Cortex-M4 (ARMv7E-M) acrescenta as instruções DSP, SIMD e, opcionalmente, as de ponto flutuante. Como os conjuntos são cumulativos, um programa compilado para o M0 é executado sem alterações no M4, mas o inverso não se verifica. É exatamente por essa razão que informamos o núcleo exato ao compilador, por meio da opção `-mcpu=cortex-m4`.

Figura: conjuntos de instruções cumulativos, em círculos concêntricos: Cortex-M0/M0+ (ARMv6-M) ⊂ Cortex-M3 (ARMv7-M) ⊂ Cortex-M4 (DSP/SIMD) ⊂ Cortex-M4F (ponto flutuante).

3.5.1 Modos de endereçamento concretos no ARMv7-M

No capítulo de fundamentos de arquitetura, verificamos que os processadores RISC seguem a regra *Load-Store*: as operações lógicas e aritméticas operam unicamente sobre registradores. O acesso à memória para leitura ou escrita é realizado exclusivamente por meio de instruções dedicadas (**LDR** e **STR**, além de suas variantes de byte **LDRB/STRB** e de meia-palavra **LDRH/STRH**).

No conjunto de instruções T32 (Thumb-2) do Cortex-M, essas instruções dispõem de cinco formas distintas de calcular o endereço. A compreensão de sua sintaxe é fundamental para a interpretação do código *assembly* gerado pelo compilador C:

1. **Endereçamento com deslocamento imediato (*Offset Addressing*):** O endereço efetivo corresponde à soma do registrador base (*Rn*) com uma constante imediata (*offset*), sem que o valor contido no registrador base seja modificado:

```
LDR R0, [R1, #4]      @ Endereço = R1 + 4; R1 permanece inalterado
STR R0, [R1, #12]     @ Grava R0 em (R1 + 12); R1 não muda
```

Trata-se do modo gerado pelo compilador quando se acessam membros de estruturas em C (por exemplo, `ptr->membro`) ou registradores de periféricos a partir de um endereço base (`STM32_GPIOC_BASE + OFFSET`).

2. **Endereçamento pré-indexado (*Pre-indexed Addressing*):** O endereço efetivo também corresponde à soma do registrador base com a constante, porém a presença do operador `!` (*writeback*) faz com que o registrador base seja **atualizado com o novo endereço** antes que a transferência se efetive:

```
LDR R0, [R1, #4]!     @ R1 = R1 + 4; em seguida lê R0 de [R1]
```

3. **Endereçamento pós-indexado (*Post-indexed Addressing*):** O dado é lido a partir do endereço atualmente indicado pelo registrador base (*Rn*); imediatamente **após** a leitura (ou a escrita), o registrador base é somado à constante de deslocamento:

```
LDR R0, [R1], #4      @ Lê R0 do endereço R1; em seguida R1 = R1 + 4
```

Este modo corresponde ao equivalente exato da expressão idiomática em linguagem C `dado = *ptr++`; amplamente empregada em laços de cópia e no processamento de fluxos de dados. Mediante uma única instrução, o Cortex-M carrega o dado e avança o ponteiro, sem que se torne necessária uma instrução `ADD` separada. (Cumpra observar que uma única instrução não equivale a um único ciclo: um `LDR` isolado consome tipicamente dois ciclos no Cortex-M4.)

4. **Endereçamento por registrador com deslocamento (*Register Offset e Scaled Register*):** O endereço é calculado somando-se o registrador base a outro registrador (*Rm*), o qual pode ser deslocado para a esquerda de 0 a 3 bits (`LSL #0` a `LSL #3`), isto é, multiplicado por 1, 2, 4 ou 8. Nas instruções de acesso à memória do Thumb-2, esse constitui o único deslocamento permitido; as operações `LSR`, `ASR` e `ROR` existem apenas nas instruções de processamento de dados:

```
LDR R0, [R1, R2]      @ Endereço = R1 + R2
LDR R0, [R1, R2, LSL #2] @ Endereço = R1 + (R2 << 2)
```

A segunda forma corresponde exatamente à maneira pela qual o GCC traduz a indexação de vetores em C (`array[i]`): para um vetor de inteiros de 32 bits (4 bytes), multiplicar o índice *i* contido em *R2* por 4 equivale a um deslocamento à esquerda de 2 bits (`LSL #2`). Esse deslocamento é realizado no interior da própria instrução de acesso, sem que seja necessária uma instrução adicional para o cálculo do endereço.

5. **Endereçamento relativo ao PC (*literal*):** Nesse modo, o registrador base é o próprio PC. Trata-se da forma empregada para carregar constantes de 32 bits que não cabem no campo imediato de uma instrução, inclusive **endereços**, como o de uma variável global. O compilador armazena essas constantes imediatamente após o código da função, em uma área denominada **literal pool**, lendo-as por meio de um `LDR` relativo ao PC:

```
LDR R3, [PC, #24]      @ Lê a constante guardada 24 bytes adiante (literal pool)
LDR R3, [R3, #0]      @ Usa-a como endereço: lê o valor da variável global
```

É exatamente esse par de instruções que figura no `objdump` da prática a seguir, ao ler `g_a` e `g_b`: o primeiro `LDR` traz o endereço da variável, e o segundo, o seu valor.

3.6 O modelo de programação

3.6.1 Modos de operação, estados de execução e níveis de acesso

Os processadores Cortex-M3 e Cortex-M4 possuem dois **modos de operação** — *Thread Mode* e *Handler Mode* — e dois **estados de execução** — *Thumb State* e *Debug State*. Possuem, ademais, dois **níveis de acesso**: privilegiado e não-privilegiado. No nível **privilegiado**, o software é capaz de acessar todos os recursos do processador; no nível **não-privilegiado**, determinadas regiões de memória tornam-se inacessíveis, e certas operações não podem ser executadas. Em alguns documentos, o nível não-privilegiado figura como “*User state*”, termo herdado do ARM7TDMI.

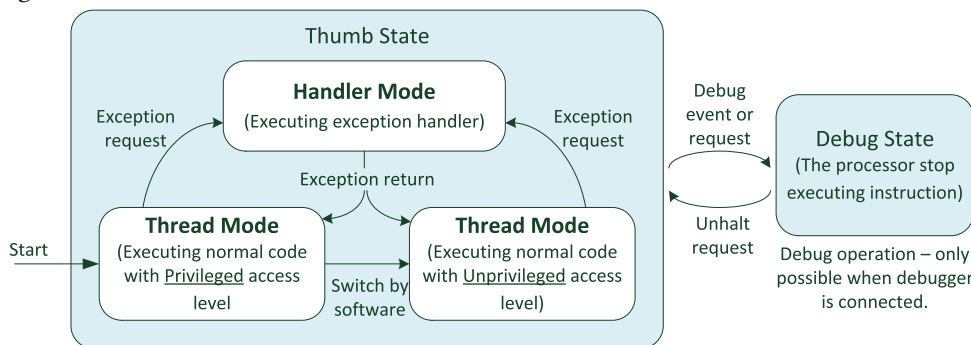


Figure 3.1: Modos de operação (Thread/Handler) e estados de execução (Thumb/Debug) do Cortex-M. O **Thread Mode** é o modo no qual é executado o “código normal” do programa, podendo operar tanto no nível privilegiado quanto no não-privilegiado. No nível privilegiado, o programa dispõe de acesso completo a todos os recursos, aos registradores de sistema e à **MPU** (*Memory Protection Unit*). No nível não-privilegiado, o programa não tem permissão para alterar registradores críticos do sistema — praticamente todos os registradores do controlador de interrupções, por exemplo, são de acesso exclusivamente privilegiado —, seu acesso à memória fica restringido pela MPU, e determinadas instruções especiais não podem ser utilizadas.

Essa separação entre os dois níveis possibilita a implementação de um modelo básico de segurança e robustez. Um sistema pode, por exemplo, manter o *kernel* de um sistema operacional de tempo real (RTOS) em execução no nível privilegiado, ao passo que as tarefas da aplicação permanecem no não-privilegiado; a MPU é então configurada de modo a impedir que uma tarefa venha a corromper a memória e os periféricos utilizados pelo *kernel* ou por outras tarefas. Caso uma tarefa venha a falhar, o restante do sistema continua em funcionamento normal.

O **Handler Mode**, por sua vez, é **exclusivo** do tratamento de exceções (incluindo interrupções) e opera **sempre** no nível privilegiado. Essa característica garante que a rotina de tratamento disponha de controle total sobre o processador e possa reagir ao evento de forma eficiente. O processador ingressa automaticamente em Handler Mode quando uma exceção é aceita, retornando ao Thread Mode uma vez concluído o tratamento.

O **estado de execução** define qual conjunto de instruções o processador se encontra executando. Diferentemente dos processadores ARMv7-A e ARMv7-R, que suportam o conjunto ARM32, os processadores ARMv7-M suportam **exclusivamente o Thumb-2** — possuindo, portanto, unicamente o **Thumb State**. Não existe “estado ARM” para o qual se possa trocar, o que simplifica o hardware, reduz o custo do chip e favorece a densidade de código. A arquitetura específica, ainda, o **Debug State**, empregado apenas durante a depuração: o processador nele ingressa quando é interrompido (*halted*) por uma requisição do depurador ou por um evento de depuração, como um *breakpoint*, deixando de executar instruções e permitindo que o depurador leia e modifique seus registradores. É esse o estado utilizado pelo ST-LINK quando o programa é pausado no GDB.

Por padrão, um processador Cortex-M emerge do reset no **Thumb State**, em **Thread Mode** e no nível **privilegiado**. Programas simples, que dispensam o isolamento entre partes do software, podem ser executados sempre nessa configuração — e é exatamente o que faremos em todos os programas *bare-metal* deste curso.

Quando necessário, o software pode transitar para o nível não-privilegiado mediante a alteração do bit **nPRIV** do registrador especial **CONTROL**. Ocorre que o próprio **CONTROL** só pode ser modificado no nível privilegiado. Isso implica que, uma vez estabelecido o nível não-privilegiado, um programa em Thread Mode perde o acesso ao **CONTROL** e **não consegue retornar por conta própria** ao nível privilegiado — comportamento esse intencional: uma aplicação não confiável não deve ser capaz de se

autopromover. Para efetuar o retorno, torna-se necessário recorrer ao mecanismo de exceções, tipicamente por meio da instrução **SVC** (*Supervisor Call*). O SVC provoca uma exceção e, com ela, o ingresso em Handler Mode; como esse modo é sempre privilegiado, a rotina de tratamento pode limpar o bit nPRIV, de modo que, ao retornar, o Thread Mode volte a ser privilegiado.

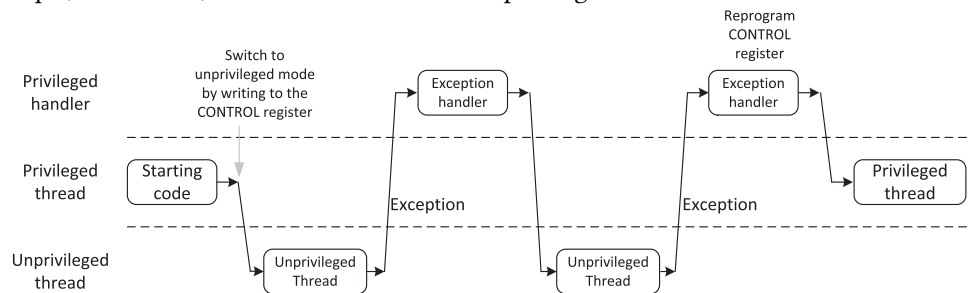


Figure 3.2: Troca para o nível não-privilegiado via CONTROL e retorno ao privilegiado via exceção (SVC).

3.6.2 O banco de registradores

No capítulo anterior, verificamos que, em uma arquitetura *Load-Store*, qualquer operação aritmética ou lógica só pode ser realizada com dados que já se encontrem nos registradores: caso um dado da memória necessite ser processado, ele é primeiramente carregado para um registrador, processado nesse local e, se necessário, escrito novamente na memória. Os registradores funcionam, portanto, como a área de trabalho da ULA. Dispor de registradores em número suficiente permite que o compilador mantenha diversas variáveis neles ao longo de um trecho de código, evitando o acesso à memória a cada uso — e é exatamente essa característica que confere eficiência ao código gerado pelo compilador C.

O banco de registradores do Cortex-M3/M4 é composto por **16 registradores de 32 bits**, de **R0 a R15**, dos quais treze são de propósito geral e três possuem uso específico.

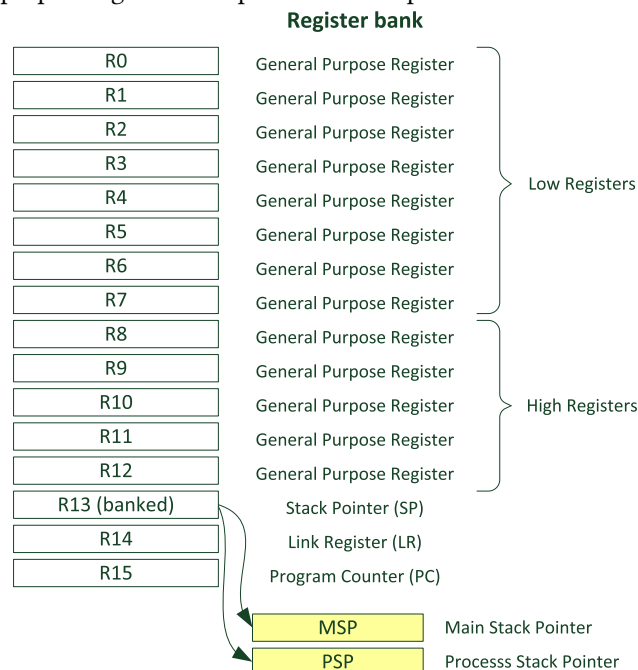


Figure 3.3: Banco de registradores do Cortex-M: R0–R7 (baixos), R8–R12 (altos) e os registradores de uso específico SP (MSP/PSP), LR e PC.

R0 a R12 — propósito geral. Os oito primeiros, **R0 a R7**, recebem a denominação de **registradores baixos** (*low registers*). Como uma instrução de 16 bits dispõe de pouco espaço para codificar o número do registrador (3 bits permitem alcançar, no máximo, 8 registradores), boa parte das instruções Thumb de 16 bits só é capaz de acessar esses oito. Os **registradores altos** (*high registers*), **R8 a R12**, são acessados principalmente pelas instruções de 32 bits do Thumb-2 e por um número reduzido de instruções de 16 bits, como **MOV**. O valor inicial de **R0–R12** após o reset é indefinido.

R13 — Stack Pointer (SP). Trata-se do ponteiro de pilha, utilizado pelas instruções `PUSH` e `POP`. Fisicamente, existem **dois** ponteiros de pilha, dos quais apenas um se encontra visível como **R13** a cada instante:

- o **MSP** (*Main Stack Pointer*), ponteiro padrão, selecionado logo após o reset e **sempre** utilizado em Handler Mode. Seu valor inicial é lido automaticamente a partir da primeira palavra da memória durante o reset (assunto que será examinado na seção sobre a sequência de reset);
- o **PSP** (*Process Stack Pointer*), que só pode ser empregado em Thread Mode e cujo valor inicial é indefinido — devendo ser inicializado por software antes de qualquer uso.

A escolha entre os dois é realizada por meio do registrador `CONTROL`. O **PSP** mostra-se pertinente quando há um sistema operacional embarcado, que separa a pilha do *kernel* (**MSP**) das pilhas de cada tarefa (**PSP**); programas simples, como os que desenvolveremos neste curso, utilizam apenas o **MSP**. Os dois bits menos significativos do **SP** permanecem sempre nulos: no Cortex-M, as instruções `PUSH` e `POP` transferem sempre palavras de 32 bits, alinhadas a endereços múltiplos de 4.

R14 — Link Register (LR). Armazena o **endereço de retorno** no momento em que uma função é chamada: a instrução de chamada (`BL`) salva automaticamente em **LR** o endereço da instrução seguinte, e a função retorna copiando o conteúdo de **LR** para o **PC** (`BX LR`). Caso uma função efetue a chamada de outra, torna-se necessário salvar previamente o seu **LR** na pilha — caso contrário, a nova chamada sobrescreveria o endereço de retorno original. Durante o tratamento de exceções, **LR** recebe um valor especial, o `EXC_RETURN`, cujo carregamento no **PC** dispara o retorno da exceção.

R15 — Program Counter (PC). Aponta para a instrução a ser executada. Uma leitura do **PC** retorna o endereço da instrução atual **somado a 4**, consequência do *pipeline* e da compatibilidade com o ARM7TDMI. A escrita no **PC** provoca um desvio. Como as instruções possuem 16 ou 32 bits e permanecem alinhadas a endereços pares, o bit 0 de um endereço de instrução seria sempre nulo — e a arquitetura aproveita esse bit, herdando a convenção estabelecida pela ARMv5: ao carregar um endereço de destino no **PC** por meio de um desvio (`BX`, `BLX`, `POP {PC}` ou a própria tabela de vetores), **o bit 0 deve valer 1**, indicando tratar-se de código Thumb. Como o Cortex-M não dispõe de estado ARM, um destino cujo bit 0 seja igual a zero constitui uma tentativa de trocar para um estado inexistente, o que gera uma exceção de falha. Em C, o compilador e o *linker* encarregam-se disso automaticamente; entretanto, ao montar a tabela de vetores ou ao depurar um endereço “estranho”, terminado em número ímpar, é conveniente conhecer o motivo.

3.6.3 Registradores especiais

Além do banco de registradores, o processador possui **registradores especiais**, responsáveis por armazenar o estado do processador, controlar o mascaramento de exceções e definir o modo de operação. Esses registradores **não se encontram mapeados em memória**: só podem ser acessados por meio das instruções dedicadas `MRS` (que lê um registrador especial e o transfere para um registrador comum) e `MSR` (que nele escreve):

```
MRS R0, CONTROL ; R0 = CONTROL
MSR CONTROL, R0 ; CONTROL = R0
```

Não se deve confundir esses registradores especiais com os *special function registers* (SFR) de outras arquiteturas de microcontroladores — denominação usual para os registradores de controle de periféricos, os quais, no STM32, encontram-se mapeados em memória e são acessados por meio de `LDR/STR` comuns. Em aplicações simples escritas em C, raramente há necessidade de manipular os registradores especiais; eles tornam-se, no entanto, indispensáveis no desenvolvimento de sistemas operacionais embarcados e no controle fino de interrupções.

3.6.3.1 Registradores de estado do programa (PSR)

O **PSR** (*Program Status Register*) é composto de três registradores que ocupam campos distintos de uma mesma palavra de 32 bits e que, por essa razão, podem ser lidos em conjunto — hipótese em que o registrador combinado recebe a denominação de **xPSR**:

- **APSR** (*Application PSR*) — armazena as *flags* de resultado das operações: **N** (negativo), **Z** (zero), **C** (*carry*), **V** (*overflow*), além do bit **Q** (saturação) e, no Cortex-M4, os bits **GE**, empregados pelas instruções `SIMD`. Corresponde ao “registrador de status” apresentado genericamente no capítulo anterior, agora com nome e posições de bits concretos.

- **IPSR** (*Interrupt PSR*) — contém o **número da exceção** em tratamento no momento corrente. Assume valor zero em Thread Mode. Trata-se de um registrador somente leitura.
- **EPSR** (*Execution PSR*) — contém o bit **T** (*Thumb*), que no Cortex-M vale sempre 1 — qualquer tentativa de zerá-lo provoca uma falha —, além dos bits de estado da instrução **IT** e de instruções de múltiplos registradores interrompidas. Não pode ser lido diretamente por software.

Bits	31	30	29	28	27	24	19:16	8:0
Campo	N	Z	C	V	Q	T	GE (M4)	Nº da exceção
Reg-istrador	APSR	APSR	APSR	APSR	APSR	EPSR	APSR	IPSR

Figura: a palavra de 32 bits do xPSR, com os campos do APSR, do IPSR e do EPSR sobrepostos e coloridos por registrador.

Quem possui familiaridade com processadores ARM clássicos notará que os bits de modo do antigo CPSR do ARM7TDMI desapareceram — o Cortex-M não dispõe daqueles modos —, e que os bits de habilitação de interrupção migraram do PSR para registradores próprios, os registradores de máscara.

As *flags* inteiras do APSR são atualizadas pelas instruções de processamento de dados e constituem a base dos desvios condicionais. A tabela a seguir apresenta alguns resultados obtidos em 32 bits:

Operação	Resultado	N	Z	C	V	Observação
<code>0x7FFFFFFF</code> <code>+ 0x00000001</code>		1	0	0	1	Dois positivos deram “negativo”: <i>overflow</i> com sinal
<code>0xFFFFFFFF</code> <code>+ 0x00000001</code>		0	1	1	0	Estouro sem sinal (<i>carry</i>); com sinal, $-1 + 1 = 0$ é correto
<code>5 - 5</code>	<code>0x00000000</code>	0	1	1	0	Iguais: $Z = 1$. Em subtrações, $C = 1$ significa “não houve empréstimo”
<code>3 - 5</code>	<code>0xFFFFFFFF</code>	1	0	0	0	Resultado

Oper- ação	Resul- tado	N	Z	C	V	Obser- vação
						-2: neg- ativo, com em- prés- timo (C = 0)

Observe que o processador não possui conhecimento sobre se os números envolvidos são ou não sinalizados: ele calcula as quatro *flags* invariavelmente, cabendo à instrução de desvio subsequente escolher quais delas interpretar. Por exemplo, **BHI** (*higher*) testa C e Z para comparações sem sinal, ao passo que **BGT** (*greater than*) testa N, Z e V para comparações com sinal. É exatamente por essa razão que declarar uma variável como `int` ou `unsigned` em C altera as instruções geradas pelo compilador. O bit **C** permite, ainda, estender somas e subtrações para além de 32 bits: em uma soma de 64 bits, o *carry* resultante da soma das palavras inferiores é incorporado à soma das palavras superiores.

3.6.3.2 Registradores de máscara: PRIMASK, FAULTMASK e BASEPRI

Toda exceção possui um **nível de prioridade**, e no Cortex-M um número **menor** corresponde a uma prioridade **maior**. Os registradores de máscara permitem bloquear temporariamente exceções com base nessa prioridade — recurso empregado para proteger um trecho crítico de código contra interrupções. Todos assumem valor zero após o reset (nenhum bloqueio ativo) e só podem ser modificados no nível privilegiado.

- **PRIMASK** — composto por um único bit. Quando ajustado em 1, bloqueia todas as exceções de prioridade configurável, permitindo a passagem apenas do NMI e do HardFault. Corresponde, na prática, a “desabilitar todas as interrupções”. Existem instruções dedicadas a essa finalidade: **CPSID i** (desabilita) e **CPSIE i** (habilita).
- **FAULTMASK** — semelhante ao anterior, mas bloqueia também o HardFault. É empregado por código de tratamento de falhas e é limpo automaticamente na saída da exceção.
- **BASEPRI** — define um limiar: bloqueia as exceções cuja prioridade seja igual ou inferior ao valor programado, permitindo a passagem apenas das mais prioritárias. Quando ajustado em zero, permanece desativado. Sua largura depende do número de níveis de prioridade implementados pelo fabricante — no STM32F411, são 16 níveis (4 bits).

3.6.3.3 O registrador CONTROL

O registrador **CONTROL** define a configuração de execução do Thread Mode. Pode ser lido em qualquer nível, mas só pode ser modificado no nível privilegiado, assumindo valor zero após o reset — o que corresponde a Thread Mode privilegiado, com utilização do MSP.

Bit	Nome	Função
0	nPRIV	Nível de acesso em Thread Mode: 0 = privilegiado (padrão), 1 = não-privilegiado. Em Handler Mode o processador é sempre privilegiado.
1	SPSEL	Ponteiro de pilha em Thread Mode: 0 = MSP (padrão), 1 = PSP. Em Handler Mode vale sempre 0.
2	FPCA	Só existe no Cortex-M4 com FPU. Indica que o

Bit	Nome	Função
		contexto atual usou instruções de ponto flutuante, e portanto os registradores da FPU precisam ser salvos se ocorrer uma exceção. É ligado automaticamente pelo hardware.

Os bits `nPRIV` e `SPSEL` são independentes entre si, e as combinações usualmente encontradas ilustram como o Cortex-M é empregado na prática: `nPRIV=0`, `SPSEL=0` corresponde à aplicação simples, integralmente privilegiada e com uma única pilha — precisamente o nosso caso; `SPSEL=1` surge em sistemas operacionais embarcados, nos quais cada tarefa possui sua própria pilha (PSP), enquanto o *kernel* e as interrupções utilizam o MSP; `nPRIV=1` acrescenta o isolamento, com tarefas executando sem privilégio.

3.6.4 Registradores da unidade de ponto flutuante

O Cortex-M4 do STM32F411 incorpora a FPU opcional, que acrescenta 32 registradores de 32 bits, `S0` a `S31`, destinados a valores de precisão simples (`float`), os quais também podem ser acessados em pares, como `D0` a `D15`, além de um registrador de estado e controle, o **FPSCR**, contendo as *flags* das comparações em ponto flutuante, o modo de arredondamento e os indicadores de exceções numéricas (divisão por zero, *overflow*, operação inválida, entre outras).

Um detalhe prático de especial relevância para quem programa em regime *bare-metal*: **a FPU emerge do reset desligada**, com o propósito de economizar energia. Antes de executar qualquer instrução de ponto flutuante, o software precisa habilitá-la mediante escrita no registrador **CPACR** (*Coprocessor Access Control Register*, mapeado no endereço `0xE000ED88`). Caso o programa utilize `float` sem essa inicialização prévia, a primeira instrução de ponto flutuante provocará uma falha. Em projetos gerados por IDEs, essa etapa permanece oculta no código de inicialização; em nosso `startup.c`, ela terá de ser realizada explicitamente quando precisarmos de ponto flutuante.

3.7 O sistema de memória

3.7.1 Um espaço de endereçamento de 4 GB

Com endereços de 32 bits, o Cortex-M enxerga um espaço linear de **4 GB** (2^{32} bytes). Nenhum microcontrolador dispõe de memória em tal quantidade — o STM32F411 possui 512 KB de Flash e 128 KB de SRAM —, mas esse espaço é utilizado para atribuir a cada elemento o seu próprio endereço: memórias, registradores de periféricos e registradores do próprio núcleo. Trata-se da ideia de **E/S mapeada em memória**: para o processador, configurar um periférico consiste, simplesmente, em escrever em um endereço, mediante as mesmas instruções `LDR/STR` empregadas para variáveis comuns.

A arquitetura divide esse espaço em regiões com uso predefinido, sendo essa divisão **idêntica em todos os microcontroladores Cortex-M**, independentemente do fabricante:

Faixa de endereços	Região	Uso típico
<code>0x00000000–0x1FFFFFFF</code>	Code (512 MB)	Memória de programa (Flash); contém a tabela de vetores
<code>0x20000000–0x3FFFFFFF</code>	SRAM (512 MB)	Memória de dados: variáveis, pilha
<code>0x40000000–0x5FFFFFFF</code>	Peripheral (512 MB)	Registradores dos periféricos do fabricante (GPIO, timers, USART...)
<code>0x60000000–0x9FFFFFFF</code>	External RAM (1 GB)	Memória externa, quando houver
<code>0xA0000000–0xDFFFFFFF</code>	External Device (1 GB)	Dispositivos externos

Faixa de endereços	Região	Uso típico
0xE0000000–0xE00FFFFF	Private Peripheral Bus	Periféricos internos do núcleo: NVIC, SysTick, SCB, depuração
0xE0100000–0xFFFFFFFF	Vendor-specific	Reservado ao fabricante

Figura: o mapa de memória de 4 GB desenhado como uma coluna de endereços (0x00000000 embaixo, 0xFFFFFFFF em cima), com as regiões Code, SRAM, Peripheral, External RAM, External Device, PPB e Vendor-specific; ao lado, onde o STM32F411 ocupa cada região (Flash em 0x08000000, SRAM em 0x20000000, periféricos em 0x40000000).

Cada fabricante ocupa apenas uma pequena parcela de cada região. No STM32F411, a Flash tem início em 0x08000000, a SRAM em 0x20000000, e os periféricos a partir de 0x40000000 — o mapa detalhado do chip será assunto do capítulo dedicado ao STM32F411. O que importa, neste momento, é que a região do núcleo (*Private Peripheral Bus*) permanece sempre no mesmo local: os registradores do NVIC e do SysTick ocupam os mesmos endereços em um STM32, em um nRF52 ou em um LPC. Essa uniformidade facilita a portabilidade de software entre fabricantes e permite que ferramentas de depuração tratem todos os núcleos Cortex-M de maneira uniforme.

O sistema de memória do Cortex-M3/M4 dispõe, ainda, de outros recursos que encontraremos ao longo do curso: suporte a acessos não alinhados, um *buffer* de escrita que permite ao processador prosseguir a execução enquanto uma escrita mais lenta se completa, a região opcional de **bit-band** (que possibilita acessar bits individuais da SRAM e dos periféricos de forma atômica) e a **MPU**.

3.7.2 Atributos arquiteturais de memória: Normal vs. Device Memory

Um aspecto de importância crucial no sistema de memória do Cortex-M — frequentemente ignorado por programadores provenientes de computadores comuns — reside no fato de que os endereços não se diferenciam apenas pelo que a eles se conecta, mas também pelos **atributos de acesso** que a arquitetura ARMv7-M associa a cada região. O mapa de memória padrão define, para cada região, um **tipo de memória** e estabelece se é ou não permitido executar código a partir dela (Yiu, Seções 6.8 e 6.9). São três os tipos existentes:

1. **Memória Normal (*Normal Memory*):** Corresponde ao tipo das regiões de **Code** (Flash), **SRAM** e RAM externa. A leitura de uma posição de memória comum não produz efeito colateral algum: ler duas vezes devolve o mesmo valor, e uma leitura adicional não compromete nada. Por essa razão, a arquitetura confere ao hardware liberdade para acelerar tais acessos: permite buscar instruções antecipadamente, inclusive no destino provável de um desvio, e, em processadores de maior porte, dotados de *cache*, agrupar e reordenar acessos. O Cortex-M3/M4, por ser mais simples, executa os acessos na ordem do programa, mas o código não deve depender desse comportamento em memória Normal.
2. **Memória de Dispositivo (*Device Memory*):** Corresponde ao tipo da região de **Periféricos** (0x40000000 a 0x5FFFFFFF) e das regiões de dispositivos externos (0xA0000000 a 0xDFFFFFFF). Em um registrador de periférico, **ler ou escrever não constitui uma operação neutra**: pode limpar uma *flag* de interrupção, transmitir um caractere pela UART ou acionar um pino de saída. Por essa razão, a arquitetura garante que:
 - o processador não realiza acessos especulativos nem repetidos: cada leitura e cada escrita do programa gera exatamente um acesso ao barramento;
 - os acessos à memória Device alcançam o barramento **na ordem em que aparecem no programa**, preservando-se o número e o tamanho dos acessos (duas escritas em um mesmo endereço não são fundidas em uma só);
 - a região é marcada como **Execute Never** (XN): o processador se recusa a buscar instruções nela, e qualquer tentativa nesse sentido gera uma falha.

Há um detalhe que costuma surpreender: a memória Device é **bufferable**. O Cortex-M3/M4 dispõe de um *buffer* de escrita de uma única posição, e uma escrita direcionada a um periférico pode nele permanecer enquanto o processador já executa a instrução seguinte. A ordem entre acessos à memória Device é preservada, mas o processador não aguarda que a escrita efetivamente alcance o periférico. Na maior parte do tempo, esse comportamento é imperceptível; o caso clássico em que se manifesta é o de uma rotina de interrupção que limpa a *flag* do periférico em sua última instrução e retorna antes que a escrita alcance o destino, fazendo com que a interrupção desapareça novamente. A solução consiste na

instrução de barreira **DSB** (*Data Synchronization Barrier*), que faz o processador aguardar a conclusão de todos os acessos pendentes. Retomaremos esse assunto quando estudarmos interrupções.

3. **Memória Fortemente Ordenada (*Strongly-ordered*)**: Corresponde ao tipo da região do **barramento privado do núcleo** (PPB, `0xE0000000` a `0xE00FFFFF`), onde se localizam o NVIC, o SysTick e o SCB. Possui as mesmas garantias da memória Device, acrescidas de mais uma: não passa pelo *buffer* de escrita. O Cortex-M3/M4 aguarda a conclusão de cada transferência antes de prosseguir para a operação seguinte, de modo que uma escrita no NVIC já produz efeito no momento em que a instrução subsequente é executada.

! Por que o hardware impõe isso e onde entra o **volatile**?

O hardware do Cortex-M garante que os acessos a periféricos alcancem o barramento na ordem e na quantidade correspondentes às instruções executadas. Entretanto, o processador só é capaz de executar as instruções que o **compilador C** efetivamente gerar! Caso o compilador julgue que duas escritas consecutivas no mesmo endereço são redundantes, eliminará a primeira delas durante a otimização (-O2), antes mesmo que o hardware tenha qualquer contato com o código. É exatamente nesse ponto que o qualificador **volatile** se revela indispensável em C: ele informa ao compilador que todo acesso àquele endereço constitui um efeito observável no hardware, que não pode ser descartado, reordenado ou mantido apenas em registrador.

3.7.3 A pilha

Conforme examinado no capítulo de fundamentos de arquitetura, a **pilha** (*stack*) é uma região de memória empregada como estrutura LIFO (*Last In, First Out*). O Cortex-M utiliza a própria SRAM do sistema para a pilha, manipulando-a por meio das instruções **PUSH** e **POP**, que ajustam automaticamente o ponteiro de pilha selecionado. Ela destina-se a armazenar temporariamente o conteúdo de registradores de que uma função necessita, a passar parâmetros que não cabem em registradores, a armazenar variáveis locais e, no caso de exceções, a salvar o estado do processador.

O modelo adotado é o **full-descending stack** (pilha cheia descendente): a pilha cresce em direção a endereços **menores**, e o SP aponta sempre para o **último dado empilhado**. Em um **PUSH**, o processador primeiro decrementa o SP em 4 e depois grava o dado no endereço apontado; em um **POP**, lê o dado apontado e depois incrementa o SP. Por isso, o valor inicial do SP deve ser o endereço **imediatamente acima** da região reservada à pilha. No STM32F411, cuja SRAM vai de `0x20000000` a `0x2001FFFF`, é comum colocar a pilha no topo da SRAM e iniciar o SP em `0x20020000`: o primeiro **PUSH** grava em `0x2001FFFC`. Figura: a pilha *full-descending* antes e depois de um **PUSH** e de um **POP**, com o SP apontando para o último dado empilhado e a pilha crescendo para endereços menores.

O uso mais frequente da pilha consiste em preservar registradores durante chamadas de função. Uma função que necessita utilizar **R4**, **R5** e **R6** — cujos valores o chamador espera encontrar intactos — e que também efetua chamadas a outras funções pode iniciar e concluir da seguinte forma:

```
my_function:
    PUSH  {R4-R6, LR}    ; salva R4-R6 e o endereço de retorno
    ...                  ; corpo da função: usa R4-R6, chama outras funções
    POP   {R4-R6, PC}    ; restaura R4-R6 e retorna (o LR salvo vai direto para o PC)
```

Cada **PUSH** requer um **POP** correspondente. Observe o artifício presente na última linha: o valor de LR empilhado na entrada é desempilhado diretamente no PC, o que restaura os registradores e efetua o retorno da função por meio de uma única instrução.

3.8 Exceções e interrupções

Uma **exceção** consiste em um evento que altera o fluxo normal do programa: o processador suspende a execução em curso, executa um trecho de código denominado **tratador de exceção** (*exception handler*) e, ao término, retoma o programa exatamente de onde havia parado. Na arquitetura ARM, as **interrupções** constituem um tipo de exceção, geradas por periféricos ou entradas externas; seus tratadores são as conhecidas **ISRs** (*Interrupt Service Routines*). O próprio processador também gera exceções — seja para sinalizar falhas, seja para oferecer suporte a sistemas operacionais.

Cada fonte de exceção possui um **número**. Os números de 1 a 15 correspondem às **exceções de sistema**, definidas pela arquitetura; a partir de 16, encontram-se as **interrupções** (IRQs), definidas pelo fabricante. O Cortex-M3/M4 suporta até 240 interrupções, embora os fabricantes costumem implementar um número bem inferior — o STM32F411 possui aproximadamente 86 posições de interrupção em sua tabela.

Nº	Exceção	Prioridade	Descrição
1	Reset	-3 (máxima)	Reinício do processador
2	NMI	-2	Interrupção não-mascarável (ex.: falha do sistema de <i>clock</i>)
3	HardFault	-1	Falha genérica, quando a falha específica não pode ser tratada
4	MemManage	configurável	Violação da MPU ou acesso a região não executável
5	BusFault	configurável	Erro retornado pelo barramento em um acesso
6	UsageFault	configurável	Instrução inválida, tentativa de trocar para o estado ARM, divisão por zero (se habilitada)
11	SVCall	configurável	Chamada de supervisor, via instrução <i>SVC</i>
12	Debug Monitor	configurável	Depuração por software
14	PendSV	configurável	Requisição pendente de serviço do sistema (troca de contexto em RTOS)
15	SysTick	configurável	Temporizador do sistema, interno ao núcleo
16+	IRQ 0, 1, 2...	configurável	Interrupções dos periféricos do fabricante

Figura: numeração das exceções: exceções de sistema 1–15 e interrupções a partir de 16, com a correspondência número da exceção = número da IRQ + 16 (IRQ 0 = exceção 16).

O **Reset** constitui um caso especial: embora se trate de uma exceção — a de maior prioridade —, seu tratador é executado em **Thread Mode**, e não em Handler Mode. Por padrão, as falhas específicas (MemManage, BusFault, UsageFault) permanecem desabilitadas, e qualquer falha é escalada para o **HardFault**, que se mantém sempre ativo. Na prática, “o programa caiu no HardFault” configura-se como um dos

sintomas mais comuns — e mais temidos — de erro em *firmware* Cortex-M; os registradores de estado de falha do processador auxiliam na identificação da causa, e aprenderemos a consultá-los por meio do depurador.

3.8.1 O NVIC

As exceções e interrupções são gerenciadas pelo **NVIC** (*Nested Vectored Interrupt Controller*), componente do próprio núcleo cujos registradores encontram-se na região privada do mapa de memória. O nome resume suas principais características:

- **Nested (aninhado)** — cada exceção possui uma prioridade própria. Caso sobrevenha uma exceção de prioridade mais elevada enquanto outra está sendo tratada, a atual é suspensa (*preempção*), a nova é atendida, e, ao final, o processador retoma a anterior — tudo isso realizado em hardware, sem qualquer código adicional.
- **Vectored (vetorado)** — o próprio processador localiza o endereço do tratador de cada exceção, consultando a tabela de vetores. Nos ARM clássicos, como o ARM7TDMI, essa etapa era realizada por software, o que aumentava a latência. No Cortex-M3/M4, a latência de entrada em uma interrupção é de apenas 12 ciclos de *clock*.
- **Controle flexível** — cada interrupção pode ser habilitada, desabilitada, marcada ou desmarcada como pendente por software, além de ter sua prioridade programada. O mascaramento global é realizado pelos registradores PRIMASK, FAULTMASK e BASEPRI, examinados anteriormente.

Ao aceitar uma exceção, o processador empilha automaticamente um conjunto de registradores (R0–R3, R12, LR, PC e xPSR) — operação denominada *stacking* —, ingressa em Handler Mode e salta para o tratador correspondente. No retorno, desempilha a totalidade desses registradores (*unstacking*), e o programa interrompido prossegue como se nenhuma interrupção houvesse ocorrido. Como esse salvamento segue a mesma convenção de registradores empregada pelo compilador C, **um tratador de interrupção no Cortex-M pode consistir em uma função C comum**, sem qualquer palavra-chave especial — o que não se verifica em diversas outras arquiteturas.

Figura: linha do tempo de uma interrupção aninhada: o programa em Thread Mode, a entrada de uma IRQ de baixa prioridade, a preempção por uma de alta prioridade e os retornos, com o empilhamento automático (R0–R3, R12, LR, PC, xPSR) mostrado no quadro de pilha.

Nas proximidades do NVIC encontra-se o **SCB** (*System Control Block*), conjunto de registradores mapeados em memória que controla configurações do processador (como os modos de baixo consumo), fornece as informações de estado de falha e permite a realocação da tabela de vetores.

3.8.2 A tabela de vetores

A **tabela de vetores** consiste em um vetor de palavras de 32 bits na memória, no qual cada posição contém o **endereço inicial do tratador** de uma exceção. A posição de cada vetor corresponde ao número da exceção multiplicado por 4: o vetor do Reset (exceção 1) situa-se em 0x00000004, o do NMI (exceção 2) em 0x00000008, o do SysTick (exceção 15) em 0x0000003C, e o da IRQ 0 (exceção 16) em 0x00000040. A primeira posição, 0x00000000, constitui um caso especial: não corresponde a um vetor, mas ao **valor inicial do MSP**.

Endereço	Conteúdo
0x00000000	Valor inicial do MSP
0x00000004	Endereço do tratador de Reset
0x00000008	Endereço do tratador de NMI
0x0000000C	Endereço do tratador de HardFault
...	... demais exceções de sistema ...
0x0000003C	Endereço do tratador de SysTick
0x00000040 em diante	Endereços dos tratadores de IRQ 0, IRQ 1, ...

Figura: a tabela de vetores na memória: MSP inicial em 0x00, vetores das exceções de sistema até 0x3C e vetores das IRQs a partir de 0x40, com setas apontando para os tratadores na Flash.

Como os tratadores consistem em código Thumb, o bit menos significativo de cada vetor deve valer 1 — a mesma convenção adotada para o PC. Após o reset, a tabela permanece no endereço `0x00000000`, podendo ser realocada por meio do registrador **VTOR** (*Vector Table Offset Register*), situado no SCB.

Em um programa *bare-metal*, **é o próprio programador quem escreve essa tabela**: trata-se de um vetor em C, posicionado pelo *linker* no início da memória Flash. É exatamente isso que faremos no `startup.c`, mais adiante no livro.

3.9 Depuração: JTAG e SWD

Não obstante sua compacidade, o Cortex-M3/M4 incorpora recursos de depuração completos: interromper e executar passo a passo, *breakpoints* de instrução, *watchpoints* de dados (interromper a execução quando uma variável é acessada), acesso a registradores e à memória com o programa em execução, *profiling* e *trace*.

O adaptador de depuração conecta-se ao chip por meio de uma de duas interfaces possíveis. O protocolo tradicional **JTAG** utiliza 4 ou 5 pinos; já o **SWD** (*Serial Wire Debug*), criado pela ARM, oferece os mesmos recursos com apenas **dois pinos** — um de *clock* (**SWCLK**) e um de dados bidirecional (**SWDIO**) — sem qualquer perda de desempenho. Ambos os protocolos compartilham pinos (o *clock* do SWD corresponde ao **TCK** do JTAG, e o dado, ao **TMS**), de modo que um mesmo conector é capaz de atender aos dois. É por meio do SWD que o ST-LINK comunica-se com a Blackpill, através dos pinos **PA13** (**SWDIO**) e **PA14** (**SWCLK**) — razão pela qual reconfigurar esses pinos como GPIO comum no programa pode “bloquear” o acesso da placa ao depurador.

Figura: a ligação SWD entre o ST-LINK e a Blackpill (SWDIO/PA13, SWCLK/PA14, GND, 3V3), com o OpenOCD/GDB do lado do computador.

Para a coleta de informações durante a execução, sem que seja necessário interromper o processador, existe ainda a interface de **trace**, seja por meio de um único pino (**SWO**, *Serial Wire Output*), seja por uma porta paralela de múltiplos pinos, capaz de registrar a execução completa do programa em tempo real.

3.10 A sequência de reset

Um microcontrolador Cortex-M pode ser submetido a três tipos de reset: o **power-on reset**, que reinicia a totalidade do sistema — processador, componentes de depuração e periféricos; o **reset de sistema**, que reinicia processador e periféricos, mas preserva os componentes de depuração (de modo que o depurador não perca a conexão); e o **reset de processador**, que reinicia unicamente o núcleo.

Concluído o reset, e antes de executar qualquer instrução, o processador realiza exatamente duas leituras da memória:

1. lê a palavra no endereço `0x00000000` e a coloca no **MSP**;
2. lê a palavra no endereço `0x00000004` — o vetor de Reset — e a coloca no **PC**.

A partir desse instante, inicia-se a execução do tratador de Reset, em Thread Mode, no nível privilegiado, no estado Thumb. O MSP é configurado antes de qualquer instrução justamente porque uma exceção como o NMI ou o HardFault pode ocorrer imediatamente após o reset, e o processador necessita de uma pilha válida para nela empilhar seu estado.

Figura: a sequência de reset: o processador lê a palavra em `0x00000000` para o MSP e a palavra em `0x00000004` para o PC, e começa a executar o tratador de Reset.

Esse comportamento diverge substancialmente daquele apresentado pelos ARM clássicos, como o ARM7TDMI, os quais iniciavam a execução de instruções a partir do endereço zero — a tabela de vetores continha instruções, e não endereços — e exigiam que o próprio software inicializasse os ponteiros de pilha. No Cortex-M, a consequência prática é significativa: **as duas primeiras palavras da Flash definem como o programa se inicia**, e, dado que o vetor de Reset corresponde ao endereço de uma função comum, o código de inicialização pode ser escrito integralmente em C.

No STM32F411, a Flash situa-se em `0x08000000`, mas, na configuração usual de *boot*, o chip espelha essa região no endereço `0x00000000`. Dessa forma, se o programa gravar no início da Flash o valor `0x20020000` (topo da SRAM), seguido do endereço da função `reset_handler` com o bit 0 ajustado em 1, o processador emergirá do reset com a pilha posicionada no topo da SRAM, executando, a seguir, `reset_handler`. Esse é, literalmente, o primeiro código que escreveremos no capítulo de inicialização do sistema.

3.11 Questionário de revisão

1. A versão da arquitetura ARM que introduziu o conjunto de instruções Thumb, de 16 bits, foi a .

2. O Thumb-2, ao contrário do Thumb original, permite misturar instruções de 16 e 32 bits dentro de um único estado, sem trocar de modo apenas alternando entre os estados ARM e Thumb.
3. O perfil da arquitetura ARMv7 voltado a microcontroladores é o perfil .
4. O Cortex-M4 implementa a arquitetura ARMv6-MARMv7-MARMv7E-MARMv8-A.
5. Código executando em Handler Mode é sempre privilegiado não-privilegiado privilegiado ou não, conforme o CONTROL.
6. Um programa que passou para o nível não-privilegiado só consegue voltar ao privilegiado escrevendo de novo no registrador CONTROL por meio de uma exceção (ex.: SVC).
7. Muitas instruções Thumb de 16 bits só acessam os registradores R0 a R7 R8 a R12 R13 a R15.
8. O ponteiro de pilha usado após o reset e sempre em Handler Mode é o MSP PSP.
9. O registrador que guarda o endereço de retorno de uma função é o .
10. Um endereço de desvio para código Thumb deve ter o bit 0 igual a .
11. O registrador especial que contém o número da exceção em tratamento é o APSR IPSR CONTROL.
12. Após a operação `0xFFFFFFFF + 1` em 32 bits, as *flags* Z e C valem, respectivamente, 0 e 00 e 11 e 11 e 0.
13. Para desabilitar todas as interrupções de prioridade configurável, usa-se o registrador .
14. No Cortex-M, os registradores dos periféricos do fabricante ficam na região de memória que começa em `0x00000000` `0x20000000` `0x40000000` `0xE0000000`.
15. Na pilha *full-descending* do Cortex-M, um `PUSH` decrementa o SP e depois grava o dado grava o dado e depois incrementa o SP grava o dado e depois decrementa o SP.
16. As exceções de número 1 a 15 são exceções de sistema; as interrupções dos periféricos começam no número .
17. Logo após o reset, o processador lê o endereço `0x00000000` para inicializar o e o endereço `0x00000004` para inicializar o .
18. Antes de usar variáveis `float` no STM32F411, é preciso nada — a FPU já vem ligada habilitar a FPU no registrador CPACR trocar para o estado ARM.
19. O protocolo de depuração de dois pinos usado pelo ST-LINK é o .
20. O Cortex-M3/M4 é classificado como uma arquitetura Von Neumann pura Harvard pura Harvard modificada: tem barramentos separados para instruções e dados, mas um único espaço de endereçamento.
21. No Cortex-M3/M4, a busca de instruções na região de código é feita pelo barramento I-Code D-Code de Sistema.
22. Quando o Cortex-M executa código a partir da SRAM, o desempenho aumenta, porque a SRAM é mais rápida instruções e dados disputam o mesmo barramento de Sistema ocorre uma falha, pois a SRAM não é executável.
23. Qual destas características do Cortex-M foge da definição clássica de RISC? Arquitetura Load-Store-Instruções de tamanho variável (16 e 32 bits) Banco com 16 registradores
24. O conjunto de instruções executado pelos processadores Cortex-M é o A32 A32 e T32, alternando entre estados T32 (Thumb-2).
25. No T32, a execução condicional de até quatro instruções é feita com a instrução .

3.12 Referências

- [1] Yiu, J. *The Definitive Guide to ARM Cortex-M3 and Cortex-M4 Processors*, 3ª edição. Newnes, 2013. Capítulo 4 (*Architecture*), pp. 75–116 — referência estrutural para o modelo de programação, o sistema de memória, as exceções e a sequência de reset.
- [2] Almeida, M. B. de. *Apostila de Sistemas Embarcados I*. Universidade Federal de Uberlândia (material didático interno, inédito) — base da seção sobre a evolução da arquitetura ARM e dos diagramas de modos, estados e registradores, adaptados aqui.
- [3] [PM0214 — STM32 Cortex-M4 MCUs and MPUs Programming Manual](#) — STMicroelectronics.

- [4] [RM0383 – STM32F411xC/E Reference Manual](#) – STMicroelectronics.
- [5] [ARMv7-M Architecture Reference Manual](#) – Arm Ltd.

4. Prática: Do C ao Binário

Note

Pré-requisito: o gcc nativo (pacote `build-essential`) e o `arm-none-eabi-gcc` instalados, conforme o roteiro de [Ambiente e Ferramentas](#). Esta prática não usa a placa.

Quando se aciona o comando “compilar” em uma IDE, uma sequência inteira de programas entra em ação, e o executável gerado depende de peças que o programador não escreveu: a biblioteca C, um código de inicialização que é executado antes do `main()` e o sistema operacional, responsável por atender aos pedidos do programa. No PC, todo esse processo permanece oculto. Em um microcontrolador, **cada uma dessas peças passa a ser responsabilidade nossa**. Por essa razão, antes de programar a placa, é necessário compreender como um programa C se transforma em um executável, desmontando esse processo no computador que já é familiar, onde cada etapa pode ser executada e observada individualmente. Em seguida, procederemos à comparação com o que ocorre quando o alvo passa a ser o Cortex-M.

Livro-Texto Oficial: Leitura de Aprofundamento no Joseph Yiu

Para acompanhar esta prática, consulte no livro-texto oficial as seções ligadas a cada parte dela:

- **Capítulo 2 (*Introduction to Embedded Software Development*):** Seções 2.3 (*Software development flow*) e 2.4 (*Compiling your applications*), pp. 30–36 — o fluxo de desenvolvimento e as etapas da compilação para um microcontrolador.
- **Capítulo 17 (*Getting Started with the GNU Compiler Collection (gcc)*):** Seções 17.1 a 17.4, pp. 561–567 — o `toolchain` GNU para ARM e as suas opções de linha de comando.
- **Capítulo 5 (*Instruction Set*):** Seções 5.3 a 5.5 (*Understanding the assembly language syntax, Use of a suffix in instructions* e *Unified assembly language*), pp. 123–131 — como ler a desmontagem do `objdump`, incluindo sufixos como o `.w` do `add.w`.
- **Capítulo 20 (*Assembly and Mixed Language Projects*):** Seção 20.2 (*Interaction between C and assembly*), pp. 648–650 — a convenção AAPCS, com a tabela de uso dos registradores nas chamadas de função (Tabela 20.2).

! Antes de começar: quanto vale `g_a + g_b`?

Observe o programa a seguir e, **antes de prosseguir com a leitura desta prática**, registre a sua resposta: o que ele imprime? (a) `g_a + g_b = 10`; (b) não compila, porque `g_b` não foi inicializada; (c) um valor aleatório, porque `g_b` contém “lixo” da memória. Justifique a escolha. A verificação ocorrerá na Parte 1.

```
#include <stdio.h>

#define INITIAL_VALUE 10

int g_a = INITIAL_VALUE;
int g_b;

int add(int x, int y)
{
    return x + y;
}

int main(void)
{
    printf("g_a + g_b = %d\n", add(g_a, g_b));
    return 0;
}
```

4.1 Do código-fonte ao executável

Quando um programa é compilado para o próprio computador, o compilador gera código para a **mesma arquitetura** da máquina em que está sendo executado: compila-se e executa-se no mesmo local. Trata-se do desenvolvimento **nativo**. Em sistemas embarcados, essa abordagem não é viável, uma vez que o microcontrolador não dispõe de memória, sistema de arquivos nem interface capazes de executar um compilador. Emprega-se, portanto, o desenvolvimento **cruzado** (*cross-development*): o código é escrito e compilado em um computador comum, o *host*, por meio de um **compilador cruzado** (*cross-compiler*), responsável por gerar código destinado a outra arquitetura, a do *target* — o microcontrolador, que recebe apenas o programa já pronto. No caso em questão, o *host* corresponde a um PC x86-64 com Linux, e o *target*, ao Cortex-M4 do STM32F411.

Seja no desenvolvimento nativo ou cruzado, “o compilador” não constitui um único programa, mas uma cadeia de ferramentas (*toolchain*) que o comando `gcc` executa em sequência, encaminhando a saída de uma etapa como entrada da seguinte. Essa cadeia compreende quatro etapas.

Figura: o fluxo de compilação — arquivos `.c` e `.S` (fonte) passam pelo `gcc` e viram objetos `.o`, que se juntam às bibliotecas `.a` (biblioteca C e do usuário) no linker, guiado pelo linker script (mapa de memória); a saída é o executável `.elf`, do qual o `objcopy` extrai a imagem `.bin` gravada na Flash.

A primeira etapa é o **pré-processamento**. O pré-processador processa as linhas que se iniciam com `#`, sem compreender absolutamente nada da linguagem C: trata-se, essencialmente, de uma ferramenta de manipulação de texto. Cada `#include` é substituído pelo conteúdo integral do arquivo incluído, cada macro definida com `#define` é substituída pelo seu respectivo valor, e os blocos delimitados por `#if` e `#endif` são mantidos ou descartados conforme a condição avaliada. O resultado permanece sendo código C, ainda que maior e desprovido de qualquer diretiva.

Antes de prosseguir, convém desfazer uma confusão bastante recorrente: `#include <stdio.h>` **não inclui a biblioteca C**. O arquivo `stdio.h` constitui um **arquivo de cabeçalho**, isto é, um texto composto por declarações: os protótipos de funções como o `printf` (o nome, os parâmetros e o tipo de retorno), macros e tipos. Ele é processado na etapa 1, sendo colado pelo pré-processador, e serve ao **compilador**, que necessita saber que o `printf` existe e de que forma deve ser chamado para que a chamada correspondente seja gerada. O **código** do `printf`, já compilado, encontra-se em outro local, no interior de uma **biblioteca**, e só é incorporado na etapa 4, por intermédio do *linker*. Caso nunca tenha sido necessário solicitar explicitamente essa biblioteca, isso se deve ao fato de o `gcc` acrescentá-la automaticamente à ligação. Cabeçalho e biblioteca constituem, portanto, elementos distintos, empregados em etapas distintas

por ferramentas distintas: um informa ao compilador **como invocar** a função, ao passo que a outra fornece ao *linker* **o que efetivamente executar**.

A segunda etapa corresponde à **compilação** propriamente dita, responsável por traduzir o código C para a linguagem de montagem (*assembly*) do processador de destino. É nessa fase que o código é analisado, os erros de sintaxe e de tipo são identificados, e as otimizações são aplicadas. É também nesse momento que o programa é dividido em **seções**, de acordo com a natureza de cada elemento: o código é destinado à seção `.text`, as variáveis globais inicializadas à `.data`, as não inicializadas à `.bss`, e as constantes à `.rodata`.

A terceira etapa é a **montagem**. O *assembler* traduz cada instrução de *assembly* para o correspondente código de máquina, gerando um **arquivo objeto relocável** (`.o`). Esse arquivo já contém código de máquina, mas não constitui, ainda, um programa completo: o *assembler* não possui conhecimento sobre em que endereço de memória cada função e cada variável será posicionada, nem sobre a localização das funções definidas em outros arquivos. Por essa razão, o objeto armazena também uma **tabela de símbolos** (contendo o que é definido e o que é utilizado sem definição própria) e uma lista de **relocações**, correspondentes aos pontos do código que necessitarão de correção uma vez que os endereços sejam conhecidos.

A quarta etapa é a **ligação** (*linking*). O *linker* reúne todos os objetos do projeto, bem como as **bibliotecas** por eles utilizadas, verifica que todo símbolo empregado foi devidamente definido em algum ponto, agrupa as seções de mesma natureza, atribui um endereço definitivo a cada uma delas e corrige as relocações pendentes. O resultado é um **executável**, que, tanto no Linux quanto em sistemas embarcados, adota o formato **ELF** (*Executable and Linkable Format*). Para determinar os endereços, o *linker* segue um roteiro, denominado **linker script**, responsável por descrever as memórias disponíveis e o posicionamento adequado de cada seção. No PC, emprega-se um roteiro padrão, o que dispensa qualquer preocupação adicional a esse respeito. Em um microcontrolador, cujas memórias Flash e SRAM ocupam endereços fixos, é o próprio programador quem elabora esse roteiro. Em sistemas embarcados, há ainda uma etapa final: o `objcopy` extrai do ELF apenas os bytes que devem ser gravados na Flash, gerando uma **imagem binária** (`.bin`), posteriormente transferida ao chip pelo gravador.

i Três ou quatro etapas?

Muitos livros e cursos descrevem a compilação em **três** etapas — pré-processamento, compilação e ligação —, tratando a montagem como parte integrante da compilação. Ambas as perspectivas estão corretas; a diferença reside exclusivamente no nível de detalhe adotado. Do ponto de vista de quem utiliza o compilador, o código C é fornecido como entrada, e o objeto é produzido como saída: a própria opção `-c` do `gcc`, responsável por interromper a cadeia no arquivo objeto, significa *compile*, e não *assemble*. Internamente, contudo, o GCC executa dois programas distintos — o compilador `cc1`, encarregado de gerar o *assembly*, e o *assembler* `as`, responsável por gerar o objeto (ambos serão observados na atividade para casa, por meio da opção `-save-temps`). Outros compiladores, como o Clang, incorporam o *assembler* internamente e geram o objeto diretamente, sem que seja produzido um arquivo de *assembly* intermediário. Neste curso, optamos por manter as duas etapas separadas justamente porque o *assembly* intermediário constitui o objeto de interesse: é precisamente nele que se torna visível o conjunto de instruções do processador.

Três peças que o programador não escreveu participam da etapa de ligação. A primeira é a **biblioteca C**, contendo o código já compilado do `printf` e das demais funções da biblioteca padrão. A ligação pode ser **estática**, quando o código da biblioteca é copiado para o interior do executável, ou **dinâmica**, quando o executável armazena apenas uma referência, e a biblioteca compartilhada é carregada a partir do disco no momento da execução. A segunda peça é o **C runtime** (CRT, ou `crt0`), o código de inicialização que o *linker* acrescenta silenciosamente a todo programa, sendo executado **antes** do `main()`. Em conjunto com o carregador do sistema operacional, é esse código que prepara o ambiente no qual o `main()` será executado: organiza a pilha, inicializa a biblioteca C e a entrada e saída padrão, monta `argc` e `argv` a partir da linha de comando, efetua a chamada ao `main()` e, quando este retorna, entrega o valor de retorno ao sistema operacional. Constatar-se-á que esse código, em conjunto com o carregador, realiza ainda outra tarefa, que responde à pergunta formulada no início desta página. A terceira peça é o próprio **sistema operacional**. Um programa comum não possui permissão para acessar diretamente o hardware: para escrever no terminal, abrir um arquivo ou solicitar memória, ele realiza uma **chamada de sistema** (*system call*), isto é, um pedido dirigido ao *kernel*. A biblioteca C atua como intermediária nesse processo: o `printf` formata o texto no interior do próprio programa e, ao final, invoca a `syscall write` para que o *kernel* o entregue ao terminal.

Em um microcontrolador *bare-metal*, nenhuma dessas três peças encontra-se disponível de forma pronta. O *toolchain* fornece uma biblioteca C enxuta, mas inexistente sistema operacional capaz de atender às chamadas de sistema, e o *crt0* de um PC não se aplica, uma vez que o Cortex-M inicia sua execução pela tabela de vetores, e não por uma função propriamente dita. É por essa razão que, ao longo dos próximos capítulos, o código de inicialização e o *linker script* serão elaborados por nós mesmos. Nesta aula, será examinado o papel dessas peças no contexto do PC, bem como o que ocorre quando o alvo passa a ser o Cortex-M4, cenário no qual elas se encontram ausentes. A atividade para casa aprofunda o lado correspondente ao PC, experimento por experimento.

4.1.1 As ferramentas do *toolchain*

“O compilador” é, na realidade, um conjunto de programas provenientes de projetos distintos. A tabela a seguir reúne aqueles que serão empregados nesta prática, na atividade para casa e nas práticas subsequentes:

Ferramenta	De onde vem	O que faz
<code>gcc</code>	GCC	O driver : não compila nada sozinho, e sim chama, na ordem certa, os programas abaixo
<code>cc1</code>	GCC	O compilador propriamente dito: pré-processa o C e o traduz para <i>assembly</i> (é chamado pelo <code>gcc</code> , nunca diretamente)
<code>as</code>	GNU Binutils	O <i>assembler</i> : traduz o <i>assembly</i> em código de máquina, gerando o objeto <code>.o</code>
<code>ld</code>	GNU Binutils	O <i>linker</i> : junta os objetos e as bibliotecas e atribui os endereços, seguindo o <i>linker script</i>
<code>objcopy</code>	GNU Binutils	Converte o executável ELF em outros formatos, como a imagem binária <code>.bin</code> gravada na Flash
<code>nm</code>	GNU Binutils	Lista a tabela de símbolos: nome, endereço e seção de cada função e variável
<code>size</code>	GNU Binutils	Mostra o tamanho das seções <code>text</code> , <code>data</code> e <code>bss</code>
<code>objdump</code>	GNU Binutils	Desmonta o código de máquina (<code>-d</code>), lista as seções (<code>-h</code>) e as relocações (<code>-r</code>)
<code>readelf</code>	GNU Binutils	Mostra a estrutura de um arquivo ELF: cabeçalho, ponto de entrada, seções
<code>gdb</code>	GDB	O depurador: executa o programa passo a passo e inspeciona registradores e memória
<code>file</code>	Utilitário do sistema	Identifica o tipo de um arquivo (objeto relocável, executável, arquitetura)
<code>ldd</code>	glibc	Lista as bibliotecas dinâmicas de que um executável depende
<code>strace</code>	Utilitário do sistema	Mostra as chamadas de sistema que um programa faz

O GCC e o GNU Binutils constituem projetos independentes, distribuídos conjuntamente no *toolchain*. As versões destinadas ao Cortex-M recebem o prefixo `arm-none-eabi-` (`arm-none-eabi-gcc`, `arm-none-eabi-objdump`, `arm-none-eabi-gdb` etc.) e são responsáveis por gerar ou examinar código ARM; as versões sem prefixo correspondem ao PC. As ferramentas `file`, `ldd` e `strace` existem exclusivamente para o PC, uma vez que as duas últimas dependem da presença de um sistema operacional. A gravação do chip não integra o *toolchain*, sendo realizada pelo OpenOCD e pelo `st-flash`.

4.2 Parte 1 — Quanto vale `g_a` + `g_b`?

Os arquivos desta prática encontram-se no `lab-02` do repositório `ufu-semb1-src`. Atualize o repositório e copie a pasta correspondente para o seu espaço de trabalho:

```
foo@bar$ cd ~/seml-workspace/ufu-sembl-src
foo@bar$ git pull
foo@bar$ cp -r lab-02 ~/seml-workspace/lab-02
foo@bar$ cd ~/seml-workspace/lab-02
```

O arquivo `program.c` contém o programa apresentado no início desta página, acrescido de comentários explicativos. Compile e execute-o de forma nativa:

```
foo@bar$ gcc program.c -o program
foo@bar$ ./program
```

A resposta correta é `g_a + g_b = 10`. O programa compila sem qualquer aviso, e `g_b` assume o valor zero, e não um valor aleatório. A linguagem C garante que toda variável **global** desprovida de inicializador começa com valor zero (o “lixo” de memória ocorre exclusivamente com variáveis **locais**, criadas na pilha). Surge, então, a pergunta de maior interesse: se a primeira função do programa a ser executada é o `main()`, e este não atribui valor algum a `g_a` nem a `g_b`, **quem foi responsável por realizar `g_a = 10` e `g_b = 0`?**

O programa foi compilado e ligado em uma única etapa aparente, mas o `gcc`, internamente, percorreu todas as fases da cadeia. Interrompa essa cadeia antes da ligação, empregando a opção `-c`, e examine o objeto relocável resultante:

```
foo@bar$ gcc -c program.c -o program.o
foo@bar$ nm program.o
```

O `nm` mostra a tabela de símbolos, com uma letra que indica a seção de cada um:

Letra	Significado
T	Código, na seção <code>.text</code>
D	Dado inicializado, na seção <code>.data</code>
B	Dado não inicializado, na seção <code>.bss</code>
R	Dado somente-leitura, na seção <code>.rodata</code>
U	Indefinido: usado aqui, mas definido em outro lugar

O símbolo `g_a` aparece identificado pela letra `D`, `g_b` pela letra `B`, `add` e `main` pela letra `T`, e o `printf` pela letra `U`, uma vez que somente o *linker* será capaz de localizá-lo, no interior da biblioteca C. A resposta à pergunta formulada começa, assim, a se delinear: o valor `10` de `g_a` encontra-se gravado **dentro do próprio arquivo**, na seção `.data`; já `g_b`, situada na `.bss`, apenas ocupa espaço, sem que nenhum valor esteja de fato armazenado.

Quem se encarrega de transportar esses valores para a memória? Certamente não é o `main()`. Ao executar `./program`, o **carregador** do sistema operacional lê o executável, copia o conteúdo da `.data` para a memória — já contendo o `10` — e reserva a `.bss` previamente zerada. Em seguida, o *C runtime* (o `crt0`), acrescentado pelo *linker* sem qualquer solicitação explícita, prepara a pilha e a biblioteca C, chamando o `main()` somente então. Nenhuma linha de código escrita pelo programador foi responsável por essa preparação. A [atividade para casa](#) investiga cada uma dessas afirmações diretamente no executável: qual é o ponto de entrada, o que ocorre na ausência do *runtime*, de onde provém o `printf` e de que forma o texto chega ao terminal. Nesta aula, será examinado o que se altera quando o alvo passa a ser o Cortex-M4.

4.3 Parte 2 — E no Cortex-M4?

Comece por questionar a cada compilador para qual alvo ele gera código:

```
foo@bar$ gcc -dumpmachine
x86_64-linux-gnu
foo@bar$ arm-none-eabi-gcc -dumpmachine
arm-none-eabi
```

O nome do alvo é composto por três partes: **arquitetura**, **sistema operacional** e **convenção binária** (ABI). O `gcc` nativo gera código x86-64 destinado ao Linux; já o `arm-none-eabi-gcc` gera código ARM **sem sistema operacional** (*none*), seguindo a *Embedded Application Binary Interface* (*eabi*). Todas as

ferramentas do pacote compartilham esse mesmo prefixo, como observado na tabela do *toolchain*: *arm-none-eabi-as*, *arm-none-eabi-ld*, *arm-none-eabi-objdump*, *arm-none-eabi-nm*, entre outras.

Compile o mesmo `program.c` destinando-o ao Cortex-M4:

```
foo@bar$ arm-none-eabi-gcc -mcpu=cortex-m4 -mthumb program.c -o program.elf
```

As opções `-mcpu=cortex-m4 -mthumb` informam ao compilador cruzado qual é o núcleo de destino e que ele deve gerar instruções Thumb, as únicas executadas pelo Cortex-M. A compilação é concluída com sucesso, mas a ligação falha, apresentando uma lista de referências indefinidas: `_write`, `_read`, `_sbrk`, `_exit`, `_close`, `_fstat`, `_isatty`, `_lseek`, `_kill` e `_getpid`. Trata-se das **chamadas de sistema**, os pedidos dirigidos ao sistema operacional já mencionados entre as três peças anteriores: escrever (`_write`), ler (`_read`), solicitar memória (`_sbrk`), encerrar o programa (`_exit`). O *arm-none-eabi-gcc* também disponibiliza uma biblioteca C, denominada **newlib**, uma implementação enxuta, concebida especificamente para sistemas embarcados. Ela implementa o `printf` em sua totalidade, mas, na última etapa, necessita chamar `_write` para entregar os bytes a algum destinatário, e no alvo *none* não existe sistema operacional disponível para atender a essa chamada. A *newlib* deixa, portanto, essas funções a cargo do programador.

Isso conduz à pergunta que resume a diferença fundamental entre os dois universos: **em um sistema embarcado, para onde é direcionado o texto do `printf`**? A placa não dispõe de terminal. Seria possível escrever uma função `_write` que enviasse os bytes pela porta serial ao computador (o que será feito ao estudarmos a USART), mas, por ora, a resposta honesta é: para lugar nenhum. Torna-se necessária, então, uma versão do programa desprovida de `printf`. Essa versão já se encontra disponível no *lab-02*: trata-se do *embedded.c*, que armazena o resultado em uma terceira variável global e, em vez de retornar, permanece em um laço infinito (afinal, para onde retornaria, na ausência de um sistema operacional?):

```
#define INITIAL_VALUE 10

int g_a = INITIAL_VALUE;
int g_b;
int g_c;

int add(int x, int y)
{
    return x + y;
}

int main(void)
{
    g_c = add(g_a, g_b);

    while (1)
    {
    }

    return 0;
}
```

Uma vez que ainda não se dispõe de meios para informar ao *linker* a localização de cada elemento na memória do microcontrolador, interrompa a cadeia no objeto, empregando `-c`, e examine a tabela de símbolos:

```
foo@bar$ arm-none-eabi-gcc -c -g -mcpu=cortex-m4 -mthumb embedded.c -o embedded.o
foo@bar$ arm-none-eabi-nm embedded.o
00000000 T add
00000000 D g_a
00000000 B g_b
00000004 B g_c
0000001a T main
```

O *nm* revela exatamente o mesmo padrão observado no PC: `g_a` na `.data`, `g_b` e `g_c` na `.bss`, `add` e `main` na `.text`. Os endereços permanecem **relativos ao início de cada seção**: `add` e `g_a` situam-se, ambos, em 0, uma vez que cada um corresponde ao primeiro elemento de sua respectiva seção. Os endereços definitivos somente passam a existir após a etapa de ligação.

4.3.1 Lendo a desmontagem

Proceda, agora, à desmontagem do objeto. A saída a seguir exhibe apenas a função `main`, a que reveste maior interesse (a função `add` a precede, a partir do endereço 0):

```
foo@bar$ arm-none-eabi-objdump -d embedded.o
...
0000001a <main>:
 1a: b580      push    {r7, lr}
 1c: af00      add r7, sp, #0
 1e: 4b06      ldr r3, [pc, #24] @ (38 <main+0x1e>)
 20: 681b      ldr r3, [r3, #0]
 22: 4a06      ldr r2, [pc, #24] @ (3c <main+0x22>)
 24: 6812      ldr r2, [r2, #0]
 26: 4611      mov r1, r2
 28: 4618      mov r0, r3
 2a: f7ff fffe bl 0 <add>
 2e: 4603      mov r3, r0
 30: 4a03      ldr r2, [pc, #12] @ (40 <main+0x26>)
 32: 6013      str r3, [r2, #0]
 34: e7fe      b.n 34 <main+0x1a>
 36: bf00      nop
...
```

i Como ler uma linha do objdump

Cada linha tem até cinco colunas. Tome como exemplo a linha 1e: 4b06 ldr r3, [pc, #24] @ (38 <main+0x1e>):

Coluna	No exemplo	Significado
Endereço	1e	Posição da instrução, em hexadecimal. No objeto, é contada a partir do início da seção <code>.text</code> ; no executável, é o endereço definitivo.
Codificação	4b06	A instrução em código de máquina, em hexadecimal. Quatro dígitos são 16 bits ; oito dígitos, como <code>f7ff fffe</code> , são 32 bits , mostrados como duas metades de 16.
Mnemônico	ldr	O nome da instrução. Sufixos acrescentam informação: <code>s</code> indica que ela atualiza as <i>flags</i> (<i>adds</i>), <code>.n</code> (<i>narrow</i>) marca a codificação de 16 bits e <code>.w</code> (<i>wide</i>), a de 32.
Operandos	r3, [pc, #24]	O destino vem primeiro. Colchetes indicam um acesso à memória, no endereço calculado dentro deles; <code>#</code> marca uma constante.
Comentário	@ (38 <main+0x1e>)	Calculado pelo desmontador para ajudar a leitura: aqui, o endereço efetivo (0x38) e a sua posição em relação ao símbolo mais próximo.

Além disso, `<main>`: marca o início de uma função, e `...` indica um trecho de bytes zerados que o `objdump` omitiu.

Munido dessa chave de leitura, torna-se possível examinar o `main` linha a linha, relacionando cada instrução ao código C correspondente:

Código C	Endereço	Instrução	O que faz
<code>int main(void)</code>	1a–1c	<code>push {r7, lr} add r7, sp, #0</code>	Prólogo: guarda na pilha o endereço de retorno (LR) e o <code>r7</code> , que passa a apontar para o quadro da função.

Código C	Endereço	Instrução	O que faz
<code>g_a</code>	1e	<code>ldr r3, [pc, #24]</code>	Lê, 24 bytes adiante do PC, o endereço de <code>g_a</code> , guardado no fim da função (em <code>0x38</code>).
	20	<code>ldr r3, [r3, #0]</code>	Lê o valor de <code>g_a</code> , nesse endereço.
<code>g_b</code>	22–24	<code>ldr r2, [pc, #24] ldr r2, [r2, #0]</code>	O mesmo para <code>g_b</code> (endereço guardado em <code>0x3c</code>).
<code>add(g_a, g_b)</code>	26–28	<code>mov r1, r2 mov r0, r3</code>	Põe os argumentos em R0 e R1, como manda a convenção de chamada.
	2a	<code>bl 0 <add></code>	Chama <code>add</code> , guardando o endereço de retorno em LR.
<code>g_c = ...</code>	2e	<code>mov r3, r0</code>	O resultado volta em R0.
	30–32	<code>ldr r2, [pc, #12] str r3, [r2, #0]</code>	Lê o endereço de <code>g_c</code> (em <code>0x40</code>) e grava nele o resultado.
<code>while { }</code>	(1) 34	<code>b.n 34</code>	Um desvio para si mesma: o laço infinito.
	36	<code>nop</code>	Preenchimento, para que os dados seguintes fiquem alinhados em 4 bytes.
	38–40	<code>...</code>	O literal pool : três palavras com os endereços de <code>g_a</code> , <code>g_b</code> e <code>g_c</code> , ainda zeradas.

Dois aspectos dessa tabela merecem atenção especial.

O *literal pool*. Uma instrução de 16 bits não dispõe de espaço suficiente para acomodar um endereço de 32 bits. Por essa razão, o compilador armazena os endereços das variáveis globais imediatamente após o código da função, procedendo à sua leitura por meio de um `ldr` relativo ao PC. Trata-se do endereçamento relativo ao PC, estudado no capítulo referente ao Cortex-M.

As relocações. No objeto, as palavras correspondentes ao *literal pool* encontram-se zeradas, e o `bl` apresenta a codificação `f7ff fffe`, que o faz desviar para si mesmo: nenhum dos endereços é, ainda, conhecido. Trata-se de valores provisórios, que serão devidamente corrigidos pelo *linker*. A opção `-r` exhibe, abaixo de cada ponto a ser corrigido, o valor que o *linker* deverá inserir ali, enquanto `-z` exhibe os bytes zerados em vez de ...:

```
foo@bar$ arm-none-eabi-objdump -dr -z embedded.o
...
2a: f7ff fffe bl 0 <add>
      2a: R_ARM_THM_CALL add
...
38: 00000000 .word 0x00000000
      38: R_ARM_ABS32 g_a
3c: 00000000 .word 0x00000000
      3c: R_ARM_ABS32 g_b
40: 00000000 .word 0x00000000
      40: R_ARM_ABS32 g_c
```

A relocação `R_ARM_THM_CALL add` solicita ao *linker* que ajuste o `bl` de modo a alcançar `add`; já `R_ARM_ABS32 g_a` solicita que se insira, naquela palavra específica, o endereço absoluto de `g_a`.

4.3.2 A regra Load-Store: x86 × ARM

Para proceder à comparação entre as duas arquiteturas, gere o *assembly* do `program.c` para ambas, com otimização habilitada (`-O1`), e localize a função `main` em cada um dos arquivos:

```
foo@bar$ gcc -S -O1 program.c -o program-x86.s
foo@bar$ arm-none-eabi-gcc -S -O1 -mcpu=cortex-m4 -mthumb program.c -o program-arm.s
```

Etapa	x86-64	Cortex-M4
Ler <code>g_a</code>	<code>movl g_a(%rip), %edx</code>	<code>ldr r3, .L4 ldr r1, [r3]</code>
Somar <code>g_b</code>	<code>addl g_b(%rip), %edx</code>	<code>ldr r3, .L4+4 ldr r3, [r3] add r1, r1, r3</code>
Chamar <code>printf</code>	<code>call __printf_chk@PLT</code>	<code>bl printf</code>

No x86, a instrução `addl` lê `g_b` **diretamente da memória** e efetua a soma em uma única instrução: trata-se de uma arquitetura CISC, do tipo registrador-memória. No ARM, ambos os valores precisam, primeiramente, ser carregados em registradores, e somente então o `add` opera entre eles: é a regra **Load-Store**, característica do RISC. Observe, ainda, que, com `-O1`, a função `add` deixou de aparecer no `main`, em ambos os casos: o compilador incorporou a soma diretamente ao `main`, em vez de efetuar a chamada da função (*inlining*). No arquivo correspondente ao ARM, `.L4` é o rótulo do *literal pool*, que no `objdump` figurava apenas como um endereço numérico. (No x86, o `printf` foi convertido em `__printf_chk`, uma versão que incorpora verificações de segurança, empregada pelo `gcc` do Ubuntu quando há otimização habilitada.)

4.3.3 A convenção de chamada

Retorne à tabela referente ao `main`: antes de `bl add`, o valor de `g_a` foi transferido para `r0`, e o de `g_b`, para `r1`, retornando o resultado em `r0`. No interior de `add`, sem otimização, o compilador primeiramente armazena `r0` e `r1` na pilha, procede à sua releitura, efetua a soma (`add r3, r2`), deposita o resultado em `r0` (`mov r0, r3`) e retorna por meio de `bx lr`, desviando-se para o endereço armazenado em LR pelo `bl`. Tais escolhas não são arbitrárias por parte do compilador: elas seguem uma convenção padronizada, integrante da EABI, denominada **AAPCS** (*Procedure Call Standard for the Arm Architecture*). É justamente essa convenção que possibilita a compilação separada de arquivos, inclusive com compiladores distintos, e a sua posterior ligação.

Registradores	Papel na AAPCS (Cortex-M)	No x86-64 (Linux)
R0–R3	Os quatro primeiros argumentos inteiros ou ponteiros; os demais vão pela pilha	Seis argumentos em <code>rdi</code> , <code>rsi</code> , <code>rdx</code> , <code>rcx</code> , <code>r8</code> , <code>r9</code>
R0 (e R1, para 64 bits)	Valor de retorno	<code>rax</code> (<code>eax</code> para 32 bits)
R0–R3 e R12	Podem ser alterados pela função chamada: quem chama deve salvá-los, se precisar deles depois (<i>caller-saved</i>)	<code>rax</code> , <code>rcx</code> , <code>rdx</code> , <code>rsi</code> , <code>rdi</code> , <code>r8</code> – <code>r11</code>
R4–R11	Devem ser preservados pela função chamada: se ela os usar, salva e restaura (<i>callee-saved</i>)	<code>rbx</code> , <code>rbp</code> , <code>r12</code> – <code>r15</code>
LR (R14)	Endereço de retorno, gravado pelo <code>bl</code>	Na pilha, gravado pelo <code>call</code>
SP (R13)	Ponteiro de pilha	<code>rsp</code>

4.3.4 Instruções de 16 e de 32 bits

Na coluna referente à codificação, praticamente todas as instruções do `main` apresentam 4 dígitos hexadecimais, ou seja, **16 bits**: trata-se da densidade de código característica do Thumb. Já o `bl` apresenta 8 dígitos, correspondendo a uma instrução de **32 bits**. Modifique o `return` de `add` para `return x + y + 1000`; recompile e proceda a uma nova desmontagem:

```
foo@bar$ arm-none-eabi-objdump -d embedded.o
...
   e: 4413      add r3, r2
  10: f503 737a  add.w  r3, r3, #1000 @ 0x3e8
...
```

A soma envolvendo a constante passou a empregar uma instrução de 32 bits, `add.w`, uma vez que o valor 1000 não se ajusta aos poucos bits de constante disponíveis na forma curta. Instruções de 16 e de 32 bits,

mescladas sem qualquer troca de estado: eis o **Thumb-2**. Restitua o `return` à sua forma original antes de prosseguir.

4.3.5 O que o *linker* fez, e o que ainda falta

Ligue o `embedded.o`, sem a biblioteca C e sem *runtime*, exatamente como se procederá no microcontrolador, e observe onde cada símbolo foi posicionado:

```
foo@bar$ arm-none-eabi-gcc -nostdlib -mcpu=cortex-m4 -mthumb embedded.o -o embedded.elf
.../arm-none-eabi/bin/ld: warning: cannot find entry symbol _start; defaulting to 00008000
foo@bar$ arm-none-eabi-nm -n embedded.elf
      U _start
00008000 T add
0000801a T main
00009044 D g_a
00009048 B g_b
0000904c B g_c
...
```

O aviso referente ao `_start` já é familiar: na ausência de *runtime*, não existe ponto de entrada, e o *linker* opta por um endereço padrão. Os símbolos passam, agora, a apresentar endereços definitivos: `add` e `main` a partir de `0x8000`, e `g_a`, `g_b` e `g_c` nas proximidades de `0x9000`.

As relocações foram devidamente resolvidas. Proceda à desmontagem do executável e compare o `main` com a tabela do objeto:

```
foo@bar$ arm-none-eabi-objdump -d -z embedded.elf
...
802a: f7ff ffe9 bl 8000 <add>
...
8038: 00009044 .word 0x00009044
803c: 00009048 .word 0x00009048
8040: 0000904c .word 0x0000904c
```

Trata-se exatamente do mesmo código, agora posicionado a partir de `0x801a`, com as lacunas devidamente preenchidas. O `bl`, que no objeto correspondia a `f7ff ffe9` e desviava para si mesmo, passou a `f7ff ffe9`, apontando corretamente para `add`, em `0x8000`. As três palavras do *literal pool*, anteriormente zeradas, passam a conter os endereços de `g_a`, `g_b` e `g_c` exibidos pelo `nm`. Foi precisamente isso que as relocações `R_ARM_THM_CALL` e `R_ARM_ABS32` solicitavam.

De onde proveio o valor `0x8000`? Em nenhum momento foi fornecido ao *linker* um roteiro especificando onde posicionar cada seção. Nessas circunstâncias, o `ld` recorre ao seu **linker script padrão**, embutido no próprio programa. Examine-o por meio de `--verbose` e localize o endereço em questão:

```
foo@bar$ arm-none-eabi-ld --verbose | grep 0x8000
      PROVIDE (__executable_start = SEGMENT_START("text-segment", 0x8000));
      . = SEGMENT_START("text-segment", 0x8000);
```

O roteiro padrão determina que o código inicie em `0x8000`, um valor genérico, sem qualquer relação com um microcontrolador específico. (A saída completa pode ser examinada por meio de `arm-none-eabi-ld --verbose | less`: são aproximadamente 300 linhas, e diversos termos, como `SECTIONS`, `.text`, `.data` e `.bss`, voltarão a aparecer quando elaborarmos o nosso próprio roteiro.)

4.3.6 Caminhando para o hardware: Ponteiros e Extensão de Sinal

Até o momento, a análise do Assembly baseou-se inteiramente no processamento de dados alocados na memória de forma convencional pelas seções `.data` e `.bss`. No entanto, o objetivo fundamental de um *firmware bare-metal* consiste em interagir com periféricos através de registradores mapeados em endereços fixos de memória.

Para visualizar essa transição, bem como certos comportamentos intrínsecos da ISA ARM na manipulação de dados parciais (de 8 e 16 bits), elabore o arquivo `blink.c` com o seguinte rascunho de acesso a hardware:

```
#include <stdint.h>

/* Endereços arbitrários simulando os registradores do STM32. */
#define BUTTON_REG ((uint8_t volatile *) 0x40020010U)
#define TEMP_REG    ((int8_t volatile *) 0x40020011U)

void delay(uint32_t ticks)
{
    while (ticks--)
    {
        /* Laço de atraso simulado. */
    }
}

int main(void)
{
    /* Leitura sem sinal (gera zero-extension). */
    uint8_t btn = *BUTTON_REG;

    /* Leitura com sinal (gera sign-extension). */
    int8_t temp = *TEMP_REG;

    if (btn == 1U && temp < 0)
    {
        /* O main() invoca outra função, tornando-se não-folha. */
        delay(100000U);
    }

    return 0;
}
```

Compile este arquivo sem otimização e examine a desmontagem da função `main()`:

```
foo@bar$ arm-none-eabi-gcc -c -g -O0 -mcpu=cortex-m4 -mthumb blinky.c -o blinky.o
foo@bar$ arm-none-eabi-objdump -d blinky.o
```

Ao inspecionar a saída, atente-se a duas características essenciais:

1. **Prólogo e Epílogo (PUSH e POP):** Diferentemente das funções analisadas anteriormente, o `main()` de `blinky.c` efetua uma chamada a `delay()`. Essa dependência obriga o compilador a preservar o valor do *Link Register* (`lr`), uma vez que a instrução `bl` em direção a `delay()` inevitavelmente o sobrescreverá. Consequentemente, o início da função (o prólogo) revela a instrução `push {r7, lr}`, guardando o endereço de retorno seguro na pilha. O final da função (o epílogo) executa a instrução complementar, recarregando esse valor diretamente no *Program Counter*: `pop {r7, pc}`.
2. **Extensão de Sinal (LDRB × LDRSB):** Os processadores ARM operam internamente com dados de 32 bits. Quando a linguagem C solicita a leitura de dados de tamanho inferior (como `uint8_t` ou `int8_t`), a própria instrução de *load* precisa compatibilizar o valor lido com o registrador de destino. Na desmontagem, observe que o compilador converte o acesso ao `BUTTON_REG` (`uint8_t*`) na instrução `ldrb` (*Load Register Byte*), que complementa os 24 bits superiores com zeros. O acesso a `TEMP_REG` (`int8_t*`), contudo, traduz-se em `ldrsb` (*Load Register Signed Byte*), encarregando-se de propagar o bit de sinal do dado lido (o sétimo bit) para toda a extensão superior do registrador de 32 bits, garantindo que o valor -5, por exemplo, permaneça idêntico no interior da ULA.

O que ainda resta providenciar. Nenhum desses endereços faz sentido no contexto do STM32F411, cuja Flash tem início em `0x08000000`, e cuja SRAM tem início em `0x20000000`. E, mesmo que fizessem sentido, faltaria ainda alguém responsável por copiar o 10 de `g_a` para a SRAM e por zerar `g_b` e `g_c` antes da chamada ao `main()`, uma vez que, neste contexto, não há sistema operacional nem *runtime* disponíveis. Faltam, portanto, duas peças fundamentais: um *linker script* próprio, que substitua o padrão e descreva adequadamente a memória do chip, e um *código de inicialização*, que desempenhe o papel do carregador e do *runtime*. Nos capítulos subsequentes, essas peças serão elaboradas, e este mesmo `embedded.c` será executado na placa.

! Atividade para casa

A investigação relativa ao PC prossegue em casa, na [Atividade: Do C ao Binário no PC](#), que independe do uso da placa. O relatório dessa atividade contempla, ainda, duas questões referentes à parte da aula realizada no Cortex-M4.

4.4 Referências

- [1] Yiu, J. *The Definitive Guide to ARM Cortex-M3 and Cortex-M4 Processors*, 3ª edição. Newnes, 2013. Seções 2.3–2.4, 5.3–5.5, 20.2 e Capítulo 17.
- [2] [Using the GNU Compiler Collection \(GCC\) – Overall Options](#) — opções -E, -S, -c, -save-temps, -v, -nostartfiles, -static.
- [3] [GNU Binary Utilities](#) — nm, objdump, size, readelf.
- [4] [Procedure Call Standard for the Arm Architecture \(AAPCS\)](#) — Arm Ltd.
- [5] [Newlib – System Calls](#) — a lista das funções que a newlib espera que o programador forneça.
- [6] Manual do Linux: `man 2 syscalls`, `man 1 strace`, `man 8 ld.so`.

III

Semana 3 — O Microcontrolador STM32F411

5	Os Microcontroladores STM32 e a Série F4	87
5.1	Do núcleo ao microcontrolador	87
5.2	A linha STM32	88
5.3	A documentação de um STM32	89
5.4	A série STM32F4	89
5.5	Os barramentos do STM32F411	90
5.6	O mapa de memória do STM32F411	91
5.7	As memórias do STM32F411	92
5.8	A organização da Flash e a sua proteção	93
5.9	De onde o processador parte: boot e espelhamento	94
5.10	Reset	95
5.11	Alimentação	95
5.12	O sistema de <i>clock</i>	97
5.13	Os registradores que encontraremos primeiro	100
5.14	Do mapa de memória à organização do programa	100
5.15	Questionário de revisão	103
5.16	Referências	104
6	Prática: Código de Inicialização e <i>Linker Script</i>	105
6.1	Preparando a pasta	105
6.2	O que precisa acontecer antes do <code>main()</code>	106
6.3	Parte 1 — O código de inicialização	107
6.4	Parte 2 — O <i>linker script</i>	117
6.5	Juntando as peças	122
6.6	Referências	123

5. Os Microcontroladores STM32 e a Série F4

No capítulo sobre a arquitetura Cortex-M, estudamos o núcleo Cortex-M4, projetado e licenciado pela ARM. Um núcleo isolado, contudo, não constitui um microcontrolador: faltam as memórias, o sistema de *clock*, o circuito de reset e os periféricos, elementos que cabe ao fabricante construir ao redor dele. Neste capítulo, avançamos do núcleo até o chip propriamente dito. Examinamos, primeiramente, aquilo que é comum a todo STM32; em seguida, a série **STM32F4** e, dentro dela, o **STM32F411**, microcontrolador empregado ao longo deste curso. A placa em que ele vem montado será tratada no laboratório.

Objetivos de aprendizagem

Ao final deste capítulo, você deve ser capaz de:

- explicar o que a ST acrescenta ao núcleo Cortex-M para formar um microcontrolador STM32;
- situar a série F4 no portfólio STM32 e decifrar um código de peça como **STM32F411CEU6**;
- escolher, entre *datasheet*, *reference manual* e *programming manual*, o documento certo para cada dúvida;
- descrever a matriz de barramentos e o mapa de memória do STM32F411;
- classificar as memórias do chip (memória de sistema, OTP, Flash, SRAM e *backup*) quanto à volatilidade e à forma de gravação;
- explicar a organização da Flash em setores, o seu desgaste e os níveis de proteção de leitura;
- explicar como os pinos **B00T0** e **B00T1** e o espelhamento em **0x00000000** decidem de onde o processador parte;
- listar as fontes de reset e o estado em que o chip fica depois delas;
- descrever os domínios de alimentação, as escalas de tensão do regulador e os modos de baixo consumo;
- calcular a frequência do sistema a partir das fontes de *clock*, do PLL e dos *prescalers*;
- explicar quando vale a pena executar código da SRAM;
- relacionar o mapa de memória ao bloco **MEMORY** de um *linker script* e dizer em que memória cada parte do programa precisa ficar.

Leitura de Aprofundamento: os manuais da ST

Este capítulo se apoia nos documentos oficiais do STM32F411, que você vai consultar durante todo o curso:

- **Reference manual RM0383**, Seções 2.1 a 2.4 (*Memory and bus architecture*, pp. 36–43), 3.3 a 3.6 (organização, leitura, gravação e proteção da Flash, pp. 44–58), 5.1 a 5.3 (*Power controller*, pp. 70–86), 6.1 e 6.2 (*Reset e Clocks*, pp. 91–103) e 7.2.1 (*SYSCFG memory remap register*, p. 140).
- **Datasheet DS10314**, Seções 2 (*Description*, pp. 12–16), 3.2 (*ART Accelerator*, p. 17), 5 (*Memory mapping*, pp. 54–57), Tabela 47 (*Flash memory endurance and data retention*, p. 94) e 8 (*Ordering information*, p. 141).
- **Livro-texto (Yiu)**, Capítulo 6, Seções 6.1 a 6.3 (*Memory map e Connecting the processor to memory and peripherals*, pp. 193–202), que mostra, do lado do núcleo, como o fabricante liga memórias e periféricos aos barramentos do Cortex-M.

5.1 Do núcleo ao microcontrolador

A ARM entrega ao fabricante o projeto do núcleo, já acompanhado de seus barramentos, do NVIC, do SysTick e dos blocos de depuração (FPB, DWT e ITM). Cabe ao fabricante decidir todo o restante. Em qualquer STM32, encontra-se o mesmo conjunto de blocos ao redor do núcleo:

- **Matriz de barramentos** (*bus matrix*): liga os barramentos do núcleo e os controladores de DMA às memórias e aos periféricos, permitindo acessos simultâneos quando os destinos são diferentes.
- **Memória Flash e sua interface**: guarda o programa. A interface controla gravação e apagamento, protege a memória contra leitura e escrita indevidas e, nas séries mais rápidas, tem um acelerador que esconde a lentidão da Flash.
- **SRAM**: guarda as variáveis e a pilha.
- **Pontes AHB/APB e periféricos**: os periféricos rápidos ficam no barramento AHB, e os demais em barramentos APB mais lentos, ligados ao AHB por pontes.
- **RCC** (*Reset and Clock Control*): gera e distribui os *clocks* e concentra o controle de reset de cada periférico.
- **PWR**: supervisiona a alimentação e controla os modos de baixo consumo.
- **SYSCFG**: configurações gerais do sistema, como o remapeamento da memória vista em `0x00000000`.
- **DMA**: move dados entre periféricos e memória sem ocupar o processador.
- **Memória de sistema**: uma área gravada pela ST na fábrica com um *bootloader*, que permite regravar a Flash por uma interface serial.

Figura: diagrama de blocos genérico de um STM32. No centro, o núcleo Cortex-M com os barramentos I-Code, D-Code e System; abaixo, a matriz de barramentos, com **duas linhas distintas partindo do núcleo até a Flash** (I-Code e D-Code, destacando a natureza Harvard do acesso) e uma linha até a SRAM (System bus); os dois DMAs entrando na matriz como mestres adicionais; a Flash com o acelerador ART; do AHB saem as pontes para APB1 e APB2, com alguns periféricos em cada um; o RCC distribuindo os *clocks* para todos os blocos.

É precisamente essa organização que confere unidade à linha STM32. Um programador familiarizado com um STM32 reconhece, em qualquer outro, a mesma concepção de RCC, de GPIO e de mapa de memória. Alteram-se os endereços, a quantidade de periféricos e os limites de frequência — razão pela qual cada chip possui seus próprios manuais.

5.2 A linha STM32

A ST organiza a linha STM32 em grupos voltados a diferentes combinações de desempenho, consumo e conectividade. Não é necessário memorizar as séries; basta possuir uma noção geral do panorama:

Grupo	Foco	Exemplos de séries	Núcleos usados
Mainstream	Custo-benefício, uso geral	F0, F1, F3, G0, G4	Cortex-M0, M0+, M3, M4
High-Performance	Desempenho e periféricos avançados	F2, F4, F7, H7	Cortex-M3, M4, M7
Ultra-low-power	Consumo mínimo, aplicações a bateria	L0, L1, L4, L5	Cortex-M0+, M3, M4, M33
Wireless	Rádio integrado (BLE, sub-GHz)	WB, WL	Cortex-M4 e M0+

A lista de séries cresce a cada ano. O que efetivamente importa reter é que a letra posicionada após **STM32** indica a série, e que o mesmo núcleo pode figurar em séries distintas: o Cortex-M4, por exemplo, está presente nas séries F3, F4, G4, L4, WB e WL (essa é, precisamente, a lista de produtos abrangida pelo *programming manual* PM0214).

5.2.1 Decifrando o código de uma peça

O nome completo de um STM32 descreve o chip campo a campo. O *datasheet* apresenta a tabela de decodificação na seção *Ordering information*. Para o STM32F411:

Campo	STM32	F	411	C	E	U	6
Significado	Família de MCUs de 32 bits da ST	Tipo: uso geral (<i>general-purpose</i>)	Sub-família 411	Número de pinos: 48/49	Flash: 512 KB	Encapsulamento: UFQFPN	Temperatura: -40 a 85 °C

Ao trocar uma única letra, o chip se altera por completo. Um **STM32F411RET6** corresponde ao mesmo microcontrolador, porém em encapsulamento LQFP (T) de 64 pinos (R). Um **STM32F411CCU6**, por sua vez, dispõe de apenas 256 KB de Flash (C). Tudo o que é interno ao chip — o mapa de memória, os registradores e o sistema de *clock* — permanece válido para todas essas variações; o que efetivamente se altera é a quantidade de pinos disponíveis e o tamanho da Flash.

5.3 A documentação de um STM32

Programar um microcontrolador em nível de registrador consiste, em grande medida, em saber ler os manuais do fabricante. A ST distribui a informação em documentos com papéis bem definidos:

Documento	O que traz	Exemplo no curso
Datasheet (DS)	Características de um chip específico: memórias, encapsulamentos, pinagem, funções alternativas de cada pino, características elétricas, código de peça	DS10314 (STM32F411xC/xE)
Reference manual (RM)	Funcionamento de cada periférico e a descrição, bit a bit, de todos os seus registradores	RM0383 (STM32F411xC/E)
Programming manual (PM)	O núcleo do ponto de vista do programador: modelo de programação, conjunto de instruções, NVIC, SysTick, MPU	PM0214 (Cortex-M4 da ST)
Errata sheet (ES)	Defeitos conhecidos do silício e como contorná-los	Uma por chip
Application note (AN)	Como resolver um problema específico	AN2606 (os <i>bootloaders</i> de fábrica)
Arquivo SVD	A lista de periféricos e registradores do chip em formato legível por programas	STM32F411.svd

A regra prática é a seguinte: **o que o chip é** encontra-se no *datasheet*; **como o periférico funciona** encontra-se no *reference manual*; **como o núcleo funciona** encontra-se no *programming manual* (ou no livro do Yiu). Uma dúvida a respeito de qual pino pode servir de saída para uma USART resolve-se no *datasheet*. Uma dúvida sobre o que determinado bit do **RCC_CFGR** realiza resolve-se no *reference manual*.

O último item da tabela não constitui um documento destinado à leitura. O **SVD** (*System View Description*) corresponde a um formato definido pela ARM no padrão CMSIS: um arquivo XML que descreve, para um dado chip, cada periférico com seu respectivo endereço-base, e cada registrador com seu deslocamento, valor de reset e campos de bits (posição, largura, tipo de acesso e uma breve descrição). Trata-se, essencialmente, da parte de registradores do *reference manual*, escrita para ser processada por programas. Dois usos se destacam ao longo do curso:

- **Depuradores:** com o SVD, o depurador mostra os registradores dos periféricos pelo nome e campo a campo (o **RCC_CR** com os bits **HSION**, **HSIRDY** etc.), em vez de só um endereço e um número.
- **Geração de código:** como o SVD é estruturado, um programa pode lê-lo e gerar automaticamente as definições em C dos periféricos. Os *headers* de dispositivo que acompanham as bibliotecas dos fabricantes são produzidos assim.

O SVD, contudo, não substitui o *reference manual*. Ele indica onde cada bit se encontra, mas não explica o que o periférico faz nem em que ordem os registradores devem ser programados. Pode, ainda, conter erros: existem projetos da comunidade dedicados a manter correções para os SVDs dos STM32. O *reference manual* permanece, portanto, como a referência final, e qualquer conteúdo gerado a partir do SVD deve ser conferido em relação a ele.

5.4 A série STM32F4

A série F4 pertence ao grupo *High-Performance* e emprega o **Cortex-M4 com FPU** de precisão simples, acompanhado das instruções de DSP e da MPU. Ela reúne diversas linhas de produtos, compatíveis entre si em software e, em boa medida, também em pinagem. O *datasheet* do F411 relaciona as linhas com as quais ele é compatível:

- STM32F401, F410, F411, F412, F413 e F423;
- STM32F405/415, F407/417, F427/437 e F429/439;

- STM32F446.

A frequência máxima, os tamanhos de memória e os periféricos disponíveis variam de uma linha para outra. O STM32F407, bastante frequente em material didático mais antigo, atinge 168 MHz e dispõe de Ethernet, CAN e controlador de memória externa. O STM32F411 atinge 100 MHz e não possui nenhum desses três recursos. As ideias apresentadas neste capítulo aplicam-se a toda a série, mas **os números apresentados aplicam-se especificamente ao F411**. Para qualquer outro membro da série, é necessário consultar o *reference manual* correspondente.

5.4.1 O STM32F411

O STM32F411 integra a linha que a ST denomina *Dynamic Efficiency*, cujo propósito é buscar equilíbrio entre desempenho e consumo. As características do chip, segundo a Tabela 2 do *datasheet*, são as seguintes:

Característica	STM32F411xC	STM32F411xE
Núcleo	Cortex-M4 com FPU, até 100 MHz	Cortex-M4 com FPU, até 100 MHz
Flash	256 KB	512 KB
SRAM	128 KB	128 KB
Timers	7 de uso geral e 1 de controle avançado	7 de uso geral e 1 de controle avançado
Comunicação	3 USART, 3 I2C, 5 SPI/I2S, 1 SDIO, 1 USB 2.0 OTG <i>Full-Speed</i>	Idem
ADC	1 conversor de 12 bits, 10 ou 16 canais conforme o encapsulamento	Idem
GPIOs	36, 50 ou 81, conforme o encapsulamento (48/49, 64 ou 100 pinos)	Idem
Alimentação	1,7 V a 3,6 V	1,7 V a 3,6 V

5.5 Os barramentos do STM32F411

No capítulo sobre o Cortex-M, verificamos que o núcleo dispõe de três barramentos: I-Code, destinado à busca de instruções; D-Code, destinado à leitura de constantes e dados na região de código; e System, destinado à SRAM e aos periféricos. No STM32F411, esses três barramentos convergem para uma **matriz de barramentos** (*multi-layer AHB bus matrix*), juntamente com os barramentos dos dois controladores de DMA, totalizando seis **mestres**. Do outro lado da matriz, encontram-se cinco **escravos**:

- a Flash, pelo caminho I-Code;
- a Flash, pelo caminho D-Code;
- a SRAM;
- o barramento AHB1, com os periféricos rápidos e as duas pontes para os barramentos APB1 e APB2;
- o barramento AHB2, onde fica o USB OTG.

A matriz conecta cada mestre ao escravo que este pretende acessar, permitindo acessos simultâneos sempre que os escravos envolvidos sejam distintos: o núcleo pode, por exemplo, buscar uma instrução na Flash enquanto o DMA escreve na SRAM. Quando dois mestres disputam o mesmo escravo, a matriz arbitra a disputa em rodízio (*round-robin*).

Como evidencia a figura anterior, a Flash figura como **dois escravos distintos** no interior da matriz — um correspondente ao caminho I-Code, outro ao D-Code. É justamente essa duplicação física até a mesma memória que torna concreta, no próprio chip, a arquitetura Harvard modificada discutida no capítulo do Cortex-M: dois barramentos fisicamente distintos conduzem ao mesmo endereço, unificados, do ponto de vista de quem programa, em um único espaço de 4 GB.

Entre a Flash e a matriz situa-se o **ART Accelerator**, uma fila de pré-busca de instruções acompanhada de uma pequena memória *cache* destinada aos desvios. Sua existência justifica-se pelo fato de a Flash ser mais lenta que o processador, assunto ao qual retornaremos na seção dedicada aos *clocks*.

Cabe destacar uma regra importante, válida para todos os STM32: **imediatamente após um reset, o clock de todos os periféricos permanece desligado**, com exceção da SRAM e da interface da Flash. Antes de utilizar qualquer periférico, o programa precisa habilitar o seu *clock* no registrador de habilitação

do barramento correspondente (`RCC_AHB1ENR`, `RCC_APB1ENR` etc.). Um periférico privado de *clock* simplesmente não responde — razão pela qual o primeiro passo de todo programa que aciona um LED consiste em habilitar o *clock* da porta GPIO correspondente.

5.6 O mapa de memória do STM32F411

O Cortex-M define o mapa de memória de 4 GB em regiões fixas: código a partir de `0x00000000`, SRAM a partir de `0x20000000`, periféricos a partir de `0x40000000` e os periféricos internos do núcleo a partir de `0xE0000000`. O fabricante ocupa apenas uma pequena parte de cada região. No caso do STM32F411xE, a ocupação é a seguinte (Figura 14 do *datasheet* e Tabela 3 do RM0383):

Faixa de endereços	Conteúdo
<code>0x0000 0000 – 0x0007 FFFF</code>	Espelho da memória de partida: a Flash, a memória de sistema ou a SRAM, conforme os pinos de boot
<code>0x0800 0000 – 0x0807 FFFF</code>	Flash de programa (512 KB)
<code>0x1FFF 0000 – 0x1FFF 77FF</code>	Memória de sistema (30 KB), com o <i>bootloader</i> da ST
<code>0x1FFF 7800 – 0x1FFF 7A0F</code>	Área OTP (<i>one-time programmable</i>), 528 bytes
<code>0x1FFF C000 – 0x1FFF C00F</code>	<i>Option bytes</i> (proteções, nível do BOR, configuração dos <i>watch-dogs</i>)
<code>0x2000 0000 – 0x2001 FFFF</code>	SRAM (128 KB)
<code>0x2200 0000</code> em diante	Região <i>alias</i> de <i>bit-band</i> da SRAM
<code>0x4000 0000</code> em diante	Periféricos do barramento APB1
<code>0x4001 0000</code> em diante	Periféricos do barramento APB2
<code>0x4002 0000</code> em diante	Periféricos do barramento AHB1
<code>0x4200 0000</code> em diante	Região <i>alias</i> de <i>bit-band</i> dos periféricos
<code>0x5000 0000 – 0x5003 FFFF</code>	Periférico do barramento AHB2 (USB OTG FS)
<code>0xE000 0000 – 0xE00F FFFF</code>	Periféricos internos do Cortex-M4 (NVIC, SysTick, SCB, depuração)

Os endereços que não figuram na tabela são **reservados**: não há memória nem periférico algum a eles associado.

Cada periférico ocupa um bloco próprio, a partir de um **endereço-base** fixo. Alguns exemplos, extraídos da Tabela 1 do RM0383:

Periférico	Barramento	Endereço-base
TIM2	APB1	<code>0x4000 0000</code>
USART2	APB1	<code>0x4000 4400</code>
USART1	APB2	<code>0x4001 1000</code>
SYSCFG	APB2	<code>0x4001 3800</code>
GPIOA	AHB1	<code>0x4002 0000</code>
GPIOC	AHB1	<code>0x4002 0800</code>
RCC	AHB1	<code>0x4002 3800</code>
Interface da Flash	AHB1	<code>0x4002 3C00</code>

Cada registrador de um periférico situa-se em um **deslocamento** (*offset*) a partir do endereço-base. O `RCC_CR`, por exemplo, encontra-se no deslocamento `0x00` do RCC, ao passo que o `RCC_CFGR` encontra-se no deslocamento `0x08`, isto é, em `0x4002 3808`. Somar a base ao deslocamento e acessar o endereço resultante

por meio de um ponteiro em C é tudo o que se necessita para controlar um periférico, sem depender de nenhuma biblioteca intermediária.

5.7 As memórias do STM32F411

No capítulo de [Fundamentos da Arquitetura de Computadores](#), separamos as memórias em dois grupos: as **voláteis**, que perdem o conteúdo na ausência de energia, e as **não voláteis**, que o preservam. O STM32F411 constitui um excelente exemplo para revisitar essa divisão, pois seu mapa de memória contempla praticamente todos os tipos existentes em um microcontrolador:

Memória	Onde fica	Volátil?	Quem grava, e como
Memória de sistema	0x1FFF 0000	Não	Só a ST, na fábrica. Para o programa, funciona como uma ROM
OTP	0x1FFF 7800	Não	O programa, uma única vez : 512 bytes em 16 blocos, cada um com um byte de trava
Flash	0x0800 0000	Não	O programa, por um procedimento especial da interface da Flash (desbloquear, apagar o setor, gravar)
SRAM	0x2000 0000	Sim	O programa, livremente, com uma instrução STR comum
Registradores de backup	No RTC	Sim, mas sobrevivem com bateria em VBAT	O programa: 20 registradores de 32 bits (80 bytes)

Aquilo que diferencia, no grupo não volátil, a ROM, a OTP e a Flash reside, fundamentalmente, em **quantas vezes e por quem** a memória pode ser escrita. A memória de sistema foi escrita uma única vez, pela própria ST. A OTP pode ser escrita uma única vez, por nós, e destina-se a gravar um número de série ou uma chave de calibração que jamais deve ser alterada. A Flash, por sua vez, pode ser regravada milhares de vezes. Já a SRAM pode ser escrita a cada ciclo, mas perde todo o conteúdo quando a energia se esgota. Os registradores de *backup* situam-se numa posição intermediária: são voláteis, mas preservam o conteúdo enquanto houver uma bateria conectada ao pino VBAT.

Resta um tipo de memória presente em muitos microcontroladores, mas ausente aqui: o STM32F411 **não possui EEPROM**, a memória não volátil que se grava byte a byte. Quando uma aplicação necessita armazenar configurações capazes de sobreviver ao desligamento, ela as grava em um setor da Flash, procedimento que a ST descreve, em suas notas de aplicação, como uma forma de emular uma EEPROM.

5.7.1 Gravar a Flash não é como escrever na SRAM

Para escrever na SRAM, basta uma instrução STR. Na Flash, contudo, esse mesmo STR não grava absolutamente nada. Isso não significa que a Flash seja somente leitura: ela é, sim, gravada pelo próprio programa, por meio da interface da Flash, seguindo um procedimento que o RM0383 descreve na Seção 3.5:

1. **Desbloquear** o registrador de controle FLASH_CR, escrevendo duas chaves fixas, em sequência, no FLASH_KEYR. O bloqueio existe para que uma interferência elétrica ou um ponteiro errado não apague a memória por acidente.
2. **Apagar** o setor. Apagar leva todos os bits a 1: uma Flash apagada é lida como 0xFF.
3. **Gravar**, ativando o bit de programação e só então escrevendo os dados. A gravação só consegue levar bits de 1 para 0. Para voltar um bit a 1, é preciso apagar o setor inteiro de novo.

Enquanto uma gravação ou um apagamento está em curso, qualquer leitura da Flash permanece **suspensa**, aguardando o término da operação. Isso inclui, inclusive, a busca de instruções: se o programa responsável por gravar a Flash estiver, ele próprio, sendo executado a partir dela, permanecerá congelado até que a operação se conclua. Vale reter essa observação para a seção dedicada à execução de código a partir da SRAM.

É exatamente esse procedimento que o `st-flash` e o OpenOCD executam, por intermédio do depurador, no momento em que gravamos um programa.

5.8 A organização da Flash e a sua proteção

5.8.1 Setores

A Flash principal do STM32F411xE não se divide em blocos de igual tamanho. Seus 512 KB formam oito **setores**, distribuídos em três tamanhos distintos (Tabela 4 do RM0383):

Setor	Endereços	Tamanho
0 a 3	0x0800 0000 – 0x0800 FFFF	16 KB cada
4	0x0801 0000 – 0x0801 FFFF	64 KB
5 a 7	0x0802 0000 – 0x0807 FFFF	128 KB cada

O setor constitui a menor porção passível de ser apagada. Alterar um único byte já gravado exige apagar o setor inteiro em que ele reside e regravar integralmente aquilo que precisava ser mantido. Por essa razão, dados que se modificam com frequência não devem compartilhar o setor com o programa.

A Flash, além disso, se desgasta. O *datasheet* garante, no mínimo, **10 mil ciclos** de apagamento e gravação por setor, além de uma retenção de dados de 20 anos a 55 °C após esses 10 mil ciclos (Tabela 47 do DS10314). Para gravar o programa algumas vezes ao dia, essa margem é mais do que suficiente. Já para uma aplicação que grava um valor por segundo no mesmo setor, a Flash se esgota em poucas horas. O desgaste constitui, portanto, um requisito de projeto, e não um detalhe menor.

5.8.2 Option bytes e proteção

Os **option bytes**, situados em 0x1FFF C000, constituem uma pequena área não volátil destinada a armazenar configurações do chip que não pertencem ao programa em si: o nível de proteção de leitura, a proteção de escrita de cada setor, o limiar do *brownout reset* e o modo dos *watchdogs*. São gravados por um procedimento semelhante ao da Flash, mediante chaves próprias.

A **proteção de leitura** (RDP, *read protection*) impede que terceiros copiem o programa gravado em um produto. Ela se organiza em três níveis:

Nível	Efeito	Reversível?
0	Sem proteção: o depurador lê e grava a Flash livremente	—
1	Com depurador conectado, ou partindo da SRAM ou do <i>bootloader</i> , a Flash não pode ser lida, gravada nem apagada. Partindo da Flash, o próprio programa funciona normalmente	Sim, mas voltar ao nível 0 apaga toda a Flash
2	Tudo do nível 1 e mais: a porta de depuração é desativada para sempre , a partida pela SRAM e pelo <i>bootloader</i> é bloqueada e os <i>option bytes</i> ficam congelados	Não

⚠ O nível 2 é irreversível

Um chip com RDP no nível 2 jamais volta a aceitar um depurador. Ele continua executando o programa já gravado, e apenas esse programa — caso tenha sido escrito com esse propósito — consegue regravar a Flash. Nem mesmo a ST consegue analisar um chip nessa condição. Em um produto final, é precisamente esse o efeito desejado; em uma placa de desenvolvimento, trata-se de uma placa irremediavelmente perdida. Jamais altere os *option bytes* sem saber, com absoluta precisão, o que está fazendo.

Além da RDP, cada setor pode receber, individualmente, uma **proteção de escrita**, que impede que ele seja apagado ou gravado por engano — útil, por exemplo, para proteger um *bootloader* próprio. Há, ainda, a **PCROP** (*proprietary code readout protection*), que marca um setor como executável, porém não legível: o código ali contido é executado normalmente, mas nem o próprio programa consegue ler seus bytes como dados. Esse recurso é útil para distribuir uma biblioteca compilada sem revelar o seu conteúdo.

5.9 De onde o processador parte: boot e espelhamento

Ao sair do reset, o Cortex-M4 lê o valor inicial do SP no endereço `0x00000000` e o endereço do tratador de reset em `0x00000004` — ou seja, na região de código, acessada por meio dos barramentos I-Code e D-Code. No STM32F411, porém, a Flash tem início em `0x08000000`. Como pode, então, a mesma arquitetura partir da Flash, da memória de sistema ou da SRAM, indistintamente?

A resposta reside no **espelhamento** (*aliasing*). A faixa a partir de `0x00000000` não possui memória própria: ela apenas exhibe uma das três memórias do chip, selecionada pelos pinos `B00T0` e `B00T1`:

B00T1	B00T0	Modo de partida	Memória vista em <code>0x00000000</code>
x	0	Flash principal	Flash (<code>0x0800 0000</code>)
0	1	Memória de sistema	Bootloader da ST (<code>0x1FFF 0000</code>)
1	1	SRAM embarcada	SRAM (<code>0x2000 0000</code>)

Três detalhes completam esse mecanismo:

- **O momento da leitura dos pinos.** O valor de `B00T0` e `B00T1` é capturado na 4ª borda de subida do *clock* do sistema após o reset, e novamente na saída do modo *Standby*. A partir desse instante, os pinos podem variar sem qualquer efeito.
- **Os pinos propriamente ditos.** `B00T0` corresponde a um pino dedicado. `B00T1`, por sua vez, compartilha o pino com uma GPIO, a `PB2`, que se torna disponível para uso comum após a leitura. Em outras famílias STM32, `B00T1` pode corresponder a um bit de configuração (*option bit*), em vez de um pino físico — convém, portanto, sempre consultar o manual do chip específico.
- **A permanência da memória em seu endereço original.** Com o boot pela Flash, a primeira palavra da Flash aparece **simultaneamente** em `0x00000000` e em `0x08000000`. O espelho constitui apenas uma segunda janela para a mesma memória. Após a partida, o software ainda pode alterar a memória espelhada, escrevendo no campo `MEM_MODE` do registrador `SYS_CFG_MEMRMP`.

O modo **memória de sistema** executa o *bootloader* gravado pela ST na fábrica, o qual regrava a Flash por meio de uma interface serial: USART1, USART2, I2C1 a I2C3, SPI1 a SPI3 ou USB OTG FS no modo DFU (*Device Firmware Upgrade*). A nota de aplicação AN2606 descreve esse protocolo em detalhe. É justamente esse modo que possibilita gravar um chip sem gravador algum, valendo-se apenas de um cabo USB. Neste curso, contudo, a gravação ocorre pelo SWD com o ST-LINK, procedimento independente do modo de partida.

O modo **SRAM** é empregado em depuração e em situações particulares. Como a SRAM perde seu conteúdo na ausência de alimentação, o programa precisa ser carregado nela por um depurador antes de cada partida — circunstância que suscita uma pergunta legítima: se a SRAM nasce vazia, **quem**, então, grava o programa nela antes que qualquer código seja executado?

! Quem copia o programa para a SRAM, se nenhum código ainda rodou?

Quem realiza essa tarefa é o próprio **depurador**, e não algum código do chip. O ST-LINK acessa a SRAM pela porta de depuração (SWD), caminho elétrico que escreve diretamente na memória por meio do barramento, sem depender de nenhuma instrução em execução pelo núcleo — é esse exatamente o mesmo canal que o depurador utiliza para ler e escrever registradores e memória quando se estabelece um *breakpoint*. A sequência típica transcorre da seguinte forma:

1. o depurador mantém o núcleo **parado**, logo na saída do reset, antes que este leia o SP de `0x00000000` e o PC de `0x00000004`;
2. com o núcleo parado, o depurador grava o programa inteiro na SRAM, palavra por palavra, por meio da porta de depuração;
3. o depurador, então, libera o núcleo — ou, o que é mais frequente na prática, ajusta diretamente os registradores SP e PC pela própria porta de depuração, sem depender da leitura automática do espelho;
4. com `B00T1 = 1` e `B00T0 = 1`, o endereço `0x00000000` espelha a SRAM; assim, caso o núcleo seja liberado pelo caminho “normal”, ele lerá ali exatamente aquilo que o depurador acabou de gravar.

É precisamente esse mesmo mecanismo — escrever diretamente na memória pela porta de depuração, com o núcleo parado — que será empregado “manualmente” na atividade [Explorando o Cortex-M com o Depurador](#), ao escrever instruções em `0x20000000` e ajustar o PC de forma direta.

Figura: sequência em quatro passos, lado a lado. (1) Núcleo em reset, SRAM vazia (hachurada), depurador conectado por SWD; (2) depurador escrevendo bytes na SRAM pela porta de depuração, núcleo ainda parado; (3) depurador ajustando SP e PC diretamente (ou liberando o núcleo); (4) núcleo executando a partir da SRAM, com uma seta de `0x00000000` apontando para o conteúdo gravado no passo 2.

5.10 Reset

O reset conduz o chip a um estado conhecido. O RM0383 distingue três tipos, cada um com alcance distinto.

Reset de sistema. Conduz todos os registradores aos seus valores de reset, com exceção dos *flags* de reset no `RCC_CSR` e do domínio de *backup*. É gerado por:

- nível baixo no pino `NRST` (reset externo, o botão de reset de uma placa);
- fim de contagem do *watchdog* de janela (WWDG);
- fim de contagem do *watchdog* independente (IWDG);
- reset por software, pedido pelo bit `SYSRESETREQ` do registrador `AIRCR` do núcleo;
- reset de gerenciamento de baixo consumo, quando configurado nos *option bytes* para ocorrer no lugar dos modos *Stop* ou *Standby*.

Reset de alimentação (*power reset*). Realiza tudo o que o reset de sistema realiza, e adicionalmente zera os *flags* de reset. É gerado:

- ao ligar ou desligar a alimentação (POR/PDR, *power-on/power-down reset*);
- quando a tensão cai abaixo de um limiar configurável (BOR, *brownout reset*);
- na saída do modo *Standby*.

Reset do domínio de *backup*. Afeta exclusivamente o RTC e os registradores de *backup*, que podem permanecer alimentados por uma bateria (`VBAT`) mesmo quando o restante do chip está desligado.

Qualquer fonte interna de reset também conduz o pino `NRST` a nível baixo por, no mínimo, 20 μ s, de modo que circuitos externos conectados a esse pino sejam igualmente reiniciados. Para determinar o que causou o último reset, o programa lê os *flags* do registrador `RCC_CSR` (`PINRSTF`, `PORRSTF`, `BORRSTF`, `SFTRSTF`, `IWDGRSTF`, `WWDGRSTF` e `LPWRRSTF`) e, em seguida, os apaga escrevendo 1 no bit `RMVF`. Em um produto real, esse diagnóstico permite distinguir, por exemplo, um desligamento normal de um travamento que disparou o *watchdog*.

Após qualquer reset, o chip encontra-se invariavelmente no mesmo estado:

- o *clock* do sistema vem do oscilador interno **HSI, de 16 MHz**;
- a Flash está configurada com **zero estados de espera**;
- os *clocks* de todos os periféricos estão desligados, com exceção da SRAM e da interface da Flash;
- os pinos de boot já foram lidos, e o núcleo carrega o SP de `0x00000000` e o PC de `0x00000004`.

A partir desse ponto, tudo o que o chip vier a realizar passa a ser responsabilidade exclusiva do nosso programa.

5.11 Alimentação

5.11.1 Domínios de alimentação

O chip recebe energia por mais de um pino, cada qual alimentando uma parte distinta (Seção 5.1 do RM0383):

Pino	Faixa	Alimenta
<code>VDD</code>	1,7 V a 3,6 V	Os pinos de I/O e o regulador interno
<code>VDDA</code>	Deve ser ligado ao <code>VDD</code>	O ADC e os circuitos analógicos (osciladores RC, PLL), separado para poder ser filtrado contra ruído
<code>VBAT</code>	1,65 V a 3,6 V	O domínio de <i>backup</i> (RTC, oscilador LSE e registradores de <i>backup</i>) quando <code>VDD</code> está desligado

Na ausência de bateria, o `VBAT` deve ser conectado ao `VDD`.

O núcleo, as memórias e os periféricos digitais **não** operam em 3,3 V. Um **regulador linear interno** gera, a partir do `VDD`, uma tensão de aproximadamente 1,2 V destinada a toda essa lógica, exigindo um

capacitor externo no pino `VCAP_1`. O microcontrolador possui, portanto, duas tensões distintas: aquela presente nos pinos, mensurável com um multímetro, e aquela do núcleo, interna ao chip.

5.11.2 Escalas de tensão e a frequência máxima

A tensão de saída do regulador pode ser ajustada por software em três **escalas** distintas, no campo `VOS` do registrador `PWR_CR`. Uma tensão inferior reduz o consumo, mas, em contrapartida, limita a frequência máxima do *clock*:

Escala	Frequência máxima do HCLK
3	64 MHz
2 (valor após o reset)	84 MHz
1	100 MHz

Isso completa o que já foi discutido a respeito dos *clocks*: os 100 MHz do STM32F411 só são permitidos na escala 1, ao passo que o chip deixa o reset na escala 2. Para alcançar 100 MHz, o programa precisa habilitar o *clock* do controlador de energia (bit `PWREN` do `RCC_APB1ENR`) e alterar o `VOS` para a escala 1 **antes** de ligar o PLL, uma vez que esse campo só admite alteração com o PLL desligado. Com o PLL desligado, o regulador permanece sempre na escala 3, o que não constitui problema, já que, sem PLL, o *clock* não excede os 16 MHz do HSI ou a frequência do cristal.

5.11.3 Supervisão da alimentação

Três circuitos distintos vigiam a tensão de alimentação:

- **POR/PDR** (*power-on reset/power-down reset*): mantém o chip em reset enquanto o `VDD` permanecer abaixo de um limiar mínimo, tanto ao ligar quanto ao desligar. Dispensa, assim, um circuito de reset externo.
- **BOR** (*brownout reset*): reinicia o chip quando o `VDD` cai abaixo de um limiar mais elevado, selecionável entre três níveis nos *option bytes*. Vem **desligado** de fábrica. Sua função é fazer com que uma queda parcial de tensão, comum em baterias fracas, produza um reset limpo, em vez de um comportamento errático.
- **PVD** (*programmable voltage detector*): não reinicia absolutamente nada. Compara o `VDD` a um limiar programável e gera uma interrupção, permitindo que o programa salve o que for necessário antes que a energia se esgote.

Os dois primeiros constituem, precisamente, as fontes do reset de alimentação discutidas na seção anterior.

5.11.4 Modos de baixo consumo

Após o reset, o chip encontra-se no modo **Run**, executando o programa. Quando não há nenhuma tarefa a realizar, ele pode ingressar em um de três modos de baixo consumo, os quais trocam economia de energia por tempo de retorno (Tabela 14 do RM0383):

Modo	O que para	O que se mantém	Como acorda
Sleep	Só o <i>clock</i> do núcleo	Periféricos, SRAM e registradores	Qualquer interrupção
Stop	Todos os <i>clocks</i> do domínio de 1,2 V; HSI e HSE desligados	SRAM e registradores	Uma linha do EXTI (um pino, o RTC etc.)
Standby	O regulador de 1,2 V	Só o domínio de <i>backup</i>	Pino <code>WKUP</code> , alarme do RTC, <code>NRST</code> ou <code>IWDG</code> . O retorno é um reset

O modo *Standby* é o mais econômico de todos, mas, ao despertar, o programa recomeça inteiramente do zero, uma vez que o conteúdo da SRAM se perde. A escolha do modo constitui uma das decisões centrais no projeto de um produto alimentado por bateria, assunto ao qual retornaremos no módulo dedicado ao baixo consumo.

5.12 O sistema de *clock*

Todo circuito digital síncrono necessita de um *clock*. No microcontrolador, a frequência do *clock* determina a velocidade de execução, o consumo de energia (que se eleva com a frequência) e a base de tempo dos periféricos: a taxa de uma USART, a frequência de um PWM e o período de um *timer* derivam-se, todos, dele. O RCC existe justamente para gerar esse *clock* a partir de fontes distintas e distribuí-lo, devidamente dividido, a cada parte do chip.

5.12.1 As fontes

O STM32F411 tem quatro osciladores e um PLL:

Fonte	Tipo	Frequência	Uso típico
HSI (<i>High-Speed Internal</i>)	Oscilador RC interno	16 MHz	<i>Clock</i> do sistema após o reset; entrada do PLL
HSE (<i>High-Speed External</i>)	Cristal ou ressonador externo, ou <i>clock</i> externo	4 a 26 MHz	<i>Clock</i> do sistema ou entrada do PLL, quando a precisão importa
LSI (<i>Low-Speed Internal</i>)	Oscilador RC interno	cerca de 32 kHz	<i>Watchdog</i> independente; RTC, opcionalmente
LSE (<i>Low-Speed External</i>)	Cristal externo	32,768 kHz	RTC (relógio e calendário)
PLL principal	Multiplicador de frequência	até 100 MHz no <i>clock</i> do sistema	<i>Clock</i> do sistema em alta frequência; 48 MHz para o USB

O HSI não requer componente externo algum e parte rapidamente, mas é impreciso: a ST o calibra na fábrica para um erro de 1% a 25 °C, e seu valor varia conforme a tensão e a temperatura. O HSE, por sua vez, depende de um cristal presente na placa, mas é significativamente mais preciso. Determinadas interfaces não toleram o erro do HSI; o próprio *bootloader* da ST, por exemplo, só emprega o USB quando há um HSE presente. O valor do cristal depende da placa utilizada, e o valor correspondente à placa deste curso será examinado no laboratório.

O LSE, de 32,768 kHz, possui esse valor por um motivo simples: $32\,768 = 2^{15}$, de modo que um contador binário de 15 bits alimentado por ele transborda exatamente uma vez por segundo.

5.12.2 O PLL

Um cristal de alta frequência é dispendioso e consome mais energia. Em seu lugar, o chip emprega um **PLL** (*Phase-Locked Loop*), circuito capaz de gerar uma frequência elevada, múltipla de uma frequência de referência mais baixa. O PLL principal do STM32F411 recebe o HSI ou o HSE como entrada e dispõe de três fatores configuráveis no registrador `RCC_PLLCFGR`:

$$f_{VCO} = f_{entrada} \times \frac{N}{M} \quad f_{PLL} = \frac{f_{VCO}}{P} \quad f_{USB} = \frac{f_{VCO}}{Q} \quad (5.1)$$

Cada fator tem limites que o programa precisa respeitar:

Fator	Função	Valores permitidos	Restrição
M	Divide a entrada	2 a 63	Entrada do VCO entre 1 e 2 MHz (2 MHz recomendado, para menor <i>jitter</i>)
N	Multiplica	50 a 432	Saída do VCO entre 100 e 432 MHz
P	Divide para o <i>clock</i> do sistema	2, 4, 6 ou 8	Resultado de no máximo 100 MHz
Q	Divide para USB e SDIO	2 a 15	O USB exige exatamente 48 MHz

Tome-se, como exemplo, o HSI de 16 MHz:

- **M** = 8 conduz a entrada do VCO a $16 / 8 = 2$ MHz;
- **N** = 100 conduz o VCO a $2 \times 100 = 200$ MHz;
- **P** = 2 conduz o *clock* do sistema a $200 / 2 = 100$ MHz.

Nessa configuração, nenhum valor inteiro de **Q** produz 48 MHz, uma vez que $200 / 48$ não resulta em número inteiro. Quem necessita do USB deve escolher outro ponto de operação — um VCO de 192 MHz (**N** = 96), por exemplo, resulta em 96 MHz no sistema com **P** = 2 e em 48 MHz no USB com **Q** = 4. Selecionar os fatores do PLL consiste, quase invariavelmente, em negociar entre as frequências de que cada parte do sistema necessita.

Os fatores só admitem alteração com o PLL desligado. Por essa razão, a ordem a seguir deve ser sempre respeitada: configurar **M**, **N**, **P** e **Q**; ligar o PLL (bit **PLLON** do **RCC_CR**); e aguardar que o bit **PLLRDY** indique que ele travou. Para exceder 84 MHz, a escala de tensão do regulador precisa ser ajustada antes de ligar o PLL, como já discutido na seção sobre alimentação.

5.12.3 O *clock* do sistema e os *prescalers*

O *clock* do sistema (SYSCLK) pode originar-se do HSI, do HSE ou do PLL, seleção feita pelo campo **SW** do **RCC_CFGR**. A troca só se efetiva quando a nova fonte estiver pronta, e o campo **SWS** do mesmo registrador informa qual fonte está, de fato, em uso. A partir do SYSCLK, três divisores (*prescalers*) geram os *clocks* dos barramentos:

<i>Clock</i>	Alimenta	Divisor (a partir de)	Máximo no F411
HCLK	Núcleo, barramento AHB, memórias e DMA	AHB: 1, 2, 4, ..., 512 (do SYSCLK)	100 MHz
PCLK2	Periféricos do barramento APB2	APB2: 1, 2, 4, 8, 16 (do HCLK)	100 MHz
PCLK1	Periféricos do barramento APB1	APB1: 1, 2, 4, 8, 16 (do HCLK)	50 MHz

Com o SYSCLK em 100 MHz, os divisores do AHB e do APB2 podem permanecer em 1, mas o do APB1 precisa valer, no mínimo, 2, de modo a não exceder 50 MHz. Três detalhes do RM0383 manifestam-se com frequência na prática:

- **Os *timers* recebem um fator 2.** Na configuração padrão (bit **TIMPRE** do **RCC_DCKCFGR** em 0), sempre que o divisor de um barramento APB for diferente de 1, os *timers* a ele conectados recebem o dobro da frequência desse barramento. No exemplo dado, os *timers* do APB1 recebem $2 \times 50 = 100$ MHz, e não 50 MHz.
- **O SysTick pode receber HCLK / 8.** O RCC fornece ao SysTick um *clock* externo equivalente ao HCLK dividido por 8, podendo o SysTick empregar esse *clock* ou o próprio HCLK, conforme sua configuração.
- **Existe um vigia para o HSE.** O *Clock Security System* (CSS), quando habilitado, detecta uma eventual falha no HSE, retorna o sistema ao HSI e gera uma NMI, de modo que o programa possa reagir adequadamente.

O RCC também é capaz de conduzir um *clock* interno a um pino (saídas **MC01** e **MC02**), o que possibilita medir, com um osciloscópio, a frequência efetivamente empregada pelo chip.

Figura: árvore de *clock* simplificada do STM32F411. À esquerda, as fontes HSI (16 MHz) e HSE (4–26 MHz) entrando no seletor do PLL, seguido dos blocos /M, $\times N$, /P e /Q; no centro, o seletor SW escolhendo entre HSI, HSE e PLL para formar o SYSCLK; à direita, o *prescaler* AHB gerando o HCLK (100 MHz no máximo) e, a partir dele, os *prescalers* APB1 (PCLK1, 50 MHz no máximo) e APB2 (PCLK2, 100 MHz no máximo); abaixo, LSI e LSE indo para o IWDG e o RTC.

5.12.4 A Flash não acompanha o processador

A Flash é consideravelmente mais lenta que o núcleo. Com alimentação entre 2,7 V e 3,6 V, ela só consegue entregar uma leitura por ciclo até 30 MHz. Além desse limiar, o processador precisa aguardar ciclos adicionais, os **estados de espera** (*wait states*), configurados no campo **LATENCY** do registrador **FLASH_ACR**:

HCLK (alimentação de 2,7 V a 3,6 V)	Estados de espera	Ciclos por leitura
até 30 MHz	0	1

HCLK (alimentação de 2,7 V a 3,6 V)	Estados de espera	Ciclos por leitura
de 30 a 64 MHz	1	2
de 64 a 90 MHz	2	3
de 90 a 100 MHz	3	4

Com alimentação mais baixa, a tabela exige um número maior de estados de espera para a mesma frequência (Tabela 5 do RM0383). Disso decorrem três consequências:

- **A ordem é determinante.** Para **e**levar a frequência, deve-se aumentar primeiro os estados de espera e só então trocar o *clock*. Para **re**duzi-la, procede-se de forma inversa: troca-se primeiro o *clock*, reduzindo-se os estados de espera somente depois. Na ordem incorreta, o processador lê a Flash mais rapidamente do que ela é capaz de responder, e passa a executar lixo.
- **É exatamente para isso que existe o ART Accelerator.** A fila de pré-busca e a *cache* de desvios ocultam a maior parte desses ciclos adicionais. Segundo o *datasheet*, o desempenho medido no *benchmark* CoreMark equivale ao de uma Flash sem estados de espera a 100 MHz.
- **O reset parte, sempre, do caso seguro.** Após o reset, o chip opera no HSI de 16 MHz com zero estados de espera — combinação que funciona sob qualquer tensão de alimentação permitida.

5.12.5 Executar código da SRAM

A SRAM não possui estados de espera: responde na velocidade do próprio processador, a qualquer frequência. Isso sugere uma pergunta natural: por que, então, não executar o programa diretamente da SRAM?

No mapa de memória padrão do Cortex-M, a região da SRAM é executável, de modo que o processador pode, sim, buscar instruções em `0x20000000`. Existem, contudo, três limitações relevantes:

i Isto não é o modo de boot SRAM

Não se deve confundir esta seção com o modo de boot `B00T1=1/B00T0=1`, examinado anteriormente. Naquele caso, o programa **inteiro** é executado a partir da SRAM, carregado por um depurador antes mesmo do reset, sem que a Flash entre em cena. Aqui, o cenário é justamente o oposto: o programa continua realizando o boot **normalmente pela Flash**, e é o próprio `reset_handler` — já em execução, buscando instruções da Flash — quem decide copiar deliberadamente **algumas funções específicas** para a SRAM, da mesma forma como copia a `.data`, desviando a execução para lá somente depois.

- **O barramento.** No endereço `0x20000000`, as instruções são buscadas pelo barramento System, o mesmo que atende aos dados e aos periféricos. Instrução e dado passam, então, a disputar o mesmo caminho, e o RM0383 descreve essa busca como menos eficiente do que a realizada pelo I-Code. Para obter melhor desempenho, o manual recomenda **remapear a SRAM para `0x00000000`** (por meio dos pinos de boot ou do `SYSCFG_MEMRMP`), de modo que ela passe a ser acessada pelos barramentos I-Code e D-Code.
- **O tamanho.** Trata-se de 128 KB de SRAM contra 512 KB de Flash, sendo que a SRAM ainda precisa acomodar as variáveis e a pilha.
- **A volatilidade.** O código não pode residir permanentemente na SRAM, pois ela se apaga na ausência de energia. É necessário que esteja gravado na Flash e seja **copiado** para a SRAM a cada partida.

Por essa razão, o programa, em geral, é executado a partir da Flash, e apenas algumas funções selecionadas são destinadas à SRAM. Os casos típicos são:

- **Funções que gravam a própria Flash.** Como já discutido, a Flash não pode ser lida enquanto está sendo gravada. Uma rotina responsável por gravar configurações na Flash precisa, portanto, ser executada de outro local, sob pena de congelar a si mesma.
- **Rotinas criticamente dependentes do tempo,** cuja duração precisa ser breve e previsível, sem depender do acerto ou erro da *cache* do ART Accelerator.
- **Economia de energia.** O STM32F411 permite desligar a Flash durante a execução, e o *datasheet* indica que, para tanto, o código precisa estar em execução a partir da SRAM.

A terceira limitação carrega uma consequência interessante: uma função executada a partir da SRAM enfrenta exatamente o mesmo problema de uma variável global inicializada. Seu conteúdo precisa estar

gravado na Flash, mas precisa também estar presente na SRAM no momento de ser utilizado. Retomaremos esse problema no capítulo seguinte.

5.13 Os registradores que encontraremos primeiro

Este capítulo percorreu diversos registradores, cada um apresentado no ponto em que fazia sentido explicá-lo: o RCC em meio aos *clocks*, o `FLASH_ACR` junto à velocidade da Flash, o `AIRCR` em meio às fontes de reset. Não é necessário memorizar nenhum deles — basta saber que existem e onde procurá-los (Seção 6.3 do RM0383, no caso dos registradores do RCC). Esta tabela reúne todos em um único lugar, como referência rápida antes da prática:

Registrador	Endereço	O que controla	Onde foi visto
<code>RCC_CR</code>	<code>0x4002 3800</code>	Liga e desliga HSI, HSE e PLL, e indica quando cada um está pronto (bits <code>...ON</code> e <code>...RDY</code>)	As fontes
<code>RCC_PLLCFGR</code>	<code>0x4002 3804</code>	Fatores <code>M</code> , <code>N</code> , <code>P</code> e <code>Q</code> do PLL e a fonte de entrada do PLL	O PLL
<code>RCC_CFGR</code>	<code>0x4002 3808</code>	Fonte do <code>SYSCLK</code> (<code>SW/SWS</code>), <i>prescalers</i> AHB e APB, saídas MCO	O clock do sistema e os prescalers
<code>RCC_AHB1ENR</code>	<code>0x4002 3830</code>	Liga o <i>clock</i> dos periféricos do AHB1, entre eles as portas GPIO	Os barramentos do STM32F411
<code>RCC_CSR</code>	<code>0x4002 3874</code>	Flags da causa do último reset; liga o LSI	Reset
<code>FLASH_ACR</code>	<code>0x4002 3C00</code>	Estados de espera da Flash e o ART Accelerator	A Flash não acompanha o processador
<code>AIRCR</code>	<code>0xE000ED0C</code>	Reset por software, pelo bit <code>SYSRESETREQ</code> (registrador do núcleo, não do RCC)	Reset
<code>SYSCFG_MEMRMP</code>	<code>0x4001 3800</code>	Campo <code>MEM_MODE</code> : troca, por software, qual memória fica espelhada em <code>0x00000000</code>	De onde o processador parte: boot e espelhamento

Na prática, será feita a leitura de diversos desses registradores diretamente no chip, por meio do depurador, com a finalidade de confirmar que o estado posterior ao reset corresponde exatamente ao aqui descrito.

5.14 Do mapa de memória à organização do programa

O *script* padrão do *linker* — aquele fornecido por padrão com o `arm-none-eabi-gcc` — desconhece por completo o STM32F411: não sabe onde se encontram a Flash nem a SRAM do chip, e posiciona o programa em endereços genéricos, concebidos para um alvo qualquer, sem qualquer relação com o mapa de memória real. Caso esses endereços recaíssem sobre o espelho da memória de partida — que, com o boot pela Flash, corresponde à própria Flash —, nada ali informaria ao processador onde se encontra a pilha e onde iniciar, uma vez que essa informação é lida de `0x00000000` e `0x00000004`. Uma variável como a `g_delay_count` do programa de teste do ambiente (o *blink* da Blackpill), nesse cenário, estaria na Flash, onde um `STR` não grava absolutamente nada. Para que um programa possa ser executado no STM32F411, cada seção precisa, portanto, ser posicionada deliberadamente na memória correta — e é exatamente essa decisão que tomamos a seguir, antes de qualquer sintaxe de *linker script* (essa última reservada ao laboratório).

5.14.1 Como um programa ocupa a memória

Do ponto de vista do compilador, um programa já compilado divide-se em **seções**: o código, as constantes, os dados inicializados e os não inicializados, cada qual identificado por uma letra própria na tabela de símbolos (`T` para código, `D` para dados inicializados, `B` para dados não inicializados) e por uma coluna de tamanho própria (`text`, `data`, `bss`). Falta, contudo, a outra metade da história: onde essas seções acabam por se situar quando o programa efetivamente é executado.

No **PC**, a resposta clássica corresponde à figura de um processo na memória. Em uma versão simplificada, o sistema operacional carrega o executável na RAM e o organiza da seguinte forma, dos endereços mais baixos para os mais altos:

- a `.text` e a `.rodata`, que o sistema marca como somente leitura;
- a `.data`, copiada do arquivo;
- a `.bss`, que não ocupa espaço no arquivo e é criada já zerada;
- o **heap**, a área do `malloc`, que cresce para cima;
- a **pilha**, que começa no alto e cresce para baixo.

Tudo reside na mesma RAM, cabendo ao sistema operacional copiar, zerar e proteger cada parte.

No **microcontrolador**, esse quadro se altera, porquanto **existem duas memórias físicas de fato**, com propriedades opostas entre si, e nenhum sistema operacional disponível para decidir absolutamente nada em nosso lugar. O restante desta seção resolve, isoladamente, aquilo que no PC constituía tarefa do *kernel*.

i Seção, segmento e “segmentação”

Três termos semelhantes surgem quando se pesquisa esse assunto:

- **Seção** corresponde à divisão que o compilador e o *linker* percebem (`.text`, `.data`, `.bss`), exibida pelo `objdump -h`.
- **Segmento**, no formato ELF, corresponde à divisão que interessa a quem **carrega** o programa: um bloco contíguo, dotado das mesmas permissões, capaz de reunir diversas seções. É exibido pelo `readelf -l`.
- **Segmentação de memória** trata de algo bastante distinto: um mecanismo de endereçamento próprio dos processadores x86, dotado de registradores de segmento, inexistente no Cortex-M.

Neste livro, tratamos exclusivamente de seções.

Figura (adaptada da Figura 14 do DS10314, redesenhada): três colunas. **À esquerda**, o mapa de 4 GB do STM32F411, simplificado, com a Flash (`0x0800 0000–0x0807 FFFF`) e a SRAM (`0x2000 0000–0x2001 FFFF`) destacadas e as demais regiões em cinza. **No centro**, a ampliação dessas duas memórias com o programa dentro: na Flash, de baixo para cima, a tabela de vetores, a `.text`, a `.rodata` e a cópia dos valores iniciais da `.data`; na SRAM, a `.data`, a `.bss`, o espaço livre (*heap/margem*) e a pilha descendo a partir de `0x2002 0000`; uma seta da cópia da `.data` na Flash para a `.data` na SRAM, com um ponto de interrogação (“quem copia?”). **À direita**, os rótulos de atributo de cada memória: Flash (leitura, execução) e SRAM (leitura, escrita, execução).

5.14.2 As duas memórias do STM32F411

Deixemos de lado, por um instante, qualquer sintaxe de *linker*: o que importa neste momento são exclusivamente as memórias em si.

Memória	Onde começa	Tamanho	Atributos	Volátil?
Flash	<code>0x08000000</code>	512 KB	leitura e execução	Não
SRAM	<code>0x20000000</code>	128 KB	leitura, escrita e execução	Sim

i Note

Esses atributos não constituem uma proteção de hardware — a Flash pode, sim, ser gravada, mediante o procedimento apresentado em “Gravar a Flash não é como escrever na SRAM”, e a SRAM é executável de qualquer forma. Eles descrevem, isto sim, **o uso pretendido** de cada memória, informação que, mais adiante, no laboratório, orientará o *linker*.

Estabelecidas essas duas memórias, resta a pergunta: para qual delas deve ir cada seção do programa?

5.14.3 Onde cada seção deveria morar

A decisão será tomada seção por seção.

.text (código) e .rodata (constantes). Nenhuma das duas se altera durante a execução, e ambas precisam sobreviver ao desligamento. Existe, portanto, um único lugar possível: **Flash**.

.bss (globais sem valor inicial). O programa altera essas variáveis constantemente, e não há qualquer valor inicial que precise ser preservado — elas sempre se iniciam zeradas. Existe, também aqui, um único lugar possível: **SRAM**.

.data (globais com valor inicial). Aqui, a resposta simples deixa de funcionar. Recorde-se do programa de teste do ambiente: o laço de atraso entre uma piscada e outra contabiliza as iterações em uma variável, `static volatile uint32_t g_delay_count = LED_DELAY;`, decrementada a cada volta. O valor `LED_DELAY` (100 mil) do inicializador precisa estar gravado em uma memória não volátil, por se tratar de um valor que deve sobreviver ao desligamento — o que exige Flash. Todavia, `g_delay_count` também precisa poder ser livremente alterada durante a execução, uma vez que decresce a cada iteração do laço — o que exige SRAM. A **.data**, portanto, necessita das **duas memórias simultaneamente**, cada uma cumprindo um papel distinto.

Disso decorrem dois nomes distintos para os dois endereços de uma mesma seção:

- o **LMA (Load Memory Address)**: local em que a seção fica efetivamente **gravada**. No caso da **.data**, trata-se de um endereço na Flash;
- o **VMA (Virtual Memory Address)**: local em que a seção existe **durante a execução** — isto é, o endereço utilizado pelo código para acessá-la. No caso da **.data**, trata-se de um endereço na SRAM.

Para **.text** e **.rodata**, LMA e VMA correspondem a um único e mesmo endereço, uma vez que são executadas a partir de onde foram gravadas. Apenas a **.data** apresenta valores distintos para ambos — e é exatamente essa diferença que torna necessária a existência de uma **cópia**: alguém precisa transportar o conteúdo do LMA até o VMA antes que o programa utilize a variável pela primeira vez. **Quem realiza essa cópia, e em que momento**, é o assunto do capítulo seguinte.

Seção	LMA	VMA	Por quê
.text / .rodata	Flash	Flash (igual ao LMA)	Roda de onde foi gravada
.bss	—	SRAM	Não tem valor a preservar; nasce zerada
.data	Flash	SRAM	O valor inicial sobrevive na Flash; a variável é alterada na SRAM

5.14.4 Segmentação da SRAM: pilha, *heap* e a falta de proteção

Decidido que **.data** e **.bss** residem na SRAM, resta ainda encaixar o restante daquilo que também depende dela: o *heap* (caso haja `malloc`) e a pilha.

A SRAM do STM32F411 estende-se de `0x20000000` a `0x2001FFFF` (128 KB). O arranjo mais comum, dos endereços mais baixos para os mais altos, é o seguinte:

1. **.data**, logo no início;
2. **.bss**, na sequência;
3. espaço livre (a margem para o *heap*, se houver);
4. a **pilha**, que começa no topo, em `0x20020000` — um endereço **fora** da SRAM, porque a pilha é *full-descending*: o primeiro `PUSH` decrementa o ponteiro antes de gravar, então o primeiro dado cai em `0x2001FFFC`, a última palavra válida.

! Estouro de pilha, sem rede de segurança

O Cortex-M4 não possui MMU, e a MPU vem desabilitada por padrão: **não existe, portanto, proteção de memória alguma**. A pilha cresce em direção à **.bss** e à **.data**, e nada a impede de ultrapassá-las. No PC, um estouro de pilha provoca um *segmentation fault*, interrompendo o programa de imediato. No microcontrolador, por sua vez, ele **sobrescreve variáveis globais silenciosamente**: o sintoma só se manifesta depois, sob a forma de uma variável que muda de valor sem que nenhuma linha de código explique o motivo — um dos problemas mais difíceis de rastrear em sistemas embarcados, já que a causa raiz costuma estar bastante distante de onde o sintoma se manifesta.

Enquanto não empregarmos `malloc` neste curso, o espaço entre a **.bss** e a pilha corresponde apenas a essa margem de segurança — e, quanto maior for, mais difícil (embora não impossível) se torna que a pilha alcance as variáveis globais.

O que se obtém, neste ponto, é uma decisão clara — cada seção, em qual memória, e por qual motivo — sem que se tenha escrito uma única linha de *linker script*. Essa decisão é exatamente aquilo que o *linker script*, no laboratório correspondente, expressará em sintaxe.

5.14.5 A distribuição em um executável real

Para tornar essa arquitetura concreta antes mesmo da próxima prática, inspecione o executável de demonstração que você gravou e depurou durante a validação inicial do ambiente de trabalho:

```
foo@bar$ arm-none-eabi-nm -n blinky.elf | grep -E " (delay|g_delay_count)$"
080000cc T delay
20000000 D g_delay_count
foo@bar$ arm-none-eabi-objdump -h blinky.elf | grep -E "\.(text|data|bss) "
```

0	.text	00000108	08000000	08000000	00010000	2**2
1	.data	00000004	20000000	08000108	00020000	2**2
2	.bss	00000000	20000004	0800010c	00020004	2**0

Os resultados comprovam, na prática, a separação exigida pelo hardware: * O código (a função `delay`, listada na `.text`) encontra-se posicionado na região da **Flash** (`0x08000000`). * O dado inicializado (a variável global `g_delay_count`, listada na `.data`) situa-se no primeiro endereço da **SRAM** (`0x20000000`). * No cabeçalho de seções (`objdump -h`), a `.data` expõe os seus dois endereços distintos: o de execução, na SRAM (`20000000`), e o de carga (LMA), posicionado logo após o término do código, na Flash (`08000108`).

O desafio a ser resolvido a partir de agora reside unicamente em redigir o roteiro para o *linker* e o código de inicialização que reproduzam esse exato cenário.

5.15 Questionário de revisão

1. O STM32F411 pertence à série F1F4L4, do grupo Ultra-low-powerHigh-PerformanceWireless da ST.
2. No código `STM32F411CEU6`, a letra `E` indica o encapsulamento tamanho da Flash (512 KB) a faixa de temperatura.
3. Para descobrir o que faz cada bit do registrador `RCC_CFGR`, o documento a consultar é o datasheet reference manual programming manual.
4. Com `B00T0 = 0`, o processador parte da Flash principal memória de sistema SRAM, e a primeira palavra da Flash pode ser lida tanto em `0x08000000` quanto em `0x`.
5. Depois de um reset, o *clock* do sistema vem do oscilador, com frequência de MHz.
6. Depois de um reset, o *clock* dos periféricos (GPIO, USART, *timers*) está ligado/desligado, e o programa precisa ligá-lo no RCC.
7. Com HSE de 8 MHz, `M = 4`, `N = 100` e `P = 2`, o *clock* do sistema vale MHz.
8. Com o SYSCLK em 100 MHz e o divisor do AHB em 1, o menor divisor permitido para o APB1 é.
9. Para aumentar a frequência do *clock*, o programa deve aumentar primeiro os estados de espera da Flash trocar primeiro a fonte do clock.
10. O flag `IWDGRSTF` no `RCC_CSR` indica que o último reset foi causado pelo botão de reset pelo watchdog independentemente de queda de tensão.
11. Uma Flash apagada é lida como `0x`, e a menor porção que pode ser apagada é o.
12. Um `STR` comum em um endereço da Flash grava o valor, só que mais devagar não grava nada: a gravação exige o procedimento da interface da Flash.
13. Voltar a proteção de leitura (RDP) do nível 1 para o nível 0 é impossível apaga toda a Flash não tem efeito colateral; sair do nível 2 é impossível apaga toda a Flash.
14. Depois do reset, o regulador está na escala 2, que limita o HCLK a MHz.
15. No modo SleepStopStandby, o conteúdo da SRAM se perde e o chip acorda por um reset.
16. Os atributos (`rx`) e (`rw`) do bloco `MEMORY` impedem, no hardware, que o programa escreva na Flash só dizem ao linker que tipo de seção pode ir para cada memória.
17. A seção `.data` precisa estar só na Flash só na SRAM na Flash (o valor inicial) e na SRAM (a variável).

5.16 Referências

- [1] [DS10314 – STM32F411xC/E Datasheet](#) – STMicroelectronics. Seções 2, 3.2, 5, 6.3 (Tabela 47) e 8.
- [2] [RM0383 – STM32F411xC/E Advanced ARM-based 32-bit MCUs Reference Manual](#) – STMicroelectronics. Seções 2, 3, 5, 6 e 7.2.1.
- [3] [PM0214 – STM32 Cortex-M4 MCUs and MPUs Programming Manual](#) – STMicroelectronics.
- [4] Yiu, J. *The Definitive Guide to ARM® Cortex®-M3 and Cortex®-M4 Processors*, 3ª edição (Newnes, 2013). Capítulo 6, Seções 6.1 a 6.3, pp. 193–202.

6. Prática: Código de Inicialização e *Linker Script*

Note

Pré-requisito: o `arm-none-eabi-gcc` instalado, conforme o roteiro de [Ambiente e Ferramentas](#), a [prática de Do C ao Binário](#) e o capítulo [Os Microcontroladores STM32 e a Série F4](#), em especial o mapa de memória, o boot e a seção “Do mapa de memória à organização do programa”. **Esta prática não usa a placa.** O programa construído aqui é gravado e conferido na placa na [prática seguinte](#).

Warning

Antes desta aula: a atividade de casa desta semana é entregue em um repositório **Git privado no GitHub**, configurado no roteiro de [Ambiente e Ferramentas de Desenvolvimento](#). Se você ainda não tem uma conta no GitHub, crie uma antes da aula, em github.com.

Na [prática de Do C ao Binário](#), o `embedded.c` foi compilado para o Cortex-M4, mas ligado pelo *script* padrão do *linker*, que o colocou perto de `0x8000`, longe da memória do STM32F411. A aula teórica de [Os Microcontroladores STM32 e a Série F4](#) terminou com a decisão de onde cada seção deveria morar (Flash ou SRAM) e uma pergunta: se `g_a = 10` precisa estar gravado na Flash e ser usado na SRAM, **quem copia o valor, e quando?** No PC, eram o carregador do sistema operacional e o *runtime* da biblioteca C. Na placa, não há nenhum dos dois.

Nesta prática vamos escrever as duas peças que faltam: o **código de inicialização** (`startup.c`), que faz o papel do carregador e do *runtime*, e o **linker script** (`stm32f411-rom.ld`), que descreve a memória do chip. Ao final, o `embedded.elf` estará ligado nos endereços certos, e vamos conferir, só com as ferramentas vistas na [prática de Do C ao Binário](#), que cada byte está onde o processador vai procurá-lo.

Livro-Texto Oficial: Leitura de Aprofundamento no Joseph Yiu

- **Capítulo 4 (Architecture):** Seção 4.8 (*Reset and reset sequence*), pp. 113–116 — o que o hardware faz ao sair do reset.
- **Capítulo 17 (Getting Started with the GNU Compiler Collection (gcc)):** Seções 17.2 e 17.3 (*Typical development flow* e *Simple blinky project*), pp. 562–566 — um projeto completo com código de inicialização e *linker script* para o GCC.

! Antes de começar: o que o processador vai ler primeiro?

Ao ligar a placa com o boot pela Flash, o Cortex-M4 lê duas palavras de 32 bits: a do endereço `0x00000000` e a do endereço `0x00000004`. Com o que vimos na aula teórica, **anote agora:** que valores deveriam estar nessas duas posições para o nosso programa rodar? Em que endereço da Flash elas estão gravadas de fato? Vamos conferir no fim da Parte 2.

6.1 Preparando a pasta

O ponto de partida está no `lab-03` do repositório `ufu-sembl-src`: só o `embedded.c`, o mesmo da [prática de Do C ao Binário](#). Atualize o repositório e copie a pasta da placa para o seu espaço de trabalho (se você usa outra placa, consulte o `README.md` do `lab-03`):

```
foo@bar$ cd ~/seml-workspace/ufu-sembl-src
foo@bar$ git pull
foo@bar$ cp -r lab-03/stm32f411-blackpill ~/seml-workspace/lab-03
foo@bar$ cd ~/seml-workspace/lab-03
foo@bar$ ls
embedded.c
```

Como nas semanas anteriores, a sua pasta de trabalho tem o mesmo nome do laboratório no repositório oficial: `~/seml-workspace/lab-03`. Todos os arquivos desta prática serão criados nela.

6.2 O que precisa acontecer antes do `main()`

Na [prática de Do C ao Binário](#), vimos no PC que alguém prepara o terreno antes do `main()`. No Linux, o *kernel* lê o executável, cria a memória do processo, carrega a `.data` com os valores iniciais e reserva a `.bss` já zerada. Depois, salta para o `_start`, que vem do *runtime* da biblioteca C e chama o `main()`. O *runtime* também prepara a entrada e a saída padrão, os argumentos da linha de comando e o *heap*.

Em um microcontrolador *bare-metal*, nada disso existe. O que existe é o **reset**, e os capítulos anteriores mostraram, cada um, um pedaço dele: a sequência de reset do Cortex-M (vista em [Introdução aos Processadores ARM Cortex-M](#)), os pinos de boot e o espelhamento do STM32F411 (vistos em [Os Microcontroladores STM32 e a Série F4](#)) e, agora, o código que roda depois. Juntando os pedaços, esta é a linha do tempo completa, do botão de reset ao `main()`:

#	Quem faz	O que acontece
1	Chip	Um evento de reset acontece: a energia é ligada (POR), o botão puxa o pino <code>NRST</code> para nível baixo, um <i>watchdog</i> estoura ou o software pede um reset.
2	Chip	O circuito de reset mantém tudo parado por um pulso de pelo menos 20 μ s. Os registradores voltam aos valores de reset, e o <i>clock</i> passa a vir do HSI, de 16 MHz.
3	Chip	Na 4ª borda de subida do <i>clock</i> depois do reset, o STM32F411 lê os pinos <code>BOOT0</code> e <code>BOOT1</code> e decide qual memória aparece espelhada em <code>0x00000000</code> . Com <code>BOOT0 = 0</code> , é a Flash.
4	Núcleo	O Cortex-M4 lê a palavra em <code>0x00000000</code> e a coloca no MSP . Como a Flash está espelhada ali, essa é a primeira palavra gravada em <code>0x08000000</code> .
5	Núcleo	O Cortex-M4 lê a palavra em <code>0x00000004</code> , o vetor de reset , e a coloca no PC . O bit 0 dessa palavra precisa ser 1, indicando código Thumb.
6	Núcleo	O processador começa a executar o tratador de reset, em <i>Thread Mode</i> , no nível privilegiado, já com uma pilha válida.
7	Nosso código	O tratador de reset copia a <code>.data</code> da Flash para a SRAM e zera a <code>.bss</code> .
8	Nosso código	O tratador de reset chama o <code>main()</code> .

Figura: a linha do tempo do reset, em traço de rascunho, da esquerda para a direita. Primeiro, o pino `NRST` (ou a energia ligando) e o pulso de reset; depois, os pinos `BOOT0/BOOT1` escolhendo a memória espelhada em `0x00000000`; depois, as duas setas das leituras do núcleo (`0x0` para o MSP, `0x4` para o PC), saindo da tabela de vetores no início da Flash; por fim, o `reset_handler` com as setas da cópia da `.data` (Flash para SRAM) e da zeragem da `.bss`, e a chamada do `main()`. Uma linha separa as etapas feitas pelo hardware (1 a 6) das feitas pelo nosso código (7 e 8).

Os passos 1 a 6 acontecem sozinhos, no hardware, e são iguais para qualquer programa. Mas eles dependem de duas palavras que **nós** precisamos gravar no lugar certo, e os passos 7 e 8 não acontecem se nós não os escrevermos. O MSP é carregado antes de qualquer instrução porque uma exceção, como uma NMI ou uma HardFault, pode acontecer logo depois do reset, e o processador precisa de uma pilha para guardar o seu estado.

Falta uma peça: a linha do tempo diz **quando** cada coisa acontece, mas não **onde**, na memória, cada parte do programa mora. Essa é a “outra metade”, vista na aula teórica — vale reaproveitar aqui, porque é exatamente o que o *linker script* da Parte 2 vai descrever:

Parte do programa	Onde precisa estar	Por quê
Tabela de vetores	Início da Flash, <code>0x08000000</code>	É o endereço espelhado em <code>0x00000000</code> , de onde o núcleo lê o SP e o vetor de reset (passos 4 e 5 da tabela acima)
<code>.text</code> (código) e <code>.rodata</code> (constantes)	Flash	Não mudam, e precisam sobreviver ao desligamento
<code>.bss</code> (globais sem valor inicial)	SRAM	O programa as altera, e não há valor a preservar
<code>.data</code> (globais com valor inicial)	Nas duas: Flash (LMA) e SRAM (VMA)	O valor inicial precisa sobreviver ao desligamento, mas a variável precisa poder ser alterada
Pilha	Topo da SRAM, <code>0x20020000</code> , crescendo para baixo	É alterada o tempo todo

Figura: a mesma figura da aula teórica (mapa do STM32F411, adaptado da Figura 14 do DS10314), reaproveitada aqui — na Flash, de baixo para cima, a tabela de vetores, a `.text`, a `.rodata` e a cópia dos valores iniciais da `.data`; na SRAM, a `.data`, a `.bss` e a pilha descendo a partir de `0x20020000`.

Essa tabela é descrita pelo *linker script* em duas linguagens: o bloco `MEMORY` (onde ficam a Flash e a SRAM) e o bloco `SECTIONS` (o que vai em cada uma) — os dois blocos que escrevemos na Parte 2.

A pilha já vem pronta. Todo o resto é responsabilidade nossa, e se divide em três peças:

1. uma **tabela de vetores** gravada no início da Flash, em `0x08000000`, que o STM32F411 espelha em `0x00000000`. As duas primeiras palavras são o topo da pilha e o endereço do tratador de reset;
2. um **tratador de reset** que faça o trabalho do carregador e do *runtime*: copiar a `.data` da Flash para a SRAM, zerar a `.bss` e, só então, chamar o `main()`;
3. uma descrição da memória que diga ao *linker* onde ficam a Flash e a SRAM, onde vai cada seção e onde começa e termina cada uma, para que o tratador de reset saiba o que copiar e o que zerar.

As peças 1 e 2 são o **código de inicialização**, que escreveremos em C, na Parte 1. A peça 3 é o *linker script*, na Parte 2. Da lista do PC ficam de fora o *heap*, a entrada e a saída padrão e os argumentos da linha de comando: não há `malloc`, terminal nem linha de comando.

6.3 Parte 1 — O código de inicialização

O código de inicialização é o que, no PC, ficava a cargo do carregador do sistema operacional e do *runtime* da biblioteca C: preparar o ambiente antes do `main()` rodar. Na placa não há nenhum dos dois, então escrevemos isso à mão, em C. O arquivo `startup.c` vai ter duas peças: a **tabela de vetores**, que o processador lê no boot para saber onde está o topo da pilha e o primeiro código a executar, e o **tratador de reset**, que copia a `.data` da Flash para a SRAM, zera a `.bss` e só então chama o `main()`.

Vamos construir isso em seis passos, compilando ao fim de cada um. Em cada passo, o texto diz **o que fazer no arquivo**: acrescentar no fim, ou substituir um trecho que já existe.

6.3.1 Passo a passo do `startup.c`

Passo 1 — Configuração inicial do MSP (Main Stack Pointer). Este passo ainda não produz nenhum código executável: seu propósito é preparar o valor que ocupará a primeira posição da tabela de vetores — o valor inicial do MSP (*Main Stack Pointer*), lido pelo processador no endereço `0x00000000` antes da execução de qualquer instrução.

Esse valor deve corresponder necessariamente ao **topo da SRAM**, em razão do modelo de pilha adotado pelo Cortex-M, o *full-descending*: a cada operação `PUSH`, o ponteiro de pilha é decrementado antes da gravação do dado. Caso o MSP apontasse para um endereço interno à SRAM, a primeira palavra empilhada ocuparia uma posição inferior ao topo, desperdiçando parte do espaço disponível; ao apontar exatamente para o topo, garante-se que o primeiro dado gravado ocupe, de fato, a última palavra válida da memória. Figura: a SRAM como uma coluna vertical, de `0x20000000` (embaixo) a `0x2001FFFF` (última palavra válida, no topo); acima dela, fora da faixa, o rótulo `STACK_START = 0x20020000`, com uma seta apontando para baixo; um primeiro `PUSH` decrementando o SP em 4 antes de gravar, chegando em `0x2001FFFC`, destacado como o primeiro endereço de fato usado pela pilha.

O cálculo do valor exato é direto: a SRAM do STM32F411 tem início em `0x20000000` e capacidade de 128 KB, de modo que seu topo corresponde a $0x20000000 + 128 \times 1024 = 0x20020000$ — um endereço que, tecnicamente, situa-se **fora** dos limites da SRAM, cujo último endereço válido é `0x2001FFFF`. Isso não constitui um erro: trata-se precisamente do endereço que, uma vez decrementado em 4 bytes pelo primeiro `PUSH`, resulta em `0x2001FFFC`, a última palavra de fato endereçável.

Uma forma adequada de realizar esse cálculo é por meio de diretivas `#define`, delegando ao pré-processador a resolução das operações aritméticas envolvidas. Para isso, abra a pasta `~/seml-workspace/lab-03` como *workspace* no VS Code — a mesma janela remota do WSL configurada previamente no roteiro de Ambiente e Ferramentas. Alternativamente, pelo terminal integrado, dentro da pasta correspondente, o comando `code .` executa exatamente essa mesma ação:

```
foo@bar$ cd ~/seml-workspace/lab-03
foo@bar$ code .
```

Com a pasta devidamente aberta no editor, crie um arquivo novo através do menu (**File** → **New Text File**) e, em seguida, salve-o com o nome `startup.c` dentro dessa mesma pasta. Digite, então, o conteúdo inicial do arquivo:

```
#include <stdint.h>

#define SRAM_START 0x20000000U          /* Início da SRAM. */
#define SRAM_SIZE  (128U * 1024U)      /* 128 KB no STM32F411. */
#define SRAM_END   (SRAM_START + SRAM_SIZE) /* Fim da SRAM. */

#define STACK_START SRAM_END           /* Topo inicial da pilha. */
```

Salve o arquivo (`Ctrl+S`) e proceda à compilação, utilizando o terminal integrado do VS Code ou qualquer outro terminal aberto na mesma pasta. O comando de compilação permanece o mesmo utilizado anteriormente, interrompendo a cadeia de compilação na etapa de geração do arquivo objeto:

```
foo@bar$ arm-none-eabi-gcc -c -g -mcpu=cortex-m4 -mthumb -O0 -Wall startup.c -o
startup.o
foo@bar$ file startup.o
startup.o: ELF 32-bit LSB relocatable, ARM, EABI5 version 1 (SYSV), with debug_info,
not stripped
```

Nesse comando, surgem três opções inéditas, que passarão a acompanhar praticamente todo comando de compilação utilizado ao longo do restante do curso:

- **-g** é a opção responsável por incorporar informações de depuração ao arquivo objeto — nomes de variáveis, tipos de dados e a correspondência exata entre cada instrução de máquina e a linha correspondente do código-fonte em `.c`. É precisamente essa informação que se manifesta como “*with debug_info*” na saída do `file`. Na ausência dessa opção, um depurador fica restrito à exibição de endereços numéricos; com ela habilitada, torna-se possível visualizar o código-fonte original, linha a linha, em vez de uma sequência arbitrária de valores.
- **-O0** desativa integralmente as otimizações do compilador, correspondendo ao nível mais baixo disponível. Sob essa configuração, o compilador produz o código mais direto e literal possível, evitando reorganizações, eliminações ou fusões de instruções. Por essa razão, essa opção é empregada sempre que o objetivo seja a leitura da desmontagem e o reconhecimento, instrução a instrução, do código C que a originou: sob otimização (`-O1`, `-O2`...), o compilador reescreve o código de maneiras que, embora preservem o resultado final, comprometem essa correspondência direta. Essa comparação entre níveis de otimização será retomada na atividade de casa.
- **-Wall** habilita o conjunto de avisos (*warnings*) mais relevantes do compilador, abrangendo situações como variáveis utilizadas sem inicialização prévia, comparações potencialmente equivocadas e funções que não retornam valor em todos os caminhos de execução, entre outras. Embora um aviso não impeça a compilação, ele quase invariavelmente indica um erro real de programação — sendo, portanto, recomendável corrigir todos os avisos antes de prosseguir.

Resultado esperado: a execução do `gcc` não deve produzir nenhuma saída no terminal — a ausência de mensagens constitui, neste contexto, o indicativo de sucesso da operação. Na eventualidade de surgir algum erro ou aviso, é imprescindível corrigi-lo antes de prosseguir. O comando `file`, por sua vez, confirma a criação do arquivo objeto e evidencia a presença de “*with debug_info*”, efeito direto da opção

-g discutida anteriormente. Cabe observar que, nesta etapa, o arquivo contém apenas diretivas do pré-processador; consequentemente, o comando `arm-none-eabi-nm startup.o` não retorna nenhum símbolo.

Passo 2 — A tabela de vetores mínima. Segundo a *Table 37* do RM0383, a tabela completa do STM32F411 tem uma posição para o valor inicial do SP, 15 para as exceções de sistema e 86 para as interrupções dos periféricos: 102 palavras de 4 bytes, 408 bytes ao todo. Por enquanto, vamos implementar só a parte definida pela arquitetura: o SP e as 15 exceções de sistema, todas ainda com zero.

Figura: as 16 palavras de `g_vectors[]`, uma abaixo da outra, cada uma anotada com o deslocamento em hexadecimal e o nome da exceção correspondente (RM0383/PM0214): `0x00` MSP inicial, `0x04` Reset, `0x08` NMI, `0x0C` HardFault, `0x10` MemManage, `0x14` BusFault, `0x18` UsageFault, `0x1C–0x28` reservadas, `0x2C` SVCcall, `0x30` Debug Monitor, `0x34` reservada, `0x38` PendSV, `0x3C` SysTick. Um recorte só das exceções do núcleo — as 86 IRQs dos periféricos ficam de fora deste desenho.

Como a tabela de vetores consiste apenas em uma sequência contínua de palavras de 32 bits, podemos declará-la diretamente como um array comum em C. A escolha do tipo aqui é crucial: usamos `uint32_t` (do cabeçalho `<stdint.h>`), que garante largura fixa de exatamente 32 bits em qualquer arquitetura e compilador. Embora na arquitetura Cortex-M o `unsigned int` tenha largura de 32 bits, não é correto utilizá-lo, pois o padrão C garante apenas uma largura mínima, não um tamanho exato. Como o formato da tabela é imposto diretamente pelo hardware, confiar em `unsigned int` funcionaria apenas por coincidência, tornando-o conceitualmente inadequado. Esse mesmo princípio de previsibilidade de tamanho se aplica a cada registrador de periférico manipulado adiante.

Dessa forma, podemos declarar a primeira parte da tabela de vetores ao final de `startup.c`, inicializando apenas o ponteiro de pilha e preenchendo as demais entradas com zeros por enquanto:

```
uint32_t g_vectors[] =
{
    STACK_START,                /* 0x0000 0000 */
    0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, /* 0x0000 0004 a 0x0000 0030 */
    0, 0, 0                      /* 0x0000 0034 a 0x0000 003c */
};
```

Compile o arquivo objeto e faça uma inspeção em duas etapas: utilize primeiro o `arm-none-eabi-nm` para identificar o tipo e a visibilidade atribuídos ao símbolo `g_vectors`, e em seguida o `arm-none-eabi-objdump` para verificar as dimensões e os atributos da seção correspondente:

```
foo@bar$ arm-none-eabi-gcc -c -g -mcpu=cortex-m4 -mthumb -O0 -Wall startup.c -o
startup.o
foo@bar$ arm-none-eabi-nm startup.o
00000000 D g_vectors
foo@bar$ arm-none-eabi-objdump -h startup.o

startup.o:      file format elf32-littlearm

Sections:
Idx Name          Size      VMA          LMA          File off  Algn
  0 .text          00000000  00000000  00000000  00000034  2**1
CONTENTS, ALLOC, LOAD, READONLY, CODE
  1 .data          00000040  00000000  00000000  00000034  2**2
CONTENTS, ALLOC, LOAD, DATA
  2 .bss           00000000  00000000  00000000  00000074  2**0
ALLOC
  3 .debug_info     0000009f  00000000  00000000  00000074  2**0
CONTENTS, RELOC, READONLY, DEBUGGING, OCTETS
  4 .debug_abbrev   0000005a  00000000  00000000  00000113  2**0
CONTENTS, READONLY, DEBUGGING, OCTETS
  5 .debug_aranges  00000018  00000000  00000000  0000016d  2**0
CONTENTS, RELOC, READONLY, DEBUGGING, OCTETS
  6 .debug_line     000000fc  00000000  00000000  00000185  2**0
CONTENTS, READONLY, DEBUGGING, OCTETS
  7 .debug_str      0000016e  00000000  00000000  00000281  2**0
CONTENTS, READONLY, DEBUGGING, OCTETS
  8 .comment        00000045  00000000  00000000  000003ef  2**0
```

```

CONTENTS, READONLY
9 .ARM.attributes 0000002e 00000000 00000000 00000434 2**0
CONTENTS, READONLY

```

A saída do comando `nm` mostra `g_vectors` em `00000000`, com a letra `D` — símbolo global, numa seção de dados inicializados e graváveis. O `objdump -h` detalha as seções do objeto: cada linha traz o índice, o nome, o tamanho (`Size`), os endereços VMA e LMA (ainda `00000000` nos dois, porque a ligação final ainda não aconteceu) e a posição no arquivo.

Três grupos interessam. `.text` (0 bytes) continua vazia — ainda não há nenhuma instrução, só dados —, e `.data` (0x40 bytes) recebeu o vetor inteiro: para o compilador, `g_vectors` é uma variável global comum, com um valor inicial diferente de zero, e é isso que decide `.data`. `.bss` (0 bytes) permanece vazia, porque ainda não existe nenhuma variável global sem valor inicial. Já `.debug_info`, `.debug_abbrev`, `.debug_aranges`, `.debug_line` e `.debug_str` são o “*with debug_info*” que o `file` mostrou no Passo 1 — as seções que o `-g` acrescentou, consultadas pelo depurador. `.comment` e `.ARM.attributes` são metadados do compilador. Nenhuma dessas seis seções vai para a Flash ou a SRAM do chip: o *linker* as descarta na ligação final.

Para não repetir essa lista inteira a cada compilação, é comum filtrar a saída com `grep`, pedindo só as seções que importam no momento:

```

foo@bar$ arm-none-eabi-objdump -h startup.o | grep -E "\.text|\.data"
0 .text          00000000 00000000 00000000 00000034 2**1
1 .data          00000040 00000000 00000000 00000034 2**2

```

O problema é que a `.data` precisa residir na **SRAM** para permitir escrita em tempo de execução, enquanto a tabela de vetores **deve estar na Flash**, pronta e acessível logo no momento do *reset*, antes mesmo de qualquer rotina de inicialização rodar. O compilador desconhece o papel especial dessa estrutura: sem instruções explícitas, ele trata `g_vectors` como um bloco de dados modificável qualquer.

Para retirar `g_vectors` da seção `.data` padrão e vinculá-la a uma seção exclusiva — a qual o *linker script* posicionará no início da Flash —, utilizamos o atributo `__attribute__((section(".isr_vectors")))`. Trata-se de uma extensão do GCC que não integra o padrão C; outros compiladores adotam diretivas proprietárias (como `#pragma`), razão pela qual o código de inicialização (*startup*) é historicamente uma das camadas menos portáveis de um projeto de firmware.

Substitua a declaração de `g_vectors` por esta versão com o atributo:

```

uint32_t g_vectors[] __attribute__((section(".isr_vectors"))) =
{
    STACK_START, /* 0x0000 0000 */
    0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, /* 0x0000 0004 a 0x0000 0030 */
    0, 0, 0 /* 0x0000 0034 a 0x0000 003C */
};

```

Compile novamente e confirme a criação e o dimensionamento da nova seção:

```

foo@bar$ arm-none-eabi-gcc -c -g -mcpu=cortex-m4 -mthumb -O0 -Wall startup.c -o startup.o
foo@bar$ arm-none-eabi-nm startup.o
00000000 D g_vectors
foo@bar$ arm-none-eabi-objdump -h startup.o | grep -E "\.data|isr"
1 .data          00000000 00000000 00000000 00000034 2**0
3 .isr_vectors   00000040 00000000 00000000 00000034 2**2

```

Ao comparar essa saída com a anterior, observe a transferência de espaço: a seção `.data` agora aparece com 0 bytes, enquanto os 0x40 bytes migraram para `.isr_vectors`, que passa a abrigar as nossas 16 palavras de 4 bytes. Um detalhe curioso é que o `arm-none-eabi-nm` continua exibindo a letra `D` ao lado de `g_vectors`. Isso acontece porque o `nm` classifica os símbolos por categorias gerais de acesso: como declaramos o vetor sem o qualificador `const`, o compilador assume que esses dados podem ser modificados em tempo de execução e marca `.isr_vectors` com permissão de leitura e escrita (`CONTENTS, ALLOC, LOAD, DATA`). Ao inspecionar o arquivo objeto, o `nm` identifica essas permissões e cataloga o símbolo na categoria padrão de dados graváveis (`D`), independentemente do nome da seção. É a saída do `objdump -h` que confirma que a tabela agora possui uma seção exclusiva; caso adicionássemos `const` à declaração, o compilador marcaria a área como somente leitura e o `nm` passaria a exibir a letra `R`.

Passo 3 — O tratador de reset. A segunda posição da tabela deve conter, obrigatoriamente, o endereço da função que o processador executará imediatamente após o reset.

É importante observar que o tratador de reset consiste em uma função C perfeitamente comum, e essa possibilidade decorre exclusivamente do fato de o Cortex-M já ter carregado o MSP antes da execução da primeira instrução: no momento em que o tratador é iniciado, já existe, portanto, uma pilha válida à sua disposição. Em diversas outras arquiteturas, incluindo os processadores ARM clássicos, o início do código de inicialização precisa necessariamente ser escrito em *assembly*, justamente com o propósito de configurar a pilha antes que qualquer código em C possa ser executado.

Há, contudo, um detalhe relevante quanto ao endereço que será inserido na tabela: uma vez que o tratador é invocado por meio de um desvio, o bit 0 de seu endereço deve obrigatoriamente valer 1, indicando tratar-se de código Thumb (conforme discutido na Questão 8 do [Estudo Dirigido: Arquitetura ARM Cortex-M](#)). Na prática, entretanto, não há necessidade de preocupação adicional com isso: ao compilar com a opção `-mthumb`, o próprio *linker* já produz endereços de função com esse bit devidamente ativado.

Substitua o vetor `g_vectors` em sua totalidade (desde `uint32_t g_vectors[]` até o `};` correspondente) pelo trecho a seguir, que declara o protótipo da função `reset_handler`, insere o seu endereço na posição adequada da tabela e define a função, por ora, com corpo vazio:

```
void reset_handler(void);

uint32_t g_vectors[] __attribute__((section(".isr_vectors"))) =
{
    STACK_START,                /* 0x0000 0000 */
    (uint32_t)reset_handler,     /* 0x0000 0004 */
    0, 0, 0, 0, 0, 0, 0, 0, 0, /* 0x0000 0008 a 0x0000 0030 */
    0, 0, 0                     /* 0x0000 0034 a 0x0000 003c */
};

void reset_handler(void)
{
}
```

Compile o arquivo e verifique que o símbolo `reset_handler` passou a constar na tabela, identificado pela letra T, correspondente a código executável:

```
foo@bar$ arm-none-eabi-gcc -c -g -mcpu=cortex-m4 -mthumb -O0 -Wall startup.c -o startup.o
foo@bar$ arm-none-eabi-nm startup.o
00000000 D g_vectors
00000000 T reset_handler
```

Resultado esperado: o símbolo `reset_handler` deve aparecer na tabela de símbolos com a letra T, referente à categoria *text* — isto é, código, e não dado. Observe que ambos os símbolos figuram no endereço `00000000`; isso ocorre porque cada um deles é mensurado a partir do início de **sua própria** seção (`.isr_vectors`, no caso de `g_vectors`, e `.text`, no caso de `reset_handler`), e essas seções somente serão posicionadas de forma definitiva, uma após a outra na memória, quando o *linker script*, na Parte 2, determinar os endereços finais.

Passo 4 — Um tratador padrão. As demais posições da tabela deveriam, em princípio, apontar cada uma para o tratador correspondente à sua respectiva exceção. Entretanto, escrever uma função individual para cada uma delas configuraria uma redundância considerável, sobretudo porque a grande maioria jamais será disparada em um programa tão simples quanto o nosso. A solução adotada consiste em um **tratador padrão**, denominado `default_handler`, cuja função é interromper deliberadamente a execução do programa, por meio de um laço infinito, evitando que o processador salte para um endereço arbitrário e imprevisível.

Substitua a linha `void reset_handler(void);`, bem como o vetor `g_vectors` em sua totalidade, pelo trecho a seguir, que declara adicionalmente o `default_handler` e o posiciona em todas as entradas correspondentes às exceções de sistema, mantendo zeradas apenas as posições reservadas pela própria arquitetura. A função `reset_handler`, ainda vazia, permanece inalterada em sua posição original:

```
void reset_handler(void);
void default_handler(void);
```

```
uint32_t g_vectors[] __attribute__((section(".isr_vectors"))) =
{
    STACK_START,                /* 0x0000 0000 */
    (uint32_t)reset_handler,    /* 0x0000 0004 */
    (uint32_t)default_handler,  /* 0x0000 0008: NMI. */
    (uint32_t)default_handler,  /* 0x0000 000c: HardFault. */
    (uint32_t)default_handler,  /* 0x0000 0010: MemManage. */
    (uint32_t)default_handler,  /* 0x0000 0014: BusFault. */
    (uint32_t)default_handler,  /* 0x0000 0018: UsageFault. */
    0,                          /* 0x0000 001c: reservado. */
    0,                          /* 0x0000 0020: reservado. */
    0,                          /* 0x0000 0024: reservado. */
    0,                          /* 0x0000 0028: reservado. */
    (uint32_t)default_handler,  /* 0x0000 002c: SVCcall. */
    (uint32_t)default_handler,  /* 0x0000 0030: Debug Monitor. */
    0,                          /* 0x0000 0034: reservado. */
    (uint32_t)default_handler,  /* 0x0000 0038: PendSV. */
    (uint32_t)default_handler,  /* 0x0000 003c: SysTick. */
};
```

Em seguida, **acrescente ao final** do arquivo, imediatamente após a função `reset_handler`, a definição do tratador padrão:

```
void default_handler(void)
{
    while (1)
    {
    }
}
```

Compile o arquivo e verifique o resultado:

```
foo@bar$ arm-none-eabi-nm startup.o
0000000c T default_handler
00000000 D g_vectors
00000000 T reset_handler
```

Passo 5 — Um nome para cada exceção. A tabela construída no passo anterior é funcional, mas apresenta um inconveniente relevante: no momento em que se tornar necessário implementar um tratador efetivo — para o SysTick, por exemplo —, seria preciso editar diretamente a tabela dentro de `startup.c`. Esse problema é resolvido mediante a combinação de dois atributos do GCC:

- o atributo **alias** estabelece um apelido: a função `systick_handler` passa, na prática, a corresponder exatamente à mesma função que `default_handler`, sem possuir corpo próprio;
- o atributo **weak** torna o símbolo correspondente “fraco”: caso outro arquivo do programa venha a definir uma função `systick_handler` propriamente dita, o *linker* utilizará essa definição em substituição ao apelido, sem que qualquer modificação em `startup.c` seja necessária.

Ao combinar ambos os atributos, cada exceção passa a possuir um nome próprio, que pode ser implementado em qualquer arquivo do projeto — técnica amplamente empregada em códigos de inicialização de nível profissional. **Substitua** as duas linhas de protótipo (`reset_handler` e `default_handler`), assim como o vetor `g_vectors` em sua totalidade, pelo trecho a seguir. As funções `reset_handler` e `default_handler`, definidas mais abaixo no arquivo, permanecem inalteradas em suas posições originais:

```
int main(void);

void reset_handler(void);
void nmi_handler(void) __attribute__((weak, alias("default_handler")));
void hardfault_handler(void)
__attribute__((weak, alias("default_handler")));
void memmanage_handler(void)
```

```

__attribute__((weak, alias("default_handler")));
void busfault_handler(void)
__attribute__((weak, alias("default_handler")));
void usagefault_handler(void)
__attribute__((weak, alias("default_handler")));
void svc_handler(void) __attribute__((weak, alias("default_handler")));
void debugmon_handler(void)
__attribute__((weak, alias("default_handler")));
void pendsv_handler(void) __attribute__((weak, alias("default_handler")));
void systick_handler(void)
__attribute__((weak, alias("default_handler")));

uint32_t g_vectors[] __attribute__((section(".isr_vectors"))) =
{
    STACK_START,                /* 0x0000 0000 */
    (uint32_t)reset_handler,    /* 0x0000 0004 */
    (uint32_t)nmi_handler,      /* 0x0000 0008 */
    (uint32_t)hardfault_handler, /* 0x0000 000c */
    (uint32_t)memmanage_handler, /* 0x0000 0010 */
    (uint32_t)busfault_handler, /* 0x0000 0014 */
    (uint32_t)usagefault_handler, /* 0x0000 0018 */
    0,                          /* 0x0000 001c: reservado. */
    0,                          /* 0x0000 0020: reservado. */
    0,                          /* 0x0000 0024: reservado. */
    0,                          /* 0x0000 0028: reservado. */
    (uint32_t)svc_handler,      /* 0x0000 002c */
    (uint32_t)debugmon_handler, /* 0x0000 0030 */
    0,                          /* 0x0000 0034: reservado. */
    (uint32_t)pendsv_handler,   /* 0x0000 0038 */
    (uint32_t)systick_handler,  /* 0x0000 003c */
};

```

O protótipo `int main(void)`; já antecipa o passo subsequente, no qual o tratador de reset efetuará a chamada ao `main()`. Compile o arquivo e verifique: os nove apelidos devem aparecer identificados pela letra *W* (*weak*), todos posicionados exatamente no mesmo endereço de `default_handler`:

```

foo@bar$ arm-none-eabi-nm startup.o
00000000c W busfault_handler
00000000c W debugmon_handler
00000000c T default_handler
000000000 D g_vectors
00000000c W hardfault_handler
00000000c W memmanage_handler
00000000c W nmi_handler
00000000c W pendsv_handler
000000000 T reset_handler
00000000c W svc_handler
00000000c W systick_handler
00000000c W usagefault_handler

```

Passo 6 — O trabalho do tratador de reset. Resta implementar a parte central de todo o processo: copiar a seção `.data` da Flash para a SRAM, zerar integralmente a seção `.bss` e, somente então, efetuar a chamada ao `main()`.

Neste ponto, tornam-se necessários dois nomes distintos para os dois endereços associados a uma mesma seção. O **LMA** (*Load Memory Address*) corresponde ao endereço em que a seção é efetivamente **gravada**, ou seja, à posição em que o gravador a deposita. O **VMA** (*Virtual Memory Address*), por sua vez, corresponde ao endereço em que a seção existirá durante a **execução** — isto é, o endereço que o código efetivamente utiliza para acessá-la. No caso da `.text`, ambos coincidem, uma vez que o código é executado diretamente a partir da Flash. Já no caso da `.data`, o LMA situa-se na Flash e o VMA na SRAM, e é exatamente essa discrepância que o tratador de reset tem por finalidade resolver.

Para realizar essas cópias, o tratador necessita conhecer, com precisão, onde cada seção tem início e onde ela termina. Esses endereços, contudo, somente se tornam conhecidos após a etapa de ligação, quando o *linker* já houver posicionado definitivamente todo o conteúdo do programa. A solução adotada consiste em fazer com que o *linker script*, na Parte 2, **defina símbolos** correspondentes a esses endereços, os quais *startup.c* declara como **extern**.

Um símbolo definido no *linker script*, como em `_sdata = .;`, cria uma entrada na tabela de símbolos (a mesma listada pelo `nm`) contendo um nome e um endereço, mas **sem reservar qualquer byte de memória**: o próprio endereço constitui a informação relevante. Para que o compilador aceite o nome `_sdata` no contexto da linguagem C, ele é declarado como se correspondesse a uma variável (`extern uint32_t _sdata;`), embora não exista, de fato, nenhuma variável naquele endereço. Por essa razão, utiliza-se sempre a expressão `&_sdata`, que retorna o endereço registrado na tabela, e jamais `_sdata` isoladamente, o que resultaria na leitura do conteúdo daquele endereço — precisamente o que o tratador de reset ainda não terá inicializado. O tipo `uint32_t` empregado na declaração não possui significado semântico relevante; trata-se, tão somente, de um recurso sintático para atribuir um tipo ao nome. (A distinção conceitual entre *variável* e *símbolo* é aprofundada na atividade de casa.)

Substitua a função `reset_handler`, atualmente vazia (as três linhas correspondentes a `void reset_handler(void), { e }`), pelo trecho a seguir, que contempla as declarações dos símbolos necessários e o corpo completo da função. A função `default_handler` permanece inalterada, ao final do arquivo:

```
extern uint32_t _sdata; /* Início da seção .data na SRAM. */
extern uint32_t _edata; /* Fim da seção .data na SRAM. */
extern uint32_t _la_data; /* Origem da seção .data na Flash. */

extern uint32_t _sbss; /* Início da seção .bss. */
extern uint32_t _ebss; /* Fim da seção .bss. */

void reset_handler(void)
{
    uint32_t i;
    uint32_t size;
    uint8_t *dst;
    uint8_t *src;

    /* Copia a seção .data da Flash (LMA) para a SRAM (VMA). */

    size = (uint32_t)&_edata - (uint32_t)&_sdata;
    dst = (uint8_t *)&_sdata;
    src = (uint8_t *)&_la_data;

    for (i = 0; i < size; i++)
    {
        *dst++ = *src++;
    }

    /* Preenche a seção .bss com zeros. */

    size = (uint32_t)&_ebss - (uint32_t)&_sbss;
    dst = (uint8_t *)&_sbss;

    for (i = 0; i < size; i++)
    {
        *dst++ = 0;
    }

    /* Só agora o ambiente do C está pronto. */

    main();
}
```

Compile o arquivo uma última vez e liste os símbolos resultantes:

```
foo@bar$ arm-none-eabi-gcc -c -g -mcpu=cortex-m4 -mthumb -O0 -Wall startup.c -o
startup.o
foo@bar$ arm-none-eabi-nm startup.o
                 U _ebss
                 U _edata
                 U _la_data
                 U _sbss
                 U _sdata
00000084 W busfault_handler
00000084 W debugmon_handler
00000084 T default_handler
00000000 D g_vectors
00000084 W hardfault_handler
                 U main
00000084 W memmanage_handler
00000084 W nmi_handler
00000084 W pendsv_handler
00000000 T reset_handler
00000084 W svc_handler
00000084 W systick_handler
00000084 W usagefault_handler
```

Cabem, aqui, três observações relevantes:

- os símbolos `main`, `_sdata`, `_edata`, `_la_data`, `_sbss` e `_ebss` aparecem identificados pela letra `U`, indicando que são utilizados neste arquivo, mas definidos em outro local: `main` será fornecido por `embedded.o`, ao passo que os demais cinco símbolos serão fornecidos pelo *linker script*.
- os apelidos permanecem, todos, associados ao endereço de `default_handler`, que passou de `0x0c` para `0x84` em razão do crescimento da função `reset_handler`.
- o símbolo `g_vectors` continua identificado pela letra `D`, referente a dado inicializado, uma vez que se trata, do ponto de vista do compilador, de uma variável comum. Nenhuma informação presente em `startup.c` determina que ela deva residir na Flash; essa decisão cabe exclusivamente ao *linker script*.

i Confira o seu `startup.c` completo

Se o seu arquivo não compila, ou se o `nm` mostra algo diferente, compare-o com a versão completa abaixo. A ordem é: `#include` e `#define`, protótipos, vetor `g_vectors`, símbolos `extern`, `reset_handler` e `default_handler`.

```
#include <stdint.h>

#define SRAM_START 0x20000000U          /* Início da SRAM. */
#define SRAM_SIZE  (128U * 1024U)      /* 128 KB no STM32F411. */
#define SRAM_END   (SRAM_START + SRAM_SIZE) /* Fim da SRAM. */

#define STACK_START SRAM_END           /* Topo inicial da pilha. */

int main(void);

void reset_handler(void);
void nmi_handler(void) __attribute__((weak, alias("default_handler")));
void hardfault_handler(void)
    __attribute__((weak, alias("default_handler")));
void memmanage_handler(void)
    __attribute__((weak, alias("default_handler")));
void busfault_handler(void)
    __attribute__((weak, alias("default_handler")));
void usagefault_handler(void)
    __attribute__((weak, alias("default_handler")));
void svc_handler(void) __attribute__((weak, alias("default_handler")));
void debugmon_handler(void)
    __attribute__((weak, alias("default_handler")));
void pendsv_handler(void) __attribute__((weak, alias("default_handler")));
void systick_handler(void)
    __attribute__((weak, alias("default_handler")));

uint32_t g_vectors[] __attribute__((section(".isr_vectors"))) =
{
    STACK_START,                /* 0x0000 0000 */
    (uint32_t)reset_handler,    /* 0x0000 0004 */
    (uint32_t)nmi_handler,      /* 0x0000 0008 */
    (uint32_t)hardfault_handler, /* 0x0000 000c */
    (uint32_t)memmanage_handler, /* 0x0000 0010 */
    (uint32_t)busfault_handler, /* 0x0000 0014 */
    (uint32_t)usagefault_handler, /* 0x0000 0018 */
    0,                          /* 0x0000 001c: reservado. */
    0,                          /* 0x0000 0020: reservado. */
    0,                          /* 0x0000 0024: reservado. */
    0,                          /* 0x0000 0028: reservado. */
    (uint32_t)svc_handler,      /* 0x0000 002c */
    (uint32_t)debugmon_handler, /* 0x0000 0030 */
    0,                          /* 0x0000 0034: reservado. */
    (uint32_t)pendsv_handler,   /* 0x0000 0038 */
    (uint32_t)systick_handler,  /* 0x0000 003c */
};

extern uint32_t _sdata; /* Início da seção .data na SRAM. */
extern uint32_t _edata; /* Fim da seção .data na SRAM. */
extern uint32_t _la_data; /* Origem da seção .data na Flash. */

extern uint32_t _sbss; /* Início da seção .bss. */
extern uint32_t _ebss; /* Fim da seção .bss. */

void reset_handler(void)
{
    uint32_t i;
    uint32_t size;
    uint8_t *dst;
    uint8_t *src;

    /* Copia a seção .data da Flash (LMA) para a SRAM (VMA). */
```

6.4 Parte 2 — O *linker script*

6.4.1 Antes de digitar: o que é um *linker script*

O *linker script* consiste em um arquivo de texto responsável por controlar precisamente a forma como o *linker* monta o executável: quais memórias existem, quais seções de saída serão criadas, o conteúdo de cada uma, a ordem em que aparecem e o endereço que ocupam. Na [prática de Do C ao Binário](#), o comando `arm-none-eabi-ld -verbose` exibiu o *script* padrão, concebido para um alvo completamente distinto. Neste momento, elaboramos o nosso próprio *script* e o entregamos ao *linker* por meio da opção `-T`.

O *script* compreende dois blocos principais. O bloco **MEMORY** descreve, em sintaxe apropriada, as duas memórias do chip previamente identificadas na aula teórica (Flash e SRAM, com seus respectivos endereços e capacidades). O bloco **SECTIONS**, por sua vez, monta as **seções de saída** do executável (aquelas exibidas pelo `objdump -h` do `.elf`) a partir das **seções de entrada** provenientes dos arquivos objeto (as mesmas exibidas pelo `objdump -h` de cada `.o`), determinando em qual memória cada uma delas será posicionada.

Para determinar a memória correspondente a cada seção de saída, basta responder a uma única pergunta: **o tratador de reset será responsável por copiar esta seção?**

Seção	O tratador copia?	VMA (onde roda)	LMA (onde é gravada)	No <i>script</i>
<code>.text</code> (tabela de vetores, código, constantes)	Não: roda de onde foi gravada	Flash	Flash (igual ao VMA)	<code>> FLASH</code>
<code>.data</code> (globais com valor inicial)	Sim: da Flash para a SRAM	SRAM	Flash	<code>> SRAM AT> FLASH</code>
<code>.bss</code> (globais sem valor inicial)	Não copia, só zera: não há o que gravar	SRAM	—	<code>> SRAM</code>

O *script* será elaborado em quatro passos sucessivos, um bloco por vez, culminando na ligação do programa ao final.

6.4.2 Passo a passo do `stm32f411-rom.ld`

Passo 1 — O ponto de entrada e as memórias do chip. Este passo estabelece dois elementos fundamentais que o *linker* necessita conhecer antes de montar o executável: o ponto de entrada, registrado no cabeçalho do arquivo ELF, e a descrição das duas memórias físicas do STM32F411 — a Flash e a SRAM —, com seus respectivos endereços e capacidades.

No mesmo *workspace* do VS Code, crie um arquivo novo e salve-o com o nome `stm32f411-rom.ld`, na mesma pasta utilizada anteriormente. Digite o seguinte conteúdo:

```
ENTRY(reset_handler)

MEMORY
{
    FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 512K
    SRAM (rwx) : ORIGIN = 0x20000000, LENGTH = 128K
}
```

O comando **ENTRY** define o ponto de entrada registrado no cabeçalho do ELF. Embora o Cortex-M não utilize diretamente essa informação — uma vez que a execução sempre se inicia pela tabela de vetores —, o depurador a utiliza como referência. Na ausência dessa diretiva, o *linker* reclama da falta do símbolo `_start`, exatamente como observado anteriormente na [prática de Do C ao Binário](#). O bloco **MEMORY**, por sua vez, expressa em sintaxe as duas memórias identificadas na aula teórica. Cabe lembrar que os atributos `rx` e `rwx` não constituem qualquer forma de proteção: eles apenas informam ao *linker* que tipo de seção pode ser posicionado em cada memória.

Passo 2 — O código e as constantes, na Flash. Acrescente ao final do arquivo o início do bloco **SECTIONS**, contendo a primeira seção de saída:

```
SECTIONS
```

```
{
  .text :
  {
    KEEP(*(.isr_vectors))
    *(.text*)
    *(.rodata*)
    . = ALIGN(4);
  } > FLASH
```

Cabe examinar o trecho linha a linha:

- **Curingas.** Cada linha situada no interior de uma seção de saída assume a forma `arquivos(seções)`: a expressão `*(.text*)` significa, especificamente, “de todos os arquivos, todas as seções cujo nome se inicia por `.text`”. O asterisco posicionado ao final do nome é de importância considerável, uma vez que o compilador frequentemente gera seções acompanhadas de sufixos, como `.text.add` (quando empregada a opção `-ffunction-sections`) ou `.rodata.str1.4` (destinada a literais de *string*). Na ausência desse asterisco, tais seções permaneceriam excluídas, e o *linker* as posicionaria em um local arbitrário.
- **Ordem.** A ordem em que as linhas são declaradas corresponde diretamente à ordem de posicionamento na memória. É por essa razão que a tabela de vetores figura em primeiro lugar: essa disposição garante que ela venha a ocupar precisamente o endereço `0x08000000`.
- **KEEP.** Essa diretiva impede que o *linker* descarte a tabela durante a eliminação de seções não referenciadas (procedimento realizado pela opção `--gc-sections`, comumente empregada em projetos de maior porte). Cabe observar que nenhuma função do programa efetivamente utiliza a tabela de vetores; é o próprio hardware quem a consulta.
- **O contador de posição.** O símbolo `.` representa o endereço corrente dentro da seção em processo de montagem. A expressão `. = ALIGN(4);` avança esse contador até o múltiplo de 4 imediatamente seguinte, garantindo que a seção termine em um endereço alinhado a uma palavra (32 bits). Esse alinhamento constitui uma exigência rigorosa do hardware: caso o código de inicialização empregasse instruções de carga e armazenamento de múltiplos registradores — como `LDM`, `STM`, `PUSH` ou `POP` — sobre endereços não múltiplos de 4, o processador Cortex-M4 não faria o acesso quebrado de forma transparente, mas geraria imediatamente uma exceção fatal de desalinhamento (*UsageFault*), travando o sistema logo no boot.
- **O destino.** A expressão `> FLASH`, posicionada após a chave de fechamento, determina que a `.text` seja alocada na Flash, com VMA e LMA coincidentes.

Passo 3 — Os dados inicializados, gravados na Flash e usados na SRAM. Acrescente ao final do arquivo o seguinte trecho:

```
.data :
{
  _sdata = .;
  *(.data*)
  . = ALIGN(4);
  _edata = .;
} > SRAM AT> FLASH

_la_data = LOADADDR(.data);
```

Surtem, neste ponto, os primeiros símbolos do *script*. Atribuir o contador de posição a um nome — como ocorre em `_sdata = .;` — cria uma entrada na tabela de símbolos correspondente àquele endereço específico. O símbolo `_sdata` demarca o início da seção `.data`, ao passo que `_edata` demarca o seu término. O destino `> SRAM AT> FLASH` corresponde exatamente à linha da `.data` na tabela apresentada anteriormente: o VMA situa-se na SRAM, e o LMA, na Flash, imediatamente após a `.text`.

Um detalhe de especial importância merece destaque: **o contador de posição acompanha o VMA, e não o LMA.** No interior da `.data`, o ponto corresponde a endereços da SRAM, razão pela qual `_sdata` e `_edata` representam o destino da cópia. O endereço na Flash, de onde a cópia efetivamente se origina, não é refletido pelo contador. É justamente por essa razão que se torna necessária a função `LOADADDR(.data)`, responsável por retornar o LMA da seção, valor esse armazenado no símbolo `_la_data` (*load address*). Os símbolos `_sdata`, `_edata` e `_la_data` correspondem a três dos cinco símbolos que `reset_handler` declarou

como `extern` no Passo 6 da Parte 1; os dois símbolos restantes, referentes à `.bss`, serão apresentados no passo seguinte.

Passo 4 — Os dados não inicializados, só na SRAM. Acrescente ao final do arquivo a última seção, juntamente com a chave que encerra o bloco `SECTIONS`:

```
.bss :  
{  
  _sbss = .;  
  *(.bss*)  
  *(COMMON)  
  . = ALIGN(4);  
  _ebss = .;  
} > SRAM  
}
```

A seção `.bss` destina-se exclusivamente à SRAM (`> SRAM`), sem que absolutamente nada seja gravado na Flash, uma vez que não há valor algum a ser preservado. Os símbolos `_sbss` e `_ebss` demarcam, respectivamente, o início e o término da área que `reset_handler` será responsável por zerar. A expressão `*(COMMON)` recolhe um tipo mais antigo de variável global não inicializada, ainda gerado por alguns compiladores.

Cabe observar, por fim, algo que este *linker script* **não** realiza: ele não reserva espaço destinado à pilha. É o próprio `startup.c` quem posiciona o topo da pilha ao final da SRAM, e a pilha cresce em direção decrescente, aproximando-se da `.bss`, sem qualquer forma de proteção, conforme discutido na aula teórica. *Linker scripts* empregados em projetos de caráter profissional costumam reservar uma região específica para a pilha, definindo o seu topo por meio de um símbolo (frequentemente denominado `_estack`), de modo que o *linker* seja capaz de acusar erro caso a memória disponível seja insuficiente.

i Confira o seu `stm32f411-rom.ld` completo

```
ENTRY(reset_handler)

MEMORY
{
    FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 512K
    SRAM  (rwx) : ORIGIN = 0x20000000, LENGTH = 128K
}

SECTIONS
{
    .text :
    {
        KEEP(*(.isr_vectors))
        *(.text*)
        *(.rodata*)
        . = ALIGN(4);
    } > FLASH

    .data :
    {
        _sdata = .;
        *(.data*)
        . = ALIGN(4);
        _edata = .;
    } > SRAM AT> FLASH

    _la_data = LOADADDR(.data);

    .bss :
    {
        _sbss = .;
        *(.bss*)
        *(COMMON)
        . = ALIGN(4);
        _ebss = .;
    } > SRAM
}
```

6.4.3 Ligando o programa

Passo 5 — Compile a aplicação e ligue os dois objetos. As opções empregadas na etapa de ligação são as seguintes:

- `-nostdlib` dispensa integralmente a biblioteca C e o *runtime* característicos do PC;
- `-T` fornece ao *linker* o *script* que acabamos de elaborar;
- `-Wl, -Map=embedded.map` solicita ao *linker* a geração de um relatório detalhado de todo o processo de ligação;
- `-Wl, --print-memory-usage` exibe a quantidade utilizada de cada memória.

Por fim, o `objcopy` extrai a imagem binária que será, de fato, gravada na Flash:

```
foo@bar$ arm-none-eabi-gcc -c -g -mcpu=cortex-m4 -mthumb -O0 -Wall embedded.c -o
embedded.o
foo@bar$ arm-none-eabi-gcc -nostdlib -T stm32f411-rom.ld -Wl, -Map=embedded.map -Wl, --
print-memory-usage startup.o embedded.o -o embedded.elf
Memory region      Used Size  Region Size  %age Used
      FLASH:         276 B      512 KB       0.05%
```

```

SRAM:          12 B          128 KB          0.01%
foo@bar$ arm-none-eabi-objcopy -O binary embedded.elf embedded.bin

```

O programa, em sua totalidade, ocupa 276 bytes de Flash e 12 bytes de SRAM, correspondentes às três variáveis de 4 bytes cada. Caso a ligação venha a falhar com a mensagem `undefined reference to '_sdata'` (ou referente a qualquer um dos outros quatro símbolos), verifique cuidadosamente a grafia do símbolo tanto no *linker script* quanto em `startup.c`: os nomes devem ser rigorosamente idênticos.

6.4.4 Conferindo a imagem

Passo 6 — Confira o que o *linker* fez. Antes de proceder à gravação de qualquer conteúdo na placa, é fundamental verificar a imagem gerada. Observe, primeiramente, os símbolos dispostos em ordem crescente de endereço:

```

foo@bar$ arm-none-eabi-nm -n embedded.elf
08000000 T g_vectors
08000040 T reset_handler
080000c4 W busfault_handler
080000c4 W debugmon_handler
080000c4 T default_handler
080000c4 W hardfault_handler
080000c4 W memmanage_handler
080000c4 W nmi_handler
080000c4 W pendsv_handler
080000c4 W svc_handler
080000c4 W systick_handler
080000c4 W usagefault_handler
080000cc T add
080000e6 T main
08000110 A _la_data
20000000 D _sdata
20000000 D g_a
20000004 D _edata
20000004 B _sbss
20000004 B g_b
20000008 B g_c
2000000c B _ebss

```

Compare essa saída com o que foi observado na [prática de Do C ao Binário](#), ocasião em que o *script* padrão posicionou `add` em `0x8000` e `g_a` em `0x9044`:

- o símbolo `g_vectors` encontra-se em `0x08000000`, precisamente o início da Flash, seguido do código correspondente. No arquivo `startup.o`, esse símbolo constava com a letra `D`; presentemente, exibe a letra `T`, uma vez que o *linker script* o posicionou no interior da seção de saída `.text`.
- os símbolos `g_a`, `g_b` e `g_c` situam-se em `0x20000000` e nos endereços imediatamente subsequentes, na SRAM.
- os símbolos definidos pelo *linker script* aparecem acompanhados de seus respectivos valores: `_sdata` e `_edata` delimitam `g_a`; `_sbss` e `_ebss` delimitam `g_b` e `g_c`. O símbolo `_la_data`, identificado pela letra `A` (absoluto), aponta para um endereço na Flash, situado logo após o código.

Em seguida, examine os endereços correspondentes a cada seção:

```

foo@bar$ arm-none-eabi-objdump -h embedded.elf

embedded.elf:      file format elf32-littlearm

Sections:
Idx Name          Size      VMA       LMA       File off  Algn
  0 .text         00000110  08000000  08000000  00001000  2**2
CONTENTS, ALLOC, LOAD, READONLY, CODE
  1 .data         00000004  20000000  08000110  00002000  2**2
CONTENTS, ALLOC, LOAD, DATA

```

```

2 .bss          00000008 20000004 08000114 00002004 2**2
                ALLOC

```

A saída prossegue com as seções de depuração (`.debug_*`), que, como já discutido, não são gravadas na Flash do microcontrolador. Compare, com atenção, as colunas **VMA** e **LMA**:

- na `.text`, ambos os valores coincidem, `0x08000000`: o código é executado a partir do próprio local em que foi gravado;
- na `.data`, o VMA corresponde a `0x20000000`, na SRAM, ao passo que o LMA corresponde a `0x08000110`, na Flash, imediatamente após a `.text`. Esse valor de LMA é exatamente o que se encontra armazenado em `_la_data`;
- a `.bss` apresenta apenas a marcação `ALLOC`, sem `CONTENTS` nem `LOAD`: essa seção não possui, portanto, conteúdo algum na imagem gerada. O valor de LMA exibido para ela carece de utilidade prática, uma vez que não há nada a ser gravado.

Por fim, examine as primeiras palavras da imagem — exatamente aquilo que o processador irá ler no momento em que sair do estado de reset:

```

foo@bar$ od -A x -t x4 -N 64 embedded.bin
000000 20020000 08000041 080000c5 080000c5
000010 080000c5 080000c5 080000c5 00000000
000020 00000000 00000000 00000000 080000c5
000030 080000c5 00000000 080000c5 080000c5
000040

```

Confira, neste momento, a sua resposta à pergunta formulada no início da prática:

- a primeira palavra, `20020000`, corresponde ao topo da pilha;
- a segunda, `08000041`, corresponde ao endereço de `reset_handler` (`0x08000040`, segundo indicado pelo `nm`), com o bit 0 ativado, indicando tratar-se de código Thumb;
- as demais palavras correspondem ao endereço de `default_handler` (`0x080000c4`), igualmente com o bit 0 ativado, além de zeros nas posições reservadas pela arquitetura.

Essas palavras encontram-se no início da Flash, no endereço `0x08000000`. O processador as lê, respectivamente, em `0x00000000` e `0x00000004`, porquanto o STM32F411, ao realizar o boot pela Flash, espelha o conteúdo desta última nesses endereços.

A opção `-Wl, -Map=embedded.map`, empregada no Passo 5, gerou esse mesmo relatório em formato textual, no arquivo `embedded.map`. A leitura detalhada desse arquivo fica reservada para a atividade de casa.

💡 Onde está o 10?

O valor inicial de `g_a` está gravado na imagem, no LMA da `.data`. A posição no arquivo `.bin` é o LMA menos `0x08000000`, ou seja, `0x110`:

```

foo@bar$ od -A x -t x4 -j 0x110 -N 4 embedded.bin
000110 0000000a

```

É esse `0x0000000a` que o `reset_handler` vai copiar para `0x20000000`. Repare também que o `.bin` tem exatamente 276 bytes (confira com `ls -l`): termina logo depois do valor de `g_a`. A `.bss` não ocupa nenhum byte da imagem.

6.5 Juntando as peças

Com o `startup.c` e o *linker script*, a história completa de um programa *bare-metal*, do reset ao `main()`, fica assim:

1. O *linker* coloca a tabela de vetores no início da Flash (`0x08000000`), seguida do código, das constantes e da cópia dos valores iniciais da `.data`. O `objcopy` extrai essa imagem, e o gravador a transfere para a Flash.
2. Ao ligar a placa, o STM32F411 espelha a Flash em `0x00000000`. O Cortex-M lê o topo da pilha e o endereço de `reset_handler` nas duas primeiras palavras.

3. `reset_handler` copia a `.data` da Flash para a SRAM, zera a `.bss` e chama `main()`.
4. `main()` executa com as variáveis globais valendo exatamente o que o código C promete: `g_a = 10` e `g_b = 0`.

Até aqui, tudo foi conferido **antes** de o programa rodar, olhando a imagem. Na [prática seguinte](#), vamos gravá-la na placa e confirmar, no depurador, que os passos 2 a 4 acontecem de fato. Guarde a pasta `~/seml1-workspace/lab-03`: você vai copiá-la para `~/seml1-workspace/lab-04` e continuar a partir dela.

! Atividade para casa

A investigação continua em casa, na [Atividade: Explorando o Código de Inicialização e o Linker Script](#), que não precisa da placa. A entrega é feita no seu repositório Git privado, configurado no roteiro de [Ambiente e Ferramentas de Desenvolvimento](#).

6.6 Referências

- [1] Yiu, J. *The Definitive Guide to ARM Cortex-M3 and Cortex-M4 Processors*, 3ª edição. Newnes, 2013. Seção 4.8 (*Reset and reset sequence*) e Capítulo 17 (*Getting Started with the GNU Compiler Collection (gcc)*).
- [2] [RM0383 – STM32F411xC/E Reference Manual](#) – STMicroelectronics. Mapa de memória e tabela de vetores (*Table 37*).
- [3] [Using the GNU Compiler Collection \(GCC\)](#) – seções *Common Function Attributes* e *Common Variable Attributes* (*section, alias, weak*).
- [4] [The GNU Linker \(ld\) – Linker Scripts](#) – `ENTRY, MEMORY, SECTIONS, KEEP, ALIGN, LOADADDR, AT>`.
- [5] Roteiro original “Laboratório 02 – Piscar LED (Parte 1)”, de Daniel P. Carvalho e Marcelo Barros de Almeida (UFU) – esta prática adapta a progressão pedagógica e o código-fonte desse roteiro.